

Wathiq – Intelligent Digital Archiving System

Arabic Text Genre Classification and Sentence Segmentation Extension

Senior 2 Project – Completed the requirements for obtaining a bachelor's degree
in Informatics Engineering – Software Engineering and Information Systems

Prepared by

Salam Almasri

Najat Bostaty

Bushra Alshaabani

Supervised By

Dr. Riad Sonbol

Eng. Raghad Alhossny

Table of Contents

1	Introduction and Research Motivation	1
1-1	Project Background	2
1-2	From Document Digitization to Document Understanding	2
1-3	Problem Statement	3
1-4	Preliminary Error Analysis and Research Motivation	3
1-4-1	Summary of the Previous OCR Evaluation	4
1-4-2	Character-Level and Word-Level Errors	4
1-4-3	Numeric and Punctuation Errors	4
1-4-4	Relationship to AraGenre and AraSeg	4
1-5	Research Objectives	5
1-5-1	Main Objective	5
1-5-2	Specific Objectives	5
1-6	Research Questions	5
1-7	Software Engineering Perspective	6
1-7-1	Relationship to the Existing Wathiq System	6
1-7-2	Intelligent Service Integration	6
1-7-3	Human-in-the-Loop Review	7
1-7-4	Model Traceability and Versioning	7
1-8	Project Scope	7
1-9	Constraints and Assumptions	7
1-9-1	Constraints	7
1-9-2	Assumptions	8
1-10	Research Methodology	8
1-11	Expected Contributions	9
1-12	Report Organization	9
2	Software Engineering Design of the Research Extension	10
2-1	Relationship to the Previous Wathiq System	11
2-1-1	Existing Wathiq Architecture	11
2-1-2	Previously Implemented Document Workflow	11
2-1-3	Scope of the Current Extension	11

2-2	Motivation for Extending the System	11
2-2-1	Limitations of OCR-Only Processing	11
2-2-2	Need for Structural Text Understanding	11
2-2-3	Need for Semantic Text Classification	12
2-3	Change Impact Analysis	12
2-3-1	Affected System Components	12
2-3-2	New Intelligent Processing Components	12
2-3-3	Changes to the Document Processing Workflow	12
2-3-4	Changes to Document Metadata	12
2-3-5	Changes to the User Review Process	12
2-4	Updated Intelligent Processing Pipeline	12
2-4-1	Document Upload and Validation	13
2-4-2	OCR Text Extraction	13
2-4-3	AraSeg Processing Stage	13
2-4-4	AraGenre Processing Stage	13
2-4-5	Human Review and Correction	13
2-4-6	Metadata Storage and Search Indexing	13
2-5	Updated Software Architecture	14
2-5-1	Architectural Overview	14
2-5-2	OCR Microservice	14
2-5-3	AraSeg Service	14
2-5-4	AraGenre Service	14
2-5-5	Backend Orchestration	14
2-5-6	Database and Search Integration	14
2-5-7	Frontend Review Interface	15
2-6	Service Interface Design	15
2-6-1	Common Request Structure	15
2-6-2	AraSeg Input and Output Contract	15
2-6-3	AraGenre Input and Output Contract	15
2-6-4	Confidence and Model Metadata	16
2-6-5	Error Response Structure	16
2-7	Updated System Requirements	16
2-7-1	Functional Requirements	16
2-7-2	Non-Functional Requirements	17
2-8	Human-in-the-Loop Design	18
2-8-1	Review of OCR-Extracted Text	18
2-8-2	Review of Sentence Boundaries	19
2-8-3	Review of Genre Predictions	19

2-8-4	Low-Confidence Predictions	19
2-8-5	Manual Correction and Final Approval	19
2-9	Model Lifecycle and Experiment Traceability	19
2-9-1	Model Versioning	19
2-9-2	Experiment Configuration	20
2-9-3	Prediction Logging	20
2-9-4	Model Replacement	20
2-9-5	Document Reprocessing	20
2-10	Reliability and Failure Handling	20
2-10-1	OCR Service Failure	20
2-10-2	AraSeg Service Failure	20
2-10-3	AraGenre Service Failure	21
2-10-4	Timeout and Invalid Response Handling	21
2-10-5	Fallback Processing	21
2-10-6	Document Preservation	21
2-11	Testing and Quality Assurance Strategy	21
2-11-1	Unit Testing	21
2-11-2	Service Integration Testing	22
2-11-3	End-to-End Pipeline Testing	22
2-11-4	Model Evaluation Testing	22
2-11-5	Regression Testing	22
2-11-6	Human Review Validation	22
2-12	Deployment and Computational Considerations	22
2-12-1	Independent Service Deployment	22
2-12-2	Hardware and Memory Requirements	23
2-12-3	Batch and Real-Time Processing	23
2-12-4	External Model Dependencies	23
2-12-5	Future Deployment Options	23
2-13	Risks and Mitigation Strategies	23
2-13-1	Prediction Errors	23
2-13-2	Model Availability	24
2-13-3	Distribution Shift	24
2-13-4	Computational Cost	24
2-13-5	Sensitive Document Processing	24
2-14	Chapter Summary	25
3	AraGenre: Hierarchical Arabic Text Genre Classification	26
3-1	Introduction and Relationship to Wathiq	27
3-2	AraGenre 2026 Shared Task, Dataset, and Evaluation	27

3-2-1	Task Definition and Hierarchical Label Structure	27
3-2-2	Dataset and Experimental Use	28
3-2-3	Text Characteristics and Classification Challenges	30
3-2-4	Official Evaluation Metrics	31
3-3	Related Work and Research Positioning	32
3-3-1	Automatic Genre Identification	32
3-3-2	Definition-Guided and Retrieval-Augmented Classification . . .	32
3-3-3	Hierarchical Classification and Class Imbalance	33
3-3-4	Comparison of Relevant Research Directions	33
3-3-5	Research Gap and Positioning of the Proposed System	35
3-4	Official Baselines and Reproduction	36
3-4-1	Definition-Based MiniLM Baseline	36
3-4-2	Qwen Retrieval-Augmented Baseline	37
3-4-3	Reproduced Baseline Results	37
3-4-4	Confidence-Based Ensemble	38
3-4-5	Baseline Findings	39
3-5	Development-Phase System	40
3-5-1	MiniLM and Negative Sampling Experiments	41
3-5-2	Transition to Multilingual E5	42
3-5-3	E5 Fine-Tuning	43
3-5-4	Hierarchical Candidate Definitions	44
3-5-5	Selected E5 Development Configuration	45
3-5-6	Recall at K and Error Analysis	46
3-5-7	Qwen Classification and Focused Reranking	47
3-5-8	Official Development Result	48
3-6	Final-Test Generalization Gap	48
3-6-1	First Final-Test Submission	49
3-6-2	Hierarchical Calibration	49
3-6-3	True Two-Stage E5 Classification	50
3-6-4	Corrected Broad-First Hybrid	50
3-6-5	Adaptive Broad E5	51
3-6-6	Progression of the Initial Final-Test Experiments	51
3-6-7	Lessons from the Initial Final-Test Experiments	51
3-7	Arabic Prototype Construction	52
3-7-1	Motivation and Generation Method	52
3-7-2	Broad and Specific Prototype Allocation	53
3-7-3	Quality, Diversity, and Data-Integrity Validation	54
3-7-4	Broad-Pool Imbalance	54

3-8	Retrieval-Based Hierarchical Classification	55
3-8-1	Frozen Multilingual E5 and BM25	55
3-8-2	Inverse-Distance Weighted KNN	56
3-8-3	Hierarchical Pool Expansion and Hard Gating	56
3-8-4	Weighted KNN Results	57
3-8-5	Neighbourhood Size Experiments	57
3-9	Final Class-Balanced Hybrid System	58
3-9-1	Final Classification Pipeline	58
3-9-2	Evidence Channels	59
3-9-3	Class-Balanced Broad Retrieval	60
3-9-4	Broad-Level Score Fusion	60
3-9-5	Specific Retrieval and Weighted RRF	61
3-9-6	Combination of the Selected Broad and Specific Stages	62
3-9-7	Selective Qwen Reranking	62
3-9-8	Implementation Details	63
3-10	Experimental Results and Ablation Analysis	64
3-10-1	Class-Balanced Retrieval Result	64
3-10-2	Weighted RRF and Candidate-Size Experiments	64
3-10-3	Combined Broad and Specific System	65
3-10-4	Effect of Selective Qwen	65
3-10-5	Final Official Result	65
3-11	Error Analysis, Discussion, and Limitations	66
3-11-1	Scope and Restrictions of the Error Analysis	66
3-11-2	Communicative Function and Surface Cues	66
3-11-3	Reranking Errors	67
3-11-4	Broad-to-Specific Error Propagation	67
3-11-5	Specific-Level Classification Difficulty	67
3-11-6	System Limitations	68
3-11-7	Reproducibility Limitations	68
3-11-8	Discussion	69
3-12	Integration with Wathiq	69
3-12-1	Position in the Document Workflow	69
3-12-2	Service Input and Validation	70
3-12-3	Output and Stored Metadata	70
3-12-4	Human Review	71
3-12-5	Search and Document Organization	71
3-12-6	Traceability and Failure Handling	72
3-12-7	Validation Before Operational Use	72

3-13	Chapter Summary	73
4	AraSeg: Arabic Sentence Segmentation	74
4-1	Introduction and Relationship to Wathiq	75
4-2	AraSeg 2026 Shared Task, Dataset, and Evaluation	75
4-2-1	Task Definition and Sentence-Boundary Representation	75
4-2-2	The Four Closed-Track Conditions	76
4-2-3	Dataset and Experimental Use	77
4-2-4	Closed-Track Experimental Protocol	78
4-2-5	Arabic Sentence Boundary Challenges	79
4-2-6	Official Evaluation Metrics	80
4-3	Official Baselines and Initial Reproduction	81
4-3-1	Related Work and Reference Systems	81
4-3-2	Segment Any Text Baseline	83
4-3-3	CAMeLBERT-MSA Trainable Baseline	83
4-3-4	Reproduced NoPnx-NP Baseline Results	84
4-3-5	Initial CAMeLBERT-SaT Probability Ensemble	85
4-3-6	Initial Error-Analysis Findings	86
4-4	Common Modeling and Experimental Methodology	88
4-4-1	Binary Token-Classification Formulation	88
4-4-2	Original-Token and Subtoken Alignment	89
4-4-3	Long-Document Processing	89
4-4-4	Merging Predictions from Overlapping Windows	90
4-4-5	Probability-Level Ensembling	90
4-4-6	Threshold Calibration	91
4-4-7	Track-Specific Structural Rules	91
4-4-8	Development-Only Model Selection	92
4-4-9	Document-Level Held-Out Validation	93
4-4-10	Acceptance and Rejection of Experimental Variants	93
4-4-11	Reproducibility Controls	94
4-5	PA Track Development	95
4-5-1	Track-Specific CAMeLBERT Fine-Tuning	95
4-5-2	Global Threshold Calibration	96
4-5-3	Final-Document Boundary Constraint	96
4-5-4	Punctuation-Aware Dual-Threshold Calibration	97
4-5-5	Low-Weight Two-Model Ensemble	98
4-5-6	Selective LLM-Assisted Boundary Rescue	98
4-6	NoPnx-PA Track Development	99
4-6-1	Explicit Paragraph Representation	99

4-6-2	Paragraph-Aware Boundary Rules	99
4-6-3	Independent Model Diversity	100
4-6-4	Two-Model Probability Ensemble	100
4-6-5	Decision-Threshold Calibration	101
4-6-6	Retrospective Blind-Evaluation Record	101
4-6-7	Meta-Classification Experiment	102
4-6-8	Selective LLM-Assisted Verification	102
4-7	NP Track Development	103
4-7-1	Initial CAMELBERT-MSA Model	103
4-7-2	Historical Strong Blind Result	104
4-7-3	Low-Recall Diagnostic Experiment	104
4-7-4	Document-Adaptive Boundary Calibration	104
4-7-5	Independent Model Diversity	105
4-7-6	Global Three-Model Probability Ensemble	105
4-7-7	Held-Out Comparison of Global and Adaptive Strategies	106
4-7-8	Nested Token-Level Meta-Classification	106
4-7-9	Selective LLM-Assisted Boundary Verification	107
4-8	NoPnx-NP Track Development	107
4-8-1	Direct NoPnx-NP Fine-Tuning	107
4-8-2	Cross-Track Transfer from NP	108
4-8-3	Initial Clean-Transfer Ensemble	108
4-8-4	Severe Initial Blind Recall Collapse	108
4-8-5	Alignment Verification and System Recovery	109
4-8-6	Controlled Two-Model Recovery	109
4-8-7	Diagnostic Analysis before Further Model Development	110
4-8-8	Context-Length Diversity	110
4-8-9	Context-Diverse CAMELBERT Ensemble	110
4-8-10	Held-Out Validation of Context Diversity	111
4-8-11	Blind Evaluation of the Context-Diverse Ensemble	111
4-8-12	Beyond the Context-Diverse CAMELBERT Ensemble	112
4-8-13	AraELECTRA Backbone Diversity	116
4-8-14	Further Evaluation after Backbone-Diverse Selection	117
4-9	Comparative Experimental Results	120
4-9-1	Final Blind-Evaluation Results	120
4-9-2	Effect of Removing Punctuation	121
4-9-3	Effect of Removing Paragraph Information	121
4-9-4	Structural Information and Observed Difficulty	122
4-9-5	Precision-Recall Behavior Across Conditions	122

4-10	AraSeg Error Analysis	122
4-10-1	Missed Sentence Boundaries	123
4-10-2	False Sentence Boundaries	123
4-10-3	Document-Level Effect of the Initial Ensemble	124
4-10-4	Representative Missed-Boundary Recovery	124
4-10-5	Template- and Heading-Driven False Boundaries	125
4-10-6	Punctuation-Related Boundary Difficulty	125
4-10-7	Long Documents and Sliding-Window Processing	126
4-10-8	Boundary Density and SaT Under-Segmentation	126
4-10-9	OCR-Related Error Risks in Wathiq	127
4-10-10	Lexical and Linguistic Error Attribution	127
4-10-11	Limits of the Error Analysis	128
4-11	Integration with Wathiq	128
4-11-1	Production Integration Strategy	128
4-11-2	AraSeg as an Independent AI Service	129
4-11-3	Model-Artifact Validation	129
4-11-4	Communication between the Wathiq Backend and AraSeg	129
4-11-5	Preserving the OCR Processing Pipeline	130
4-11-6	Automatic Segmentation after OCR	130
4-11-7	Persistent Sentence Representation	130
4-11-8	Failure Isolation and Retry	131
4-11-9	Sentence-Aware Elasticsearch Indexing	131
4-11-10	Segmented-Text User Interface	131
4-11-11	Integration Architecture	132
4-11-12	Validation of the Integrated Components	134
4-12	Limitations and Discussion	134
4-12-1	Training-Data Scale	134
4-12-2	Development-Based Model Selection	135
4-12-3	Track-Specific Models Rather than a Universal System	135
4-12-4	Historical Performance and Experimental Reproducibility	135
4-12-5	OCR Dependency in Wathiq	136
4-12-6	Computational Complexity	136
4-12-7	Discussion of the Main Experimental Findings	136
4-13	Chapter Summary	137
5	Conclusions and Future Work	139
5-1	Summary of the Current Work	140
5-2	Main Research Findings	140
5-2-1	AraGenre Findings	140

5-2-2	AraSeg Findings	141
5-3	Contributions to Wathiq	141
5-4	Research and Software Engineering Contributions	141
5-5	Current Limitations	142
5-6	Future Work	142
5-6-1	Evaluation on Wathiq Documents	142
5-6-2	OCR-Aware Processing	142
5-6-3	Confidence-Based Human Review	143
5-6-4	End-to-End Evaluation	143
5-7	Final Conclusion	143
Bibliography		144

Chapter 1

Introduction and Research Motivation

1-1 Project Background

The rapid digital transformation of governmental and private institutions has increased the need for secure and intelligent document management systems. Organizations handle large volumes of official documents that must be stored, processed, searched, and retrieved efficiently. Traditional paper-based archiving and basic digital storage solutions often require considerable manual effort and provide only limited support for understanding document content.

To address these challenges, the **Wathiq** system was developed during the first phase of the graduation project as an intelligent electronic document management and archiving platform. The system supports document upload, storage, text extraction, metadata review, advanced search, reporting, audit logging, and role-based access control within a multi-institution environment.

The original system combines a React and TypeScript frontend, an ASP.NET Core backend, MongoDB-based storage, and an independent Optical Character Recognition microservice. Its OCR pipeline uses a Vision-Language Model as the primary extraction method, with traditional OCR as a fallback mechanism. This architecture provides the software foundation for extending Wathiq beyond document digitization toward deeper Arabic document understanding.

1-2 From Document Digitization to Document Understanding

The first phase of Wathiq focused mainly on converting document images into searchable digital text and integrating the extracted content into a complete document management workflow. Although OCR is an essential component of digital archiving, text extraction alone does not provide a complete understanding of a document.

The extracted text may remain as a long and weakly structured block, especially when sentence-ending punctuation is missing or incorrectly recognized. Moreover, the system may not know the linguistic genre of the extracted content or the relationship between a broad document category and its more specific textual type.

Consequently, two additional natural language processing problems were selected for the current project phase:

- **Arabic Text Genre Classification:** determining the broad and specific genre of an Arabic text.
- **Arabic Sentence Segmentation:** identifying sentence boundaries in Arabic texts, including texts with missing punctuation, complex structures, or OCR-related noise.

These two tasks extend Wathiq from a system that primarily extracts and stores text into a system that can represent both the semantic type and the internal sentence structure of the extracted content.

1-3 Problem Statement

Despite the improvements achieved by the Wathiq OCR pipeline, the extracted text may still contain recognition errors, missing punctuation marks, incorrect numbers, and structural inconsistencies. Such errors affect not only text readability but also the performance of subsequent document understanding tasks.

A document management system that stores OCR output as an unstructured text block faces several limitations. Search results may lack precise context, document classification may depend on manual input, and downstream tasks such as metadata extraction, summarization, and information retrieval may process text that does not contain reliable sentence boundaries.

The problem addressed in the current work can therefore be expressed as follows:

How can Arabic text genre classification and sentence segmentation be integrated into the existing Wathiq document processing pipeline to improve the semantic and structural understanding of OCR-extracted Arabic documents?

The work investigates this problem through the AraGenre and AraSeg shared tasks, which provide standardized datasets, evaluation protocols, and competitive experimental settings for both problems.

1-4 Preliminary Error Analysis and Research Motivation

The evaluation conducted during the previous phase of Wathiq showed that the enhanced OCR pipeline improved the quality of extracted text. On a dataset of 100 governmental documents, the Character Error Rate decreased from approximately 0.62 to 0.42, while the Word Error Rate decreased from approximately 0.68 to 0.48. Text similarity increased from approximately 0.42 to 0.60.

However, the error analysis also showed that several important error types remained. Numeric errors occurred in approximately 61.6% of the evaluated documents, while punctuation-related errors appeared in approximately 46.5%. Small text and low image resolution also had a measurable negative effect on extraction quality.

These observations provide the direct motivation for the two research tasks addressed in this report.

First, punctuation errors make sentence boundary detection based only on periods and other explicit punctuation marks unreliable. Arabic sentence segmentation must therefore consider contextual and linguistic information in addition to surface punctuation.

Second, OCR errors may affect both lexical keywords and semantic representations used for document classification. A robust text genre classification system should therefore be evaluated using approaches that combine complementary semantic and lexical signals instead of depending on a single representation method.

The error analysis consequently acts as a bridge between the OCR work completed during the previous project phase and the AraGenre and AraSeg tasks investigated in the current phase.

1-4-1 Summary of the Previous OCR Evaluation

The previous OCR evaluation used a dataset of 100 governmental documents containing approximately 70% Arabic documents and 30% English documents. The dataset included documents with different layouts, text sizes, image qualities, stamps, tables, and mixed content.

The evaluation compared the initial OCR output with the enhanced pipeline using character-level, word-level, and similarity-based metrics. The results demonstrated an overall improvement, while also revealing persistent errors that affect later natural language processing stages.

1-4-2 Character-Level and Word-Level Errors

Character-level errors included substitutions, deletions, and insertions caused by low image quality, small fonts, overlapping stamps, and complex document layouts. Word-level errors were more significant because a single character error may change an entire word and affect its meaning.

These errors are particularly relevant to AraGenre because lexical retrieval methods may depend on exact words, while semantic models may also be affected when important contextual terms are distorted.

1-4-3 Numeric and Punctuation Errors

Numeric errors were among the most frequent problems observed in the previous OCR evaluation. These errors may affect dates, document numbers, legal references, monetary values, and administrative codes.

Punctuation errors are especially relevant to AraSeg because sentence boundaries are often associated with punctuation marks. Missing or incorrect punctuation makes rule-based segmentation unreliable and increases the importance of contextual modeling.

1-4-4 Relationship to AraGenre and AraSeg

The observed OCR errors show that document understanding should not be studied independently from extraction quality. AraGenre must remain robust when some important words are missing or distorted, while AraSeg must identify sentence boundaries even when punctuation is incomplete.

Therefore, both tasks represent a logical continuation of the previous OCR research and support the transition from document digitization to intelligent Arabic document understanding.

1-5 Research Objectives

The main objective of the current project phase is to extend Wathiq with experimentally evaluated Arabic text understanding capabilities for genre classification and sentence segmentation.

1-5-1 Main Objective

The main objective is to design, evaluate, and document two intelligent Arabic text processing components that can be integrated into the existing Wathiq document processing pipeline.

The first component predicts the broad and specific genre of an Arabic text. The second component identifies sentence boundaries and transforms OCR-extracted text into structured sentence units.

1-5-2 Specific Objectives

The specific objectives are:

1. To study the hierarchical Arabic text genre classification problem defined by the AraGenre shared task.
2. To develop and evaluate a broad-first classification approach for predicting broad and specific Arabic text genres.
3. To investigate semantic, lexical, retrieval-based, and ensemble-oriented methods for improving genre classification.
4. To study Arabic sentence boundary detection using the datasets and evaluation protocol of the AraSeg shared task.
5. To analyze the effect of text length, punctuation, linguistic variation, and OCR-like noise on both tasks.
6. To compare experimental configurations using the official evaluation metrics of each shared task.
7. To define a software integration strategy for adding the selected AraGenre and AraSeg models to the existing Wathiq architecture.
8. To preserve traceability, model versioning, safe failure, and human review when integrating the intelligent components into the document workflow.

1-6 Research Questions

The current work is organized around the following research questions:

1. How effectively can hierarchical Arabic text genre classification be performed using a broad-first strategy?
2. Does combining semantic and lexical retrieval signals improve classification compared with relying on a single signal?

3. How does restricting specific-genre candidates according to the predicted broad genre affect classification performance and error propagation?
4. Which linguistic and structural patterns cause the most frequent Arabic sentence segmentation errors?
5. How do punctuation loss, OCR noise, and long-text processing influence sentence boundary detection?
6. How can AraGenre and AraSeg be integrated into Wathiq as maintainable, testable, and replaceable intelligent services?

1-7 Software Engineering Perspective

AraGenre and AraSeg are not treated only as independent machine learning experiments. They are considered intelligent extensions to an existing software product. Therefore, their integration must follow the same software engineering principles used in the original Wathiq system.

Each model should be exposed through a clearly defined service interface with explicit input and output structures. The system should record the model version, experimental configuration, confidence score, and prediction timestamp. This information is necessary for traceability, reproducibility, model replacement, and future re-evaluation.

The integration should also support safe failure. If one of the intelligent services becomes unavailable, the original document and OCR output must remain accessible, and the document upload operation should not result in data loss. Low-confidence outputs should be presented to an authorized user for review rather than being accepted automatically.

From an architectural perspective, the two tasks can be integrated as independent processing stages after OCR. AraSeg produces a structured sentence representation, while AraGenre produces broad and specific genre metadata. Both outputs can then support search, indexing, metadata extraction, document review, and future summarization functions.

1-7-1 Relationship to the Existing Wathiq System

The current work extends the existing Wathiq architecture rather than replacing it. The frontend, backend, database, access-control system, document storage, and OCR microservice remain the main software foundation.

The new services operate on the text extracted by the OCR component and return structured outputs that can be stored with the document metadata or displayed for human review.

1-7-2 Intelligent Service Integration

AraGenre and AraSeg should be implemented as independent intelligent services. This design reduces coupling between the machine learning models and the main backend and allows each model to be tested, updated, or replaced independently.

The backend is responsible for coordinating the document processing workflow, sending the extracted text to the services, validating the responses, storing the results, and applying fallback behavior when a service is unavailable.

1-7-3 Human-in-the-Loop Review

Automatic predictions should not always be accepted without review, especially for official or sensitive documents. The system should display the predicted genre and the segmented text before final approval when appropriate.

Human corrections can also provide valuable evidence for future evaluation and model improvement.

1-7-4 Model Traceability and Versioning

Each stored prediction should be associated with the model version, configuration identifier, prediction timestamp, and confidence score. This enables the system to determine which model produced each result and supports later comparison or reprocessing.

1-8 Project Scope

The scope of the current report includes:

- Hierarchical classification of Arabic texts into broad and specific genres using the AraGenre task definition.
- Arabic sentence boundary detection using the AraSeg task definition.
- Experimental comparison of baseline and improved methods.
- Quantitative evaluation using official shared-task metrics.
- Qualitative analysis of representative model errors.
- Software engineering analysis of the integration of both tasks into Wathiq.

The report does not repeat the complete requirements, architecture, implementation, and testing documentation of the original Wathiq system because these aspects were documented during the previous project phase. They are discussed only when they are affected by the new intelligent components.

1-9 Constraints and Assumptions

1-9-1 Constraints

The current work is subject to the following constraints:

- Limited computational resources for training and evaluating large language models.
- Significant class imbalance within the AraGenre dataset.
- A large number of specific genres with semantic overlap.

- Considerable variation in text length, style, dialect, and noise.
- Possible error propagation from broad-genre prediction to specific-genre prediction.
- Long-text processing limitations imposed by model context windows.
- Limited direct evaluation on governmental documents from the Wathiq domain.

1-9-2 Assumptions

The following assumptions are adopted:

- The official shared-task datasets and labels are used according to their permitted experimental protocols.
- Development data are used for model selection and calibration, while test data remain excluded from training.
- The final models will be integrated as replaceable services rather than tightly coupled components.
- Human review remains available for low-confidence predictions and critical institutional documents.

1-10 Research Methodology

The current work follows an applied experimental methodology supported by software engineering principles. The research process is organized into the following stages:

1. Reviewing the task definitions, datasets, genre definitions, official baselines, and evaluation protocols.
2. Implementing or reproducing a baseline for each task.
3. Evaluating the baseline and identifying its dominant error patterns.
4. Designing task-specific improvements based on the observed errors.
5. Conducting controlled experiments in which configurations are compared using the same data splits and official metrics.
6. Performing quantitative and qualitative error analysis.
7. Selecting the most suitable approach according to accuracy, robustness, computational requirements, and integration feasibility.
8. Defining the software interfaces and integration points required to add both capabilities to Wathiq.

This methodology separates model development from final test evaluation and emphasizes reproducibility by documenting model versions, parameters, data preparation, and evaluation procedures.

1-11 Expected Contributions

The main expected contributions of the current project phase are:

- A hierarchical broad-first approach for Arabic text genre classification.
- An experimental analysis of semantic and lexical signals in AraGenre classification.
- An Arabic sentence segmentation pipeline designed to handle complex contextual boundaries.
- A detailed error analysis connecting OCR limitations with downstream Arabic text processing tasks.
- A software engineering design for integrating classification and sentence segmentation into Wathiq.
- A foundation for improving document indexing, search, metadata extraction, and future summarization capabilities.

1-12 Report Organization

The remainder of this report is organized into four chapters. Chapter 2 presents the software engineering context and the analysis that motivated the current research extension. Chapter 3 describes the AraGenre task and the complete experimental development of the hierarchical Arabic genre classification system. Chapter 4 presents the AraSeg task and the proposed Arabic sentence segmentation approach. Finally, Chapter 5 summarizes the main findings, limitations, and directions for future work.

Chapter 2

Software Engineering Design of the Research Extension

2-1 Relationship to the Previous Wathiq System

2-1-1 Existing Wathiq Architecture

The baseline architecture of the Wathiq system consists of four primary layers:

- **Frontend:** Built using React with TypeScript to deliver an interactive web interface for document review and editing.
- **Backend:** Developed with ASP.NET Core to manage business logic, system orchestration, user authentication, and role-based access control.
- **Database & Storage:** Utilizes MongoDB for managing flexible, unstructured document metadata and binary file storage.
- **OCR Microservice:** Driven by a Vision-Language Model (VLM) as the primary Arabic text extraction engine, backed by a traditional OCR fallback mechanism for resiliency.

2-1-2 Previously Implemented Document Workflow

The original document pipeline followed a strict linear processing sequence:

Upload Document → Store File → Execute OCR → Save Raw Text → Index for Search

2-1-3 Scope of the Current Extension

The current work extends Wathiq from basic **Document Digitization** to comprehensive **Structural & Semantic Document Understanding**. This is achieved by embedding two intelligent processing modules immediately after the OCR stage: sentence boundary segmentation (**AraSeg**) and hierarchical genre classification (**AraGenre**).

2-2 Motivation for Extending the System

2-2-1 Limitations of OCR-Only Processing

OCR outputs consist of flat, unstructured text blocks that lack sentence boundaries and proper punctuation. Furthermore, baseline OCR exhibits noticeable error rates, including a Word Error Rate (WER ≈ 0.48), Character Error Rate (CER ≈ 0.42), punctuation errors ($\approx 46.5\%$), and numerical errors ($\approx 61.6\%$).

2-2-2 Need for Structural Text Understanding

Explicit sentence boundary segmentation is essential for downstream text processing tasks, such as passage-level information retrieval, legal and administrative metadata extraction, and automatic text summarization.

2-2-3 Need for Semantic Text Classification

Keyword-based search fails to distinguish functional document types (e.g., differentiating an *analytical report* from an *educational explanation*). Implementing hierarchical genre classification enables categorization into broad and fine-grained genres, significantly enhancing indexing and document retrieval

2-3 Change Impact Analysis

2-3-1 Affected System Components

The extension impacts the backend workflow orchestrator, the MongoDB metadata schemas, and the frontend user interface designed for document verification.

2-3-2 New Intelligent Processing Components

Two decoupled microservices are introduced into the system architecture: the AraSeg Service and the AraGenre Service.

2-3-3 Changes to the Document Processing Workflow

The linear workflow is updated so that extracted OCR text passes through the AraSeg Service for sentence segmentation, followed by the AraGenre Service for hierarchical genre prediction before final metadata commitment.

2-3-4 Changes to Document Metadata

The MongoDB schema is expanded to store broad and specific genre predictions, model confidence scores, model version numbers, and arrays of character offset boundaries for segmented sentences.

2-3-5 Changes to the User Review Process

A Human-in-the-Loop (HITL) review process is introduced. Predictions from AraSeg or AraGenre that fall below a pre-configured confidence threshold (τ) are routed to a human review interface for manual verification and editing.

2-4 Updated Intelligent Processing Pipeline

To transition Wathiq from a basic document digitization system into a platform for structural and semantic document understanding, the core processing pipeline has been expanded into a six-stage intelligent workflow.

2-4-1 Document Upload and Validation

The document pipeline begins when a user uploads a digital document (such as a PDF or image file) through the React-based frontend interface. The system performs validation checks to ensure file integrity, verify supported MIME types, and enforce maximum file size limits before creating an initial document metadata entry in MongoDB.

2-4-2 OCR Text Extraction

The validated document is submitted to the OCR Microservice. Text extraction is primarily handled by a fine-tuned Vision-Language Model (VLM) engine. To maintain system reliability against unexpected VLM processing errors or unsupported document formats, the microservice incorporates an automatic fallback to a traditional OCR engine. This stage produces raw Arabic text extracted directly from the uploaded document.

2-4-3 AraSeg Processing Stage

The raw, unstructured OCR text is passed to the AraSeg Service. This stage addresses the issue of missing punctuation and unstructured text blocks by identifying explicit sentence boundaries. The module tokenizes the text and returns structured sentence units along with their precise character offset positions within the full text.

2-4-4 AraGenre Processing Stage

The segmented text structure is subsequently routed to the AraGenre Service for semantic classification. The service employs a broad-first hierarchical classification strategy. It predicts both a primary Broad Genre (out of 6 main classes) and a secondary Specific Genre (out of 74 fine-grained classes), accompanied by calibrated model confidence scores ($\text{Confidence}_{\text{broad}}$ and $\text{Confidence}_{\text{specific}}$).

2-4-5 Human Review and Correction

To ensure accuracy in downstream administrative workflows, the system incorporates a Human-in-the-Loop (HITL) review stage. If the confidence scores from AraSeg or AraGenre fall below a pre-defined threshold (τ), or if severe OCR corruption is flagged, the document is automatically flagged and placed into a human review queue. A reviewer can inspect, verify, or manually override the model's predictions via the dedicated frontend interface.

2-4-6 Metadata Storage and Search Indexing

Once the pipeline execution is complete—either via automatic high-confidence prediction or manual human validation—the final output is persisted. The broad and specific genre tags, sentence boundary offsets, model confidence scores, and version identifiers are saved in the

MongoDB database document schema. Simultaneously, the structured sentences and metadata are indexed in the search engine to enable precise, passage-level information retrieval.

2-5 Updated Software Architecture

The extended architecture follows a modular, microservice-based design to decouple heavy machine learning computation from core backend administrative workflows.

2-5-1 Architectural Overview

The software architecture separates core business logic from intelligent NLP modules. The system features an ASP.NET Core backend orchestrator, a React/TypeScript frontend, a MongoDB datastore, and three independent Python-based microservices (OCR, AraSeg, and AraGenre) communicating via lightweight REST/gRPC interfaces.

2-5-2 OCR Microservice

The OCR service runs as an isolated containerized service. It exposes endpoints for document layout analysis and text extraction, encapsulating the dual VLM-primary and traditional-OCR fallback pipelines to ensure service availability.

2-5-3 AraSeg Service

The AraSeg Service is a Python FastAPI/gRPC microservice hosting fine-tuned sentence boundary segmentation models (e.g., CAMELBERT-based variants). It receives raw text strings and outputs structured JSON payloads containing sentence arrays and character span mappings.

2-5-4 AraGenre Service

The AraGenre Service is an independent machine learning microservice hosting dense text embedders (such as fine-tuned multilingual-e5-base) alongside hybrid BM25 and synthetic prototype retrieval pipelines. It accepts segmented texts and returns hierarchical genre labels with confidence scores.

2-5-5 Backend Orchestration

The ASP.NET Core backend acts as the central workflow orchestrator. It manages client requests, coordinates async calls to the ML microservices, handles retry mechanisms, logs model traceability metadata, and manages user permission control.

2-5-6 Database and Search Integration

MongoDB serves as the central data store, configured with dynamic schemas to hold sentence boundary mappings and hierarchical classification metadata. The database integration layer

pushes structured text units and genre tags to the search engine to facilitate multi-faceted search and document filtering.

2-5-7 Frontend Review Interface

The React/TypeScript frontend provides dedicated views for document processing management. It includes visual highlighted overlays for sentence boundary verification, interactive dropdowns for genre label corrections, and status flags indicating whether a document was auto-approved or human-verified.

2-6 Service Interface Design

To achieve tight integration, seamless interoperability, and loose coupling between the ASP.NET Core backend orchestrator and the independent Python ML microservices, a standardized REST/JSON contract is established across all internal endpoints.

2-6-1 Common Request Structure

All microservice request payloads strictly adhere to a standardized envelope structure to ensure traceable execution, client identification, and consistent parameter propagation. Every inbound API request includes a global correlation tracking ID, a document payload identifier, and execution configuration options.

2-6-2 AraSeg Input and Output Contract

The AraSeg Service exposes endpoints dedicated to sentence boundary detection.

- **Input Payload:** Accepts the raw text string extracted by the OCR pipeline alongside optional tokenization parameters.
- **Output Payload:** Returns a structured array of segmented sentences, where each sentence object includes the extracted text segment, sequence index, and precise start and end character offset spans mapping back to the original full text.

2-6-3 AraGenre Input and Output Contract

The AraGenre Service provides semantic classification interfaces.

- **Input Payload:** Accepts either raw text or the pre-segmented sentence array produced by the AraSeg Service.
- **Output Payload:** Returns predicted classification categories structured hierarchically, comprising the predicted Broad Genre (from 6 classes) and Specific Genre (from 74 classes).

2-6-4 Confidence and Model Metadata

To support automated quality filtering and human-in-the-loop auditing, all service response payloads embed detailed inference metadata. This includes explicit confidence score float metrics, model artifact version identifiers, and internal inference execution latency timestamps.

2-6-5 Error Response Structure

In cases of processing failure (such as invalid input encoding, text length violations, or internal model exceptions), microservices return a standardized RFC-compliant error schema. The response contains a machine-readable error code, a detailed diagnostic message, the correlation ID, and fallback status flags to guide backend orchestration.

2-7 Updated System Requirements

The introduction of intelligent NLP capabilities imposes explicit functional requirements on the extended Wathiq platform.

2-7-1 Functional Requirements

The system must satisfy automated boundary extraction and semantic categorization requirements to guarantee operational correctness.

2-7-1-1 Sentence Segmentation Requirements

- **Boundary Identification:** The system shall automatically identify sentence boundaries in Arabic text extracted via OCR.
- **Offset Mapping:** The segmentation engine shall generate character-level span boundaries (start and end offsets) for every segmented sentence.
- **Noise Resilience:** The segmentation pipeline shall remain robust against common OCR artifacts, including missing whitespace and corrupted punctuation.

2-7-1-2 Genre Classification Requirements

- **Hierarchical Prediction:** The system shall classify input documents into a two-level hierarchy, assigning exactly one Broad Genre (out of 6 classes) and one Specific Genre (out of 74 classes).
- **Confidence Estimation:** The classification engine shall output calibrated numeric confidence probabilities for both broad and specific predictions.
- **Low-Confidence Routing:** The platform shall automatically flag document predictions falling below a configurable confidence threshold and route them to the human review interface.

2-7-1-3 Human Review Requirements

- **Confidence-Based Flagging:** The system shall automatically route processing outputs to the human review queue whenever AraSeg or AraGenre confidence scores fall below the configurable threshold τ .
- **Interactive Correction Interface:** The frontend shall provide a dedicated human-in-the-loop review interface allowing authorized users to inspect OCR text, modify sentence boundary offsets, and manually override broad or specific genre classifications.
- **Feedback Persistence:** Manual corrections made during human review shall be saved back to MongoDB as ground-truth overrides, alongside reviewer identity and timestamp metadata.

2-7-1-4 Metadata and Search Requirements

- **Structured Metadata Storage:** The database schema shall persist broad genre tags, specific genre tags, sentence offset mappings, model version numbers, and confidence metrics within document records.
- **Multi-Faceted Search Indexing:** The system shall push structured sentence units and genre metadata to the search engine to enable passage-level granular retrieval and faceted filtering across document types.

2-7-2 Non-Functional Requirements

The expanded architecture must fulfill stringent operational quality standards to maintain enterprise readiness and stability.

2-7-2-1 Performance

- **Inference Latency:** The combined execution time for both AraSeg and AraGenre microservices shall not exceed 2 seconds per document under standard processing workloads.
- **Throughput Capacity:** The NLP microservice infrastructure shall support concurrent document inference requests without degrading overall system responsiveness.

2-7-2-2 Reliability

- **Fault Tolerance & Graceful Degradation:** If an ML microservice fails or encounters an exception, the backend orchestrator shall fall back gracefully to store raw OCR text and maintain primary search indexing without aborting the core workflow.
- **Retry Mechanisms:** The orchestration layer shall implement automatic retry policies with backoff logic for transient API communication errors.

2-7-2-3 Maintainability

- **Microservice Decoupling:** The AraSeg and AraGenre services shall be implemented as standalone Python microservices (using FastAPI/gRPC), fully isolating ML dependencies from the C# ASP.NET Core backend.
- **Modular Model Swap:** Model checkpoints (such as CAMELBERT or multilingual-e5-base) must be easily updated or swapped without requiring codebase modifications in the core backend.

2-7-2-4 Scalability

- **Independent Container Scaling:** The architecture shall support independent horizontal scaling of individual microservices via Docker containerization based on processing load.

2-7-2-5 Security and Privacy

- **Role-Based Access Control (RBAC):** The system shall restrict access to administrative functions and human review interfaces based on user role permissions.
- **Secure Microservice Communication:** Internal communication between backend services and ML microservices shall be isolated within a secure internal network environment.

2-7-2-6 Traceability and Reproducibility

- **Model Version Tracking:** Every prediction stored in the database shall include metadata identifying the exact model artifact version, execution timestamps, and correlation IDs for complete auditability and experimental reproducibility.

2-8 Human-in-the-Loop Design

To balance system automation with absolute data accuracy in real-world document management, Wathiq incorporates a Human-in-the-Loop (HITL) review workflow. This design ensures that machine learning predictions can be inspected, verified, and manually overridden by human domain experts prior to final metadata persistence and indexing.

2-8-1 Review of OCR-Extracted Text

The initial stage of the HITL interface allows authorized users to inspect the raw Arabic text extracted by the primary VLM or fallback OCR engine. Reviewers can quickly identify and edit typical OCR recognition errors—such as misrecognized characters, numerical inaccuracies, or missing whitespaces—directly alongside the original scanned document preview.

2-8-2 Review of Sentence Boundaries

The interface provides interactive, visual overlays of sentence boundary predictions generated by the AraSeg Service. Users can view the text broken down into discrete sentence units, adjust incorrect sentence split markers, merge over-segmented fragments, or manually add missing boundaries to ensure exact offset span alignment.

2-8-3 Review of Genre Predictions

The human review dashboard displays the broad and specific genre labels assigned by the AraGenre Service. Reviewers are presented with the top predicted categories along with their respective confidence scores, enabling rapid verification of whether the document’s semantic classification matches its functional domain.

2-8-4 Low-Confidence Predictions

The workflow implements automated routing rules driven by prediction confidence scores. Whenever the confidence metric for either broad or specific genre prediction falls below a defined operational threshold (τ), or when severe text corruption is detected, the system automatically flags the document and routes it into a prioritized human review queue.

2-8-5 Manual Correction and Final Approval

Through the review interface, users can override low-confidence predictions by selecting the correct broad or specific genre from structured dropdown menus, as well as refining sentence segmentation boundaries. Once verified, the user submits final approval, which commits the updated metadata to MongoDB, updates the search index, and records the human feedback for future model retraining.

2-9 Model Lifecycle and Experiment Traceability

To maintain rigorous scientific standards and operational maintainability, the system establishes a comprehensive lifecycle framework for tracking, versioning, and deploying machine learning models across AraSeg and AraGenre tasks.

2-9-1 Model Versioning

All deployed model artifacts (including fine-tuned CAMELBERT and multilingual-e5-base checkpoints) are assigned unique version identifiers. Every prediction stored in MongoDB records the precise model version used during inference to ensure complete auditable lineage and reproducibility.

2-9-2 Experiment Configuration

Model execution and hyperparameter configurations—such as similarity loss functions, negative sampling ratios, candidate prototype sets, and classification decision thresholds (τ)—are stored as externalized configuration files, enabling systematic tracking of experimental setups without altering the core codebase.

2-9-3 Prediction Logging

The orchestration layer logs comprehensive inference metadata for every processed document. Logs include raw prediction vectors, top predicted broad and specific genres, character-level sentence offset arrays, confidence scores, and total processing latency timestamps.

2-9-4 Model Replacement

The microservice architecture isolates ML execution from the ASP.NET Core backend. This design allows new model checkpoints to be deployed or swapped within containerized environments without causing system downtime or requiring modifications to the primary business logic.

2-9-5 Document Reprocessing

The system provides backend administration utilities to trigger bulk document reprocessing. When an updated model checkpoint is deployed, historical documents can be re-evaluated to update sentence boundaries, genre predictions, and search indices automatically.

2-10 Reliability and Failure Handling

System architecture incorporates robust failure handling strategies to guarantee service availability and uninterrupted document indexing during unexpected service disruptions.

2-10-1 OCR Service Failure

If the primary Vision-Language Model (VLM) engine encounters an operational error, time-out, or unreadable document layout during text extraction, the OCR microservice automatically triggers a fallback mechanism to a traditional OCR engine. If both engines fail, the system flags the document status as unprocessed and notifies the administration queue while preserving system stability.

2-10-2 AraSeg Service Failure

If the AraSeg Service becomes unavailable or throws an exception during sentence segmentation, the backend orchestrator intercepts the error without crashing the pipeline. The raw text

extracted by the OCR module is saved as a single unsegmented text block, allowing indexing to proceed while marking the sentence segmentation status as pending or unverified.

2-10-3 AraGenre Service Failure

In the event of an AraGenre Service failure, the document metadata pipeline completes the sentence segmentation stage but bypasses automatic genre labeling. Default placeholders (e.g., Unclassified) are assigned, and the document is automatically flagged for review or queued for background re-classification upon service restoration.

2-10-4 Timeout and Invalid Response Handling

To prevent resource blocking, the backend orchestrator enforces strict timeout constraints on all microservice HTTP/gRPC requests. Requests exceeding the defined latency window, or responses failing JSON schema validation, are treated as service exceptions and handled through standardized retry and fallback routines.

2-10-5 Fallback Processing

The orchestrator incorporates a multi-tiered fallback strategy:

1. **OCR Fallback:** Transitions from primary VLM extraction to traditional OCR if VLM execution fails.
2. **NLP Fallback:** Degrades gracefully to raw text indexing if AraSeg or AraGenre microservices are unreachable.
3. **Human Review Fallback:** Automatically routes low-confidence or corrupted outputs to human operators for manual validation.

2-10-6 Document Preservation

System failures never result in data loss. Raw binary files, extracted OCR text, and partial processing states are committed to MongoDB before invoking subsequent NLP microservices, guaranteeing that document content remains preserved regardless of downstream processing errors.

2-11 Testing and Quality Assurance Strategy

A comprehensive multi-level testing matrix ensures the software architecture, microservice contracts, and machine learning models meet rigorous quality and performance standards.

2-11-1 Unit Testing

Unit tests validate isolated core components, including ASP.NET Core controller logic, MongoDB schema mappers, data validation functions, and individual text pre-processing and tokenization utilities within the Python services.

2-11-2 Service Integration Testing

Integration testing verifies API contracts between the backend orchestrator and microservices. Mock payloads validate HTTP status codes, JSON response structure parsing, timeout behaviors, and error payload compliance under simulated service failures.

2-11-3 End-to-End Pipeline Testing

End-to-end tests evaluate the complete workflow—from initial file upload, VLM/OCR text extraction, sentence segmentation via AraSeg, hierarchical classification via AraGenre, through to final database persistence and search indexing.

2-11-4 Model Evaluation Testing

Model quality is continuously monitored against evaluation datasets. AraSeg is evaluated using exact boundary matches and character offset precision, while AraGenre performance is measured using Hierarchical Macro F1 (H-MF1), Broad Macro F1, and Specific Macro F1 metrics.

2-11-5 Regression Testing

Automated regression test suites run prior to deploying new microservice container builds or updated model weights to verify that new releases do not degrade historical classification accuracy or break existing schema representations.

2-11-6 Human Review Validation

Testing routines evaluate the Human-in-the-Loop review workflow, verifying that manual overrides submitted via the frontend correctly update MongoDB metadata records, refresh search indices, and remove documents from the review queue.

2-12 Deployment and Computational Considerations

The extended system is designed for flexible deployment, accounting for memory footprints, GPU/CPU hardware availability, and microservice isolation.

2-12-1 Independent Service Deployment

All processing services (ASP.NET Core Backend, React Frontend, OCR Service, AraSeg Service, and AraGenre Service) are containerized using Docker. This enables independent scaling, isolated dependency management, and seamless deployment across cloud or on-premise environments.

2-12-2 Hardware and Memory Requirements

- **Core System:** Requires standard multi-core CPU instances with 8GB-16GB RAM for backend orchestration and database hosting.
- **ML Microservices:** AraSeg and AraGenre benefit from dedicated NVIDIA GPU acceleration (e.g., CUDA-enabled runtime) for low-latency transformer inference, or multi-threaded CPU environments with optimized ONNX/TensorRT runtimes.

2-12-3 Batch and Real-Time Processing

The system supports dual execution modes:

- **Real-Time Processing:** Synchronous pipeline processing for single document uploads via the web UI.
- **Batch Processing:** Asynchronous queue-based processing (using background workers) for bulk administrative document ingestions.

2-12-4 External Model Dependencies

Machine learning pipelines depend on pre-trained transformer checkpoints (e.g., CAMeL-BERT and multilingual-e5-base). Model weights are cached locally within container volumes to prevent external network dependencies during production runtime.

2-12-5 Future Deployment Options

Future infrastructure upgrades include deploying microservices on Kubernetes (K8s) for dynamic auto-scaling, incorporating serverless GPU endpoints for peak ingestion loads, and converting model artifacts to ONNX format to minimize memory consumption.

2-13 Risks and Mitigation Strategies

The integration of machine learning microservices into an enterprise document processing system introduces specific operational, technical, and domain-related risks. To maintain operational stability, data integrity, and prediction accuracy, potential risks have been systematically identified and paired with actionable mitigation strategies.

2-13-1 Prediction Errors

- **Identified Risk:** Machine learning models may produce inaccurate sentence boundary segmentations or incorrect broad and specific genre classifications, potentially propagating noise into search indices and automated downstream workflows.
- **Mitigation Strategy:** The architecture implements automated confidence thresholding (τ). Predictions with confidence scores falling below this pre-defined operational threshold

are automatically flagged and routed to the Human-in-the-Loop review queue, ensuring manual verification before final persistence and indexing.

2-13-2 Model Availability

- **Identified Risk:** Unavailability, crash events, or transient network timeouts within the AraSeg or AraGenre microservices could disrupt document ingestion and block the primary document pipeline.
- **Mitigation Strategy:** The ASP.NET Core backend orchestrator enforces fault tolerance through graceful degradation. If an NLP microservice fails, raw OCR text is persisted to preserve primary document search capability, while the document is queued for background re-processing once service health is restored.

2-13-3 Distribution Shift

- **Identified Risk:** A drop in segmentation or classification performance may occur when processing out-of-domain administrative records, highly corrupted OCR outputs, or newly introduced document genres not well-represented during initial model training.
- **Mitigation Strategy:** Dense embeddings (such as fine-tuned multilingual-e5-base) are combined with BM25 lexical matching and synthetic Arabic prototype representations to enhance retrieval robustness. Furthermore, corrections submitted during human review are collected to periodically retrain and adapt model weights.

2-13-4 Computational Cost

- **Identified Risk:** High computational latency and heavy memory footprints associated with transformer-based deep learning models could lead to resource exhaustion or high infrastructure costs during large batch document ingestions.
- **Mitigation Strategy:** Decoupled microservice containers allow for targeted hardware allocation and independent horizontal scaling. Model weights are cached locally within container volumes to avoid redundant network overhead, and asynchronous background worker queues handle bulk administrative workloads.

2-13-5 Sensitive Document Processing

- **Identified Risk:** Unintended exposure or unauthorized modification of confidential digitized documents and sensitive metadata during microservice inference or human review workflows.
- **Mitigation Strategy:** Strict Role-Based Access Control (RBAC) governs access to administrative views and human correction dashboards. Internal REST/gRPC communication between the orchestrator and NLP microservices is fully isolated within a secure, private network environment.

2-14 Chapter Summary

This chapter detailed the software engineering design required to extend the Wathiq platform into an intelligent document understanding system. By integrating decoupled microservices for sentence segmentation (AraSeg) and hierarchical genre classification (AraGenre), updating backend orchestration workflows, introducing a Human-in-the-Loop review interface, and establishing fault-tolerant architecture with comprehensive QA strategies, the platform achieves robust, high-accuracy semantic indexing while preserving system reliability.

Chapter 3

AraGenre: Hierarchical Arabic Text Genre Classification

3-1 Introduction and Relationship to Wathiq

The previous phase of the Wathiq project focused on transforming document images into digital content that could be stored and searched. This was achieved by extracting text through Optical Character Recognition (OCR) and associating the extracted content with the corresponding document metadata within an integrated document management and archiving system. Nevertheless, text extraction alone does not provide a sufficient understanding of the nature or communicative function of the extracted content.

Two texts may address the same subject and use similar vocabulary while differing substantially in their communicative purpose, style, and structure. One text may provide an educational explanation, whereas another may present an analytical report or a set of instructions. Therefore, the nature of a text cannot always be determined accurately from its topic or keywords alone.

Based on this limitation, the AraGenre task was selected as a research extension to the OCR stage of Wathiq. The task aims to identify the genre of an Arabic text at two related levels: a broad genre representing its principal functional family and a specific genre describing its communicative function more precisely. In future versions of Wathiq, these predictions may be used to enrich document metadata and support document organization, filtering, and retrieval according to the nature of the content, rather than relying entirely on keywords or manual classification.

In this project, the AraGenre system is developed and evaluated as an independent intelligent component that can subsequently be integrated into the Wathiq processing pipeline after text extraction. Therefore, this chapter first introduces the task, its data, and its official evaluation protocol. It then presents the official baselines, the experimental development process, the obtained results, and the final system. The proposed integration of AraGenre with Wathiq is discussed separately in Section 3-12.

3-2 AraGenre 2026 Shared Task, Dataset, and Evaluation

3-2-1 Task Definition and Hierarchical Label Structure

AraGenre 2026 is a shared task concerned with the hierarchical classification of Arabic text genres using the official genre definitions [1]. For each input text, a participating system is required to assign two related labels:

- one broad genre from a set of six broad genres; and
- one specific genre from a set of 74 specific genres.

The six broad genres defined by the task are:

- **Creative,**
- **Informative,**
- **Interactive,**

- **Learning.**
- **Legal,** and
- **Religious.**

Each specific genre belongs to exactly one broad genre. Consequently, the two predicted labels must remain compatible with the predefined hierarchical structure. For example, if a system assigns a specific genre that belongs to the Informative category, the corresponding broad prediction should also be Informative.

This hierarchical structure makes the task more complex than flat classification. A system may successfully identify the general functional family of a text while confusing semantically related specific genres within that family. Furthermore, when strict hierarchical restriction is applied, an incorrect broad prediction may completely exclude the correct specific genre from the candidate set. Broad-level reliability therefore directly influences specific-level performance.

The task does not depend exclusively on the topic of a text. Instead, it aims to identify the communicative purpose, style, and structure that characterize its genre. Texts belonging to different genres may use similar vocabulary while differing mainly in how the content is presented and in the function it performs. For this reason, the task provides official genre definitions that can be used to represent the intended meaning of each label, rather than relying only on label names or isolated keywords.

3-2-2 Dataset and Experimental Use

The shared task provides labeled training and development data, in addition to a final test set with hidden reference labels. Each data partition was assigned a clearly separated role during system development and evaluation.

Table 3.1 summarizes the files used in this work and the experimental role of each partition.

Table 3.1: AraGenre data resources and their experimental use.

Data Partition	Files	Experimental Use
Training	<code>train.json</code> and <code>train_gold.json</code>	Used to construct the training data and to benefit from the available labeled Arabic texts during system development.
Development	<code>dev.json</code> and <code>dev_gold.json</code>	Used for development-time evaluation, configuration selection, experiment comparison, and example-level error analysis.
Genre definitions	Official genre-definition files	Used to represent the genre labels, construct candidates, and associate the input texts with the official meanings of the genres.
Final test	Official final-test files	Used to generate official predictions and compare a limited number of successive configurations through aggregate Codabench scores. Hidden labels and example-level feedback remained unavailable.

The development set contains 110 labeled texts. Its limited size enabled rapid experimentation, configuration comparison, and example-level error analysis. However, the set remains small relative to the 74 specific genres and cannot be considered a complete representation of all genres, linguistic patterns, and difficult cases present in the task.

In contrast, the final test set contains 27,972 texts and is therefore approximately 254 times larger than the development set. The reference labels of these texts were not available to the project team. Predictions were evaluated by submitting the required output file to the Codabench platform [12] and receiving the official aggregate scores.

The hidden labels of the final test set were never accessed by the project team, and no example-level feedback was available. No final-test prediction was manually corrected, and the final-test texts were not assigned inferred labels for training.

However, aggregate scores returned by Codabench were used to compare a limited number of successive system configurations. The final test therefore functioned as an official leaderboard-based evaluation environment rather than as a completely untouched test set. This distinction is important because repeated comparison of aggregate leaderboard scores may introduce limited adaptation to the evaluation set, even though its reference labels and individual errors remain unknown.

Because the final-test reference labels were unavailable, it was not possible to construct a confusion matrix or conduct example-level error analysis on this partition. The qualitative error analysis presented later in this chapter is therefore based exclusively on the labeled development data.

3-2-3 Text Characteristics and Classification Challenges

The AraGenre data exhibit substantial variation in text length, writing style, linguistic register, dialect, and noise level. An analysis of the final test set produced the approximate text-length distribution shown in Table 3.2.

Table 3.2: Approximate character-length distribution of the AraGenre final test set.

Distribution Statistic	Approximate Text Length in Characters
25th percentile	88
Median	270
75th percentile	662
90th percentile	1,576

Short texts are challenging because they may not provide sufficient context for distinguishing between closely related specific genres. Long texts, in contrast, may contain more than one topic or linguistic pattern and may exceed the maximum input length supported by a model unless part of the content is truncated.

Semantic similarity between genres represents another major difficulty. Motivational writing, for example, may use language similar to that found in advice-oriented texts, while educational explanations may contain formulas and numerical expressions that make them resemble analytical reports. Topic matching and keyword identification are therefore not always sufficient; the system must also distinguish the communicative purpose of the text.

The six broad genres are not equal in their internal diversity. Informative contains 42 specific genres, whereas Legal contains only three. This difference creates a natural imbalance within the hierarchical label structure and may cause classification or aggregation methods based directly on the number of candidates or examples to favor broad categories with larger internal label pools.

Accordingly, the principal challenges of AraGenre can be summarized as follows:

- distinguishing between semantically related genres;
- classifying short or ambiguous texts with limited contextual evidence;
- processing long or multi-topic texts;
- handling variations in writing style, dialect, linguistic register, and noise;
- maintaining hierarchical consistency between broad and specific predictions;
- limiting the influence of hierarchical class imbalance; and
- reducing the propagation of broad-level errors to the specific classification stage.

These challenges indicate that a robust system should not depend unconditionally on a single representation or source of evidence. Nevertheless, the semantic and lexical methods, fusion

mechanisms, and reranking strategies examined in this work are introduced only within the corresponding experimental sections, following the actual sequence of system development.

3-2-4 Official Evaluation Metrics

AraGenre systems are evaluated separately at the broad and specific classification levels. Broad Macro F1 is calculated across the six broad genres, whereas Specific Macro F1 is calculated across the 74 specific genres.

Macro F1 assigns equal importance to every class regardless of the number of examples associated with it. It is therefore particularly appropriate for this task because it reveals weak performance on minority or difficult genres and prevents large classes from dominating the overall evaluation.

The official primary metric used to rank participating systems is the Hierarchical Macro F1-score, denoted by H-MF1. It is computed as follows:

$$\text{H-MF1} = \frac{\text{Broad Macro F1} + \text{Specific Macro F1}}{2}. \quad (3.1)$$

This metric assigns equal weight to broad-level and specific-level performance. Consequently, a substantial weakness in Specific Macro F1 cannot be compensated for solely by achieving a high Broad Macro F1-score, and vice versa.

Several supplementary metrics were also used to analyze the experimental results:

- **Broad Accuracy**, which measures the proportion of texts assigned the correct broad genre;
- **Specific Accuracy**, which measures the proportion of texts assigned the correct specific genre;
- **Broad Weighted F1**, which evaluates broad-level performance while weighting each class according to its number of examples; and
- **Specific Weighted F1**, which evaluates specific-level performance while accounting for differences in class frequency.

Comparing Accuracy and Weighted F1 with Macro F1 helps reveal the effect of class imbalance. If Accuracy or Weighted F1 is substantially higher than Macro F1, the system may be performing well on large or frequent classes while remaining less effective on rare or difficult genres.

For this reason, the results reported in the following sections are presented separately at the broad and specific levels, together with the official hierarchical score. This makes it possible to determine whether each experimental modification improves broad classification, specific classification, or both, and supports a more precise interpretation of error propagation between the two hierarchical levels.

3-3 Related Work and Research Positioning

This section positions the AraGenre task and the proposed system within previous research on automatic genre identification, Arabic and multilingual text classification, definition-guided classification, retrieval-augmented methods, hierarchical classification, and class-imbalance handling. The objective is not to provide an exhaustive survey of text classification, but to identify the research directions that directly influenced the system developed in this chapter.

3-3-1 Automatic Genre Identification

Automatic genre identification differs from conventional topic classification. Topic classification primarily determines what a text discusses, whereas genre classification attempts to identify its communicative purpose, functional role, style, and structural form. Consequently, texts that address similar subjects may belong to different genres when they differ in intention or presentation. Kuzman and Ljubešić identify this distinction as a central challenge in automatic genre identification and emphasize the effects of domain variation, annotation schemes, and textual register [13].

Multilingual shared tasks have further demonstrated that genre classification requires models capable of generalizing across languages, domains, and writing styles. SemEval-2023 Task 3 addressed news genre categorization in a multilingual setting and highlighted the importance of multilingual representations, transformer-based models, and systematic shared-task evaluation [14]. Participating systems also reported benefits from multilingual transformers and ensemble-based classification [15, 16].

Arabic shared tasks have similarly shown that pretrained Arabic and multilingual transformers provide strong foundations for classification problems involving communicative intent and discourse-level phenomena. Nevertheless, Arabic classification continues to be affected by limited labeled data, linguistic variation, dialectal content, and imbalanced categories [17]. These findings are directly relevant to AraGenre because the genre of an Arabic text cannot be determined reliably from isolated keywords or topic similarity alone.

3-3-2 Definition-Guided and Retrieval-Augmented Classification

AraGenre differs from ordinary supervised classification because official natural-language definitions are provided for both the broad and specific genres. These definitions offer an explicit semantic representation of class meaning and can support classification when labeled examples are limited. Research on label-description training has shown that incorporating textual descriptions of labels can improve zero-shot classification and robustness to weakly represented or previously unseen classes [18]. This supports representing AraGenre labels through their definitions rather than relying only on label names or memorized examples.

Retrieval-augmented text classification provides a complementary direction. Instead of producing predictions exclusively from model parameters, a retrieval-based system identifies

semantically related examples or descriptions and uses them as evidence during classification. Yu et al. [19] found that retrieval can improve few-shot text classification, while also showing that performance depends strongly on the quality of the retrieved evidence. Irrelevant neighbours may introduce misleading information, particularly when classes overlap semantically.

This observation is especially important for AraGenre. Retrieved examples can demonstrate how a genre appears in actual text, whereas the official definitions provide a more stable description of class meaning. These two evidence sources are therefore complementary but not interchangeable. This motivates the combination of definition-based similarity, Arabic semantic prototypes, and lexical retrieval examined later in this chapter.

3-3-3 Hierarchical Classification and Class Imbalance

Hierarchical text classification requires predictions at different levels of a predefined taxonomy to remain mutually consistent. In AraGenre, each specific genre belongs to exactly one broad genre; therefore, the specific prediction cannot be treated as independent of the broad decision. Research on hierarchical classification distinguishes between flat prediction, local classification within parts of the hierarchy, and global methods that model the complete taxonomy [20].

A broad-first strategy can reduce the specific candidate set and guarantee hierarchically valid outputs. However, an incorrect broad decision may remove the correct specific genre from consideration. The hierarchy therefore acts both as a useful structural constraint and as a possible source of error propagation. Recent research has similarly shown that hierarchical inference procedures can materially affect the final behavior and evaluation of a classifier [21].

Class imbalance introduces an additional challenge. The six broad AraGenre classes contain substantially different numbers of specific genres and semantic prototypes. If all retrieved prototypes contribute equally, classes with larger retrieval pools may dominate aggregation because they provide more evidence rather than stronger evidence.

Previous research distinguishes class imbalance from class overlap. Class imbalance concerns differences in the number of representatives available for each class, whereas class overlap occurs when examples from different classes are semantically difficult to separate [22, 23]. Both conditions occur in AraGenre and motivate the class-balanced aggregation strategy adopted in the final system.

3-3-4 Comparison of Relevant Research Directions

Table 3.3 summarizes the research directions most relevant to AraGenre and explains how each direction informed the proposed system.

Table 3.3: Comparison of research directions relevant to AraGenre

Study	Research Direction	Main Finding	Relevance to AraGenre
Kuzman and Ljubešić [13]	Automatic genre identification	Genre depends on communicative purpose, textual function, and form rather than topic alone.	Explains why AraGenre requires functional and stylistic evidence in addition to topical similarity.
Piskorski et al. [14]	Multilingual genre classification	Genre classification benefits from multilingual transformer models and systematic cross-language evaluation.	Provides a closely related shared-task setting and highlights the difficulty of generalization across languages and domains.
Hasanain et al. [17]	Arabic text classification	Arabic transformer-based systems are effective, but remain sensitive to linguistic variation, limited data, and imbalanced categories.	Motivates the use of Arabic-aware semantic representations and class-aware processing.
Gao et al. [18]	Label-description classification	Natural-language label descriptions improve zero-shot classification when labeled examples are limited.	Supports representing AraGenre classes through their official definitions instead of relying on label names alone.
Yu et al. [19]	Retrieval-augmented classification	Retrieved examples can improve few-shot classification, but irrelevant retrieval may introduce misleading evidence.	Motivates combining complementary retrieval channels and controlling their contribution to the final prediction.
Silla and Freitas [20]	Hierarchical classification	Hierarchy-aware methods constrain predictions but may propagate errors between classification levels.	Supports broad-first classification while motivating the analysis of broad-to-specific error propagation.

Continued on the next page

Table 3.3 continued from the previous page

Study	Research Direction	Main Finding	Relevance to AraGenre
Japkowicz and Stephen [22]; Prati et al. [23]	Class imbalance and overlap	Unequal class representation and semantic overlap affect classification through different mechanisms.	Motivates class-balanced broad aggregation instead of unrestricted voting over all retrieved prototypes.

3-3-5 Research Gap and Positioning of the Proposed System

The reviewed literature establishes several relevant research directions, including Arabic transformer-based classification, multilingual genre identification, label-description modeling, retrieval augmentation, hierarchical inference, and class-imbalance handling. However, these directions generally address the identified challenges separately. None of the reviewed approaches directly addresses their combination within a definition-guided Arabic taxonomy containing both broad and specific genres.

AraGenre combines Arabic linguistic variation, limited labeled examples, natural-language genre definitions, a two-level hierarchy, semantic overlap between genres, and substantial differences in class representation. The performance of a system must also be balanced across all classes because the official evaluation uses Macro F1 at both the broad and specific levels. Consequently, a method that performs well only on the largest or most distinctive genres remains insufficient.

The system developed in this work addresses this combined setting through a definition-guided and retrieval-based hierarchical architecture. It constructs Arabic semantic prototypes from the official definitions, represents Arabic input using the Arabic Triplet Matryoshka encoder, and combines this evidence with multilingual definition representations and BM25 lexical retrieval. Class-balanced aggregation is applied at the broad level, after which the specific candidate set is restricted according to the selected broad genre. Weighted reciprocal rank fusion combines the specific-level rankings, while selective Qwen reranking is used only for uncertain broad predictions.

Accordingly, the contribution of the proposed system is not the isolated use of an embedding model, lexical retrieval, or a large language model. Its main contribution is the design and evaluation of a class-balanced, broad-first hybrid architecture that combines these components under the hierarchical and definition-guided constraints of AraGenre. The following sections first present the official baselines and their reproduction, and then describe the development experiments and the final system.

3-4 Official Baselines and Reproduction

The development of the AraGenre system began with an examination and reproduction of the two baseline systems included in the official starter kit [1]. This stage aimed to understand the decision mechanism of each baseline, verify the reproducibility of the reported results, and identify the limitations that should guide the subsequent experiments.

The first baseline performs direct semantic matching between the input text and the official genre definitions. The second baseline combines the retrieval of similar training examples with a large language model that generates the final classification. These systems represent two different methodological directions: a relatively simple embedding-based approach and a more complex retrieval-augmented generative approach.

It is important to distinguish the MiniLM baseline described in this section from the trained MiniLM experiments presented later in this chapter. In the official baseline, MiniLM is used without fine-tuning, and its role is limited to producing embeddings and computing semantic similarity. Training with text–definition pairs and negative examples belongs to the subsequent development phase.

3-4-1 Definition-Based MiniLM Baseline

The first official baseline uses the multilingual sentence-embedding model

paraphrase-multilingual-MiniLM-L12-v2

to compare each Arabic input text directly with the official genre definitions [2, 3].

Let x denote an input text and let d_c denote the official definition of candidate genre c . The model encodes both the input text and each definition into dense vector representations. The similarity score assigned to candidate c is then calculated using cosine similarity:

$$s_c = \cos(\mathbf{e}_x, \mathbf{e}_{d_c}), \quad (3.2)$$

where $\mathbf{e}_x$ is the embedding of the input text and $\mathbf{e}_{d_c}$ is the embedding of the definition of genre c . The predicted genre is the candidate with the highest similarity score:

$$\hat{y}_M = \arg \max_{c \in C} s_c, \quad (3.3)$$

where C denotes the set of candidate genres.

No task-specific fine-tuning or generative language model is used in this baseline. It can therefore be characterized as a zero-shot, definition-based semantic matching approach.

The principal advantages of this baseline are its simplicity, relatively low computational cost, and deterministic decision procedure. It also uses the official definitions rather than relying only on the names of the labels. However, its final prediction depends entirely on the highest cosine-similarity score, even when the score difference between the leading candidates is small.

The baseline also does not use the labeled training examples or explicitly model the hierarchical relationship between broad and specific genres.

3-4-2 Qwen Retrieval-Augmented Baseline

The second official baseline combines embedding-based retrieval with an instruction-following large language model. MiniLM is first used to retrieve labeled training examples that are semantically similar to the input text. The retrieved examples, the official genre definitions, and the target text are then inserted into a prompt provided to

Qwen2.5-3B-Instruct [8].

The baseline follows four principal steps:

1. Encode the target text and the labeled training examples using MiniLM.
2. Retrieve the training examples most similar to the target text.
3. Construct a prompt containing the target text, the retrieved examples, and the relevant genre definitions.
4. Use Qwen to generate the predicted broad and specific genres.

Qwen is not fine-tuned on the AraGenre data and is not used as an embedding model. Its role is limited to instruction-following classification based on the context included in the prompt. The approach can therefore be described as retrieval-augmented few-shot in-context classification.

Compared with direct embedding similarity, this baseline provides the classifier with both definitions and labeled examples. However, it also introduces additional sources of variability. Its performance may depend on the relevance of the retrieved examples, prompt construction, generation settings, model version, and inference environment. Misleading retrieved examples may also direct the model toward an incorrect genre.

3-4-3 Reproduced Baseline Results

The MiniLM baseline was reproduced locally using the official development data and genre definitions. The reproduced system obtained the same Macro F1 reported for the official baseline, indicating that its embedding and similarity procedure was successfully reproduced.

The Qwen RAG baseline was reproduced using the available API-based inference environment. Its reproduced result was lower than the officially reported baseline score. The results are presented in Table 3.4.

Table 3.4: Official and reproduced AraGenre baseline results on the development set.

System	Broad Macro F1	Specific Macro F1	Accuracy
Official MiniLM baseline	0.4658	0.4658	–
Reproduced MiniLM baseline	0.4658	0.4658	0.4818
Official Qwen RAG baseline	0.3899	0.3899	–
Reproduced Qwen RAG baseline	0.2436	0.2436	0.2818

The reproduced MiniLM baseline achieved a Macro F1 of 0.4658 and an accuracy of 0.4818. This exact agreement with the reported Macro F1 indicates that the definition-based embedding pipeline was stable and reproducible under the available experimental conditions.

In contrast, the reproduced Qwen RAG system achieved a Macro F1 of 0.2436 and an accuracy of 0.2818, compared with the officially reported Macro F1 of 0.3899. This difference suggests that the generative pipeline was more sensitive to implementation and inference conditions than the direct embedding baseline.

The lower reproduced result should not be interpreted as a general comparison between MiniLM and large language models. Instead, it indicates that the complete Qwen RAG pipeline was affected by factors such as retrieval quality, prompt construction, generation settings, and inference stability. Under the available configuration, the simpler definition-based baseline was more reliable and easier to reproduce.

3-4-4 Confidence-Based Ensemble

Following the baseline reproduction, a confidence-based ensemble was tested to determine whether the stability of MiniLM could be combined with the contextual reasoning of Qwen. For each input text, the two baseline systems generated independent predictions.

If the two systems agreed, their shared prediction was accepted. If they disagreed, the maximum similarity score returned by MiniLM was used as an estimate of confidence. A sufficiently high score caused the ensemble to retain the MiniLM prediction, whereas a lower score caused it to use the Qwen prediction.

Let $\hat{y}_M$ and $\hat{y}_Q$ denote the MiniLM and Qwen predictions, respectively. Let s_M denote the highest MiniLM similarity score and let τ denote the selected confidence threshold. The final ensemble prediction is defined as

$$\hat{y} = \begin{cases} \hat{y}_M, & \text{if } \hat{y}_M = \hat{y}_Q, \\ \hat{y}_M, & \text{if } \hat{y}_M \neq \hat{y}_Q \text{ and } s_M \geq \tau, \\ \hat{y}_Q, & \text{otherwise.} \end{cases} \quad (3.4)$$

The observed MiniLM similarity scores ranged from approximately -0.0517 to 0.4871 . Consequently, commonly used high confidence thresholds such as 0.60 or 0.70 were outside the observed range and were not appropriate for this experiment. Thresholds between 0.20 and 0.50 were therefore evaluated.

Table 3.5 reports the results of the threshold experiment.

Table 3.5: Development results of the confidence-based baseline ensemble.

Method or Threshold	Broad Macro F1	Specific Macro F1	Accuracy
MiniLM only	0.4658	0.4658	0.4818
Qwen only	0.2436	0.2436	0.2818
$\tau = 0.20$	0.4857	0.4857	0.5000
$\tau = 0.25$	0.4608	0.4608	0.4727
$\tau = 0.30$	0.4598	0.4598	0.4727
$\tau = 0.35$	0.4523	0.4523	0.4636
$\tau = 0.40$	0.4232	0.4232	0.4364
$\tau = 0.45$	0.4010	0.4010	0.4182
$\tau = 0.50$	0.3763	0.3763	0.4000

The best ensemble result was obtained with $\tau = 0.20$, which increased Macro F1 from 0.4658 for the reproduced MiniLM baseline to 0.4857 . Accuracy also increased from 0.4818 to 0.5000 . This result indicates that selective use of Qwen in low-confidence MiniLM cases corrected a limited number of errors.

Performance declined progressively as the threshold increased. A higher threshold caused more MiniLM predictions to be rejected and transferred a larger proportion of decisions to the weaker reproduced Qwen system. The experiment therefore demonstrates that the benefit of the language model depended on selective use rather than frequent replacement of the embedding-based predictions.

3-4-5 Baseline Findings

The baseline experiments produced several findings that guided the subsequent development process:

- The definition-based MiniLM baseline was simple, stable, and reproducible under the available experimental conditions.
- The use of a large language model within a RAG pipeline did not automatically improve classification performance.

- The Qwen baseline was sensitive to retrieval quality, prompt construction, and inference conditions.
- The confidence-based ensemble produced a limited improvement when Qwen was used only for low-confidence cases.
- Increasing reliance on the weaker reproduced Qwen baseline caused a consistent decline in performance.
- The best result at this stage, a Macro F1 of 0.4857, remained insufficient for a task with substantial semantic overlap and a complex hierarchical label structure.

These findings indicated that further improvement required more than adjusting an ensemble threshold or modifying the Qwen prompt. The next stage therefore investigated task-specific representation learning using positive and negative text–definition pairs, improved negative-sampling strategies, and the transition from MiniLM to Multilingual E5.

3-5 Development-Phase System

Following the reproduction and analysis of the official baselines, the work progressed from evaluating existing systems to developing a task-specific classification approach. The experiments were not conducted as isolated modifications. Each experiment was motivated by the results and error patterns observed in the preceding stage.

The development process began by fine-tuning MiniLM for text–definition matching and investigating alternative negative sampling strategies. MiniLM was then compared with Multilingual E5, after which E5-base was selected and fine-tuned for the task. Hierarchical information was subsequently incorporated into the candidate definitions. Finally, Recall at K and development-set error analysis were used to determine whether the remaining errors were caused by retrieval failure or incorrect candidate ranking. This analysis motivated the use of Qwen as a focused reranker rather than as a direct classifier.

Figure 3.1 summarizes the experimental progression followed during this phase.

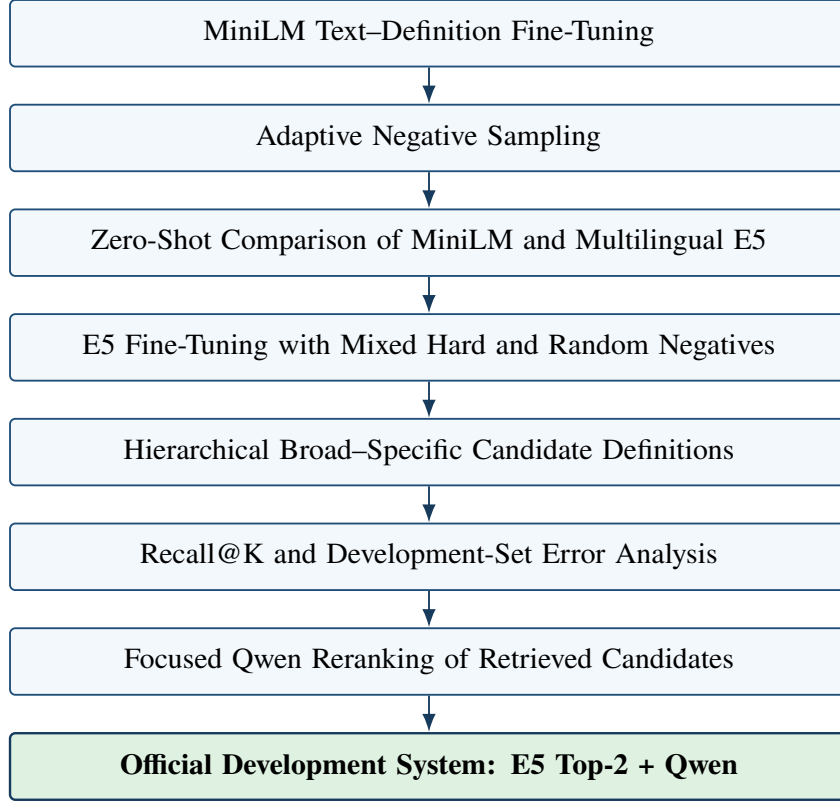


Figure 3.1: Experimental progression of the AraGenre system during the development phase.

3-5-1 MiniLM and Negative Sampling Experiments

The first development experiment used

paraphrase-multilingual-MiniLM-L12-v2

as a trainable text–definition matching model. Unlike the frozen MiniLM model used in the official baseline, this experiment fine-tuned the model using task-specific positive and negative text–definition pairs.

For every labeled Arabic text, one positive pair was constructed between the text and its correct genre definition. Three negative pairs were initially constructed using the incorrect definitions closest to the text in the semantic space. The training objective was to increase the similarity between a text and its correct definition while reducing its similarity to competing definitions.

The initial configuration achieved a Macro F1 of 0.8698 and an accuracy of 0.8727. This represented a substantial improvement over the frozen MiniLM baseline and demonstrated the value of task-specific representation learning.

However, error analysis identified false-negative cases in which a correct text–definition pair did not receive a higher score than a semantically related incorrect definition. Rather than replacing the model immediately, the next experiments investigated whether the number and type of negative examples were responsible for these errors.

The evaluated strategies included one hard negative, one random negative, two hard negatives, and a variable number of hard negatives selected according to example difficulty. Random negatives alone did not consistently provide informative competition, while applying the same fixed number of hard negatives to every text did not account for differences in classification difficulty.

The best MiniLM configuration used adaptive hard negatives. The comparison is presented in Table 3.6.

Table 3.6: Effect of adaptive negative sampling on the fine-tuned MiniLM system.

Configuration	Macro F1	Accuracy
One positive and three closest negatives	0.8698	0.8727
Adaptive hard negatives	0.8920	0.9000

Adaptive selection increased Macro F1 from 0.8698 to 0.8920 and accuracy from 0.8727 to 0.9000. The result indicates that negative-example selection should reflect the difficulty of each training instance. A text that is already well separated from incorrect genres does not require the same number of hard competitors as a text positioned near a fine-grained semantic boundary.

3-5-2 Transition to Multilingual E5

Although fine-tuned MiniLM achieved a strong improvement, the AraGenre task remained conceptually close to semantic retrieval and text–definition matching. For this reason, Multilingual E5 was investigated as an alternative representation model for multilingual semantic retrieval and cross-lingual text matching [4].

E5 is designed for multilingual semantic retrieval, which is particularly relevant because the input texts are written in Arabic, whereas the official genre definitions used during this phase were written in English. Two E5 variants were compared with MiniLM before task-specific fine-tuning:

- intfloat/multilingual-e5-base; and
- intfloat/multilingual-e5-large.

The zero-shot comparison was intended to isolate the quality of the pretrained representations from the effect of task-specific training. The results are shown in Table 3.7.

Table 3.7: Zero-shot comparison of MiniLM and Multilingual E5.

Model	Macro F1	Accuracy
MiniLM	0.4704	0.4909
Multilingual E5-base	0.6479	0.6818
Multilingual E5-large	0.6782	0.6818

Both E5 variants substantially outperformed MiniLM before fine-tuning. E5-large achieved the highest Macro F1, but its accuracy was equal to that of E5-base, and the performance difference was limited relative to the increase in computational cost and model size.

Multilingual E5-base was consequently selected for the subsequent experiments. The selection was based on the balance between semantic matching quality, memory requirements, training cost, and the need to conduct multiple controlled experiments under the available computational resources.

3-5-3 E5 Fine-Tuning

The selected

intfloat/multilingual-e5-base

model was fine-tuned for text–definition matching using `CosineSimilarityLoss`. Positive pairs associated each Arabic text with its correct genre definition, while negative pairs associated it with incorrect candidate definitions.

To estimate the difficulty of each training example, the difference between the score of the correct definition and the score of the highest-ranked incorrect definition was calculated:

$$m_i = s(x_i, d_i^+) - \max_{d \in \mathcal{D}_i^-} s(x_i, d), \quad (3.5)$$

where x_i denotes the input text, d_i^+ denotes its correct definition, $\mathcal{D}_i^-$ denotes the set of incorrect definitions, and m_i denotes the resulting margin.

A large margin indicates that the correct definition is already clearly separated from the incorrect candidates. A small or negative margin indicates a difficult example in which at least one incorrect definition competes closely with the correct one.

The number of selected negatives was adapted according to this difficulty estimate. One negative was used for relatively clear examples, two for medium-difficulty examples, and three for difficult examples. Fixed difficulty thresholds were initially evaluated, followed by thresholds derived from quantiles of the observed margin distribution.

The experiments also compared hard negatives with a mixture of hard and random negatives. Hard negatives expose the model to fine-grained semantic distinctions, whereas random negatives provide wider coverage of the complete label space.

Table 3.8 presents the principal E5 fine-tuning results.

Table 3.8: Results of negative-sampling strategies for fine-tuned E5-base.

Negative-Sampling Configuration	Macro F1	Accuracy
Adaptive hard negatives with fixed thresholds	0.8349	0.8364
Quantile adaptive, q_{33}/q_{66}	0.6477	0.6182
Quantile adaptive, q_{25}/q_{50}	0.7553	0.7273
Quantile adaptive, q_{50}/q_{75}	0.7413	0.7182
Mixed hard/random negatives, q_{33}/q_{66}	0.9107	0.9000

Quantile-based thresholds alone did not produce a consistent improvement. Their effectiveness depended on the types of negatives selected within each difficulty group. The best result was obtained by combining hard and random negatives using the q_{33}/q_{66} difficulty thresholds. This configuration achieved a Macro F1 of 0.9107.

The result suggests that hard and random negatives serve complementary purposes. Hard negatives improve discrimination between closely related genres, while random negatives extend coverage of the label space and reduce the risk of over-specializing the model around a limited set of competitors.

3-5-4 Hierarchical Candidate Definitions

In the initial E5 experiments, each candidate representation contained only the name and definition of the specific genre. The model therefore observed the 74 specific genres as a flat candidate space without an explicit description of their broad parent categories.

The candidate representation was subsequently modified to include:

- the name and definition of the broad genre;
- the name and definition of the specific genre; and
- an instruction to match the text with the specific genre within the correct broad parent.

This formulation allowed the model to represent each specific genre within its hierarchical context rather than as an isolated fine-grained definition. The effect of the hierarchical candidate representation is shown in Table 3.9.

Table 3.9: Effect of including hierarchical information in the candidate definitions.

Candidate Representation	Macro F1	Accuracy
Specific definition without hierarchy	0.9107	0.9000
Broad and specific hierarchical definition	0.9411	0.9364

Adding hierarchical information increased Macro F1 from 0.9107 to 0.9411 and accuracy from 0.9000 to 0.9364. This result demonstrates that explicitly representing the parent broad genre provides useful context for distinguishing specific genres.

In the best pre-Qwen configuration, the recorded broad and specific results were as follows:

- Broad Macro F1: 0.9411;
- Broad Accuracy: 0.9364;
- Specific Macro F1: 0.9411; and
- Specific Accuracy: 0.9364.

3-5-5 Selected E5 Development Configuration

The selected E5 configuration is summarized in Table 3.10.

Table 3.10: Selected E5 configuration during the development phase.

Component	Selected Configuration
Base model	intfloat/multilingual-e5-base
Training objective	Text-definition matching
Loss function	CosineSimilarityLoss
Negative sampling	Adaptive mixture of hard and random negatives
Difficulty thresholds	q_{33}/q_{66}
Learning rate	10^{-5}
Number of epochs	3
Batch size	4
Candidate representation	Broad and specific hierarchical definition
Score combination	0.5 trained score + 0.5 zero-shot score

The final prediction score did not depend exclusively on the fine-tuned model. Instead, the fine-tuned score was combined with the score produced by the original zero-shot model:

$$s_{\text{final}} = 0.5 s_{\text{trained}} + 0.5 s_{\text{zero-shot}}. \quad (3.6)$$

This combination aimed to retain part of the pretrained model’s general multilingual semantic representation while benefiting from task-specific adaptation.

Two additional alternatives were evaluated but did not improve the hierarchical-definition configuration. Translating the definitions into Arabic produced an approximate Macro F1 of 0.6383 and an accuracy of 0.7000. A multi-task configuration that modeled broad and specific classification as separate objectives achieved an approximate Macro F1 of 0.7400 and an accuracy of 0.7600.

These results indicate that translating the definitions into the language of the input text did not automatically improve semantic matching. The English hierarchical definitions were more effective with Multilingual E5-base, and explicitly representing the broad parent within the candidate text was more successful than training broad and specific classification as separate objectives.

3-5-6 Recall at K and Error Analysis

After selecting the E5 configuration, Recall at K was analyzed on the 110 development examples. The purpose of this analysis was to distinguish between two types of failure:

1. **Retrieval failure**, in which the correct genre is absent from the highest-ranked candidates; and
2. **Ranking failure**, in which the correct genre is retrieved but is not placed in the first position.

The results are presented in Table 3.11.

Table 3.11: Recall at K analysis of E5 on the development set.

Measure	Result	Interpretation
Recall@1	92.73%	The correct genre was ranked first for 102 of the 110 development texts.
Recall@2	100%	The correct genre appeared within the first two candidates for every development text.
Recall@3	100%	No correct genre was ranked outside the first three candidates.
Gold outside Top-3	0	No retrieval failure was observed on the development set.

E5 Top-1 achieved an accuracy of 0.9273 and a Macro F1 of 0.9230, correctly classifying 102 of the 110 development texts. The system made eight Top-1 errors, but the correct genre was ranked second in every one of these cases.

This result showed that the principal remaining problem was not the failure to retrieve the correct genre. Instead, the system had difficulty ordering two highly similar candidates correctly.

The eight errors were concentrated in three semantic confusion patterns, as shown in Table 3.12.

Table 3.12: Principal E5 Top-1 error patterns on the development set.

Correct Genre	E5 Top-1 Prediction	Cases
Motivational Writing	Advice Columns	5
Educational Explanations	Analytical Reports	2
Religious Commentary	Motivational Writing	1

The errors were semantically interpretable rather than random. Motivational writing often uses language associated with advice and guidance. Educational explanations may contain numerical expressions, formulas, and analytical structures. Similarly, religious commentary may use emotional and encouraging language resembling motivational writing.

The score margins between the first- and second-ranked candidates were also small:

- mean margin: 0.0406;
- minimum margin: 0.0181; and
- maximum margin: 0.0747.

The Recall@2 result and the narrow score margins indicated that improving retrieval coverage was not the immediate priority. A more suitable next step was to introduce a focused decision mechanism capable of distinguishing between the two closest semantic candidates.

3-5-7 Qwen Classification and Focused Reranking

Qwen was first evaluated as a direct classifier that received the input text and the complete label space. This configuration achieved a Macro F1 of 0.8633 and an accuracy of 0.8636, which were lower than the corresponding E5 Top-1 results.

Providing the full label space created a broad and difficult decision problem for the generative model. Consequently, Qwen’s role was changed from direct classification to focused reranking. E5 retrieved a small candidate set, and Qwen was asked only to distinguish between these semantically close candidates.

The comparison is presented in Table 3.13.

Table 3.13: Comparison of direct Qwen classification and focused reranking.

System	Macro F1	Accuracy
Qwen direct classification	0.8633	0.8636
E5 Top-1	0.9230	0.9273
E5 Top-3 + Qwen reranking	0.9705	0.9727

Focused reranking reduced the number of development errors from eight with E5 Top-1 to three with E5 Top-3 and Qwen. Since Recall@2 was already 100%, Top-2 was selected for

the official development system. This choice preserved the correct candidate in every observed development case while reducing the decision space presented to Qwen.

Retrieved training examples were also added to the reranking prompt in a Train RAG experiment. However, they did not alter Qwen’s decisions or produce an additional improvement. This finding suggests that the remaining errors were caused by fine semantic boundaries rather than by the absence of demonstrations in the prompt.

3-5-8 Official Development Result

The selected official development system consisted of two stages:

1. **E5 retrieval:** retrieve the two highest-ranked genres using the fine-tuned E5 model and hierarchical candidate definitions; and
2. **Qwen reranking:** select the final genre through a focused comparison between the two retrieved candidates.

Table 3.14 distinguishes the local reranking experiment from the official Codabench development result.

Table 3.14: Local and official results of the selected AraGenre development systems.

Evaluation	System	Macro F1 / H-MF1	Accuracy
Local development	E5 Top-3 with Qwen reranking	0.9705 Macro F1	0.9727
Official Codabench development	E5 Top-2 with Qwen reranking	0.9788 H-MF1	0.9818

The two rows should not be interpreted as measurements of an identical configuration. They differ in candidate-set size and evaluation environment. The official result records the performance of the selected submission on the Codabench development track, whereas the local result documents the preceding reranking experiment.

3-6 Final-Test Generalization Gap

After selecting the development-stage system, the next step was to examine whether its performance transferred to the substantially larger final-test set. The official development submission used E5 Top-2 retrieval followed by Qwen reranking, whereas the first final-test submission used direct E5 Top-1 prediction without Qwen. Consequently, the two official scores do not constitute a strict like-for-like comparison.

The first final-test submission was intended primarily to evaluate the transfer of the underlying E5 retrieval component before applying a generative reranker at scale. Its most appropriate development reference is therefore the local E5 Top-1 result, which achieved a Macro F1 of

0.9230 and an accuracy of 0.9273, rather than the complete E5 Top-2 plus Qwen development pipeline.

3-6-1 First Final-Test Submission

The direct E5 Top-1 submission achieved the following results:

- Hierarchical Macro F1: 0.178453;
- Broad Macro F1: 0.326503;
- Broad Accuracy: 0.369298;
- Specific Macro F1: 0.030403; and
- Specific Accuracy: 0.093665.

The reduction from the local E5 Top-1 development result to the direct E5 Top-1 final-test result indicated that the learned retrieval representations did not transfer reliably to the substantially larger evaluation distribution. The particularly low Specific Macro F1 also showed that the problem was not limited to a small number of closely ranked candidates that could be corrected through reranking.

The absence of final-test reference labels prevented the construction of a confusion matrix or example-level error analysis. Consequently, the causes of the performance reduction were inferred from the aggregate metrics, prediction distributions, and behavior observed across successive submissions rather than from direct inspection of the hidden gold labels.

The results suggested that the system was affected by the small size of the development sample, differences in the final-test distribution, uncalibrated label scores, and the imbalance of the hierarchical candidate space. They also indicated that inferring the broad genre indirectly from the highest-ranked specific genre was not sufficiently robust.

3-6-2 Hierarchical Calibration

The first corrective experiment applied hierarchical calibration to the E5 scores. The fine-tuned E5 score was combined with the original zero-shot score, and label-wise calibration was introduced to reduce the dominance of labels that received consistently high scores. A soft broad-level signal was also added to support hierarchical compatibility without introducing a fully independent broad classifier.

The calibrated system increased Hierarchical Macro F1 from 0.178453 to 0.189390. Broad Macro F1 increased to 0.332189, while Specific Macro F1 increased to 0.046590.

Although the changes were positive, the improvement was limited. Score calibration reduced part of the bias in the original predictions but did not resolve the structural difficulty of selecting a specific genre from the complete fine-grained label space.

3-6-3 True Two-Stage E5 Classification

The task was subsequently reformulated as a true two-stage classification process. In the first stage, a dedicated broad classifier selected one of the six broad genres. In the second stage, the specific candidate set was restricted to the genres associated with the predicted broad category.

This configuration separated broad selection from specific classification. The specific classifier was no longer required to infer the broad genre implicitly from a global ranking of all fine-grained candidates.

The two-stage system achieved a Hierarchical Macro F1 of 0.276410. Broad Macro F1 increased to 0.479686, and Broad Accuracy reached 0.492206. Specific Macro F1 increased to 0.073135, while Specific Accuracy reached 0.144001.

The improvement over calibrated E5 confirmed that broad-first classification was more suitable than direct global specific classification. Nevertheless, specific-level performance remained weak, showing that reducing the candidate space did not by itself provide sufficiently discriminative evidence within each broad category.

An intermediate configuration also exhibited severe prediction collapse toward a limited number of specific genres. It achieved an approximate Hierarchical Macro F1 of 0.2195, with Specific Macro F1 decreasing to approximately 0.0091. This configuration was not retained, but its behavior provided further evidence that global scoring over an imbalanced candidate space could cause a small number of labels to dominate the predictions.

3-6-4 Corrected Broad-First Hybrid

Following the two-stage experiment, the broad and specific decision procedures were revised separately.

At the broad level, the input text was compared with the six original broad definitions using E5. Score centering was applied with a factor of 0.8. A limited keyword bonus of 0.04, capped at 0.12, was included as supplementary evidence rather than as the primary decision signal.

After selecting the broad genre, only the associated specific genres were retained. The score of each specific candidate was calculated by combining the fine-tuned and zero-shot E5 scores:

$$s_{\text{specific}} = 0.5 s_{\text{trained}} + 0.5 s_{\text{zero-shot}}. \quad (3.7)$$

The corrected broad-first system achieved a Hierarchical Macro F1 of 0.321161. Its Broad Macro F1 was 0.447066, with a Broad Accuracy of 0.517375. Specific Macro F1 increased substantially to 0.195256, and Specific Accuracy reached 0.208709.

Although Broad Macro F1 was lower than that of the preceding two-stage system, the considerable improvement at the specific level produced a higher overall hierarchical score. This result demonstrated that broad-level performance should not be optimized in isolation from the quality of the subsequent specific decision.

Confidence thresholds of 0.8 and 0.7 were also evaluated. The 0.8 threshold did not improve the reference score, while the 0.7 threshold produced a slight reduction to approximately 0.3209. Confidence thresholding was therefore not retained in the reference configuration.

3-6-5 Adaptive Broad E5

A final E5-based experiment modified only the broad classification stage by introducing adaptive hard-negative weighting and an out-of-distribution guard. The specific-stage configuration remained based on the corrected broad-first system.

This experiment achieved a Hierarchical Macro F1 of 0.325016. Broad Macro F1 increased from 0.447066 to 0.465889, while Specific Macro F1 decreased from 0.195256 to 0.184143.

The small overall improvement resulted from stronger broad classification combined with weaker specific classification. This trade-off confirmed that the specific stage had become the principal performance bottleneck and that further modifications to E5 scoring alone were unlikely to provide a sufficient improvement.

3-6-6 Progression of the Initial Final-Test Experiments

Table 3.15 summarizes the principal experiments conducted before introducing Arabic prototype representations.

Table 3.15: Progression of the initial AraGenre final-test experiments.

System	H-MF1	Broad Macro F1	Broad Acc.	Specific Macro F1	Specific Acc.
Direct E5 Top-1	0.178453	0.326503	0.369298	0.030403	0.093665
Calibrated hierarchical E5	0.189390	0.332189	0.394859	0.046590	0.095739
True two-stage E5	0.276410	0.479686	0.492206	0.073135	0.144001
Corrected broad-first hybrid	0.321161	0.447066	0.517375	0.195256	0.208709
Adaptive broad E5	0.325016	0.465889	0.511440	0.184143	0.201880

The largest improvement in this stage resulted from separating broad and specific classification and redesigning the specific decision within the predicted broad category. Calibration and confidence thresholding produced only limited changes.

3-6-7 Lessons from the Initial Final-Test Experiments

The initial final-test experiments produced the following findings:

- Strong development performance did not guarantee reliable transfer to the larger final-test distribution.
- Score calibration provided a limited improvement but did not resolve the imbalance of the candidate space.

- Separating broad and specific classification produced a substantial improvement over direct global classification.
- Hard hierarchical gating guaranteed consistency but caused every broad-level error to propagate to the specific stage.
- Improvements in Broad Macro F1 did not necessarily improve the overall hierarchical score when Specific Macro F1 decreased.
- Further adjustment of E5 scores and confidence thresholds was insufficient to address the remaining limitations.

These findings motivated the construction of richer Arabic representations for both broad and specific genres. The following section describes the creation and validation of these prototypes before their use in retrieval-based classification.

3-7 Arabic Prototype Construction

3-7-1 Motivation and Generation Method

The initial final-test experiments showed that representing each genre using a single official definition did not provide sufficient coverage of the linguistic and semantic diversity present in unseen Arabic texts. A genre definition describes the general meaning of a label, but it does not necessarily represent the different topics, writing styles, dialects, lengths, structures, and noise patterns through which the same genre may appear.

The system was therefore extended from single-definition matching to retrieval over multiple Arabic prototypes. These prototypes were intended to provide several valid realizations of each genre and to support retrieval-based classification without fine-tuning the representation models on the final test set.

The prototypes were generated using OpenAI’s ChatGPT during July 2026 [11]. Since the exact model version was not recorded with sufficient certainty, no unverified version identifier is reported. The generation process used the official genre definitions and the resources permitted by the task.

For each generated prototype, the instructions specified the target specific genre, its broad parent, and the corresponding genre definition. The requested texts were varied according to:

- text length and structural complexity;
- topic and communicative situation;
- writing style and linguistic register;
- Modern Standard Arabic and dialectal variation, where appropriate; and
- realistic spelling, punctuation, and formatting noise.

The objective was not to create multiple paraphrases of one representative text. Instead, the generation process aimed to cover different valid forms of the same communicative genre while preserving compatibility with both the specific label and its broad parent.

3-7-2 Broad and Specific Prototype Allocation

Different allocation strategies were used for broad and specific classification.

Broad prototypes were allocated according to the internal diversity of each broad genre rather than assigning the same number to every category. The allocation considered the number of specific genres within each broad category, the expected variety of its textual forms, and its potential overlap with other categories.

At the specific level, exactly ten Arabic prototypes were generated for each of the 74 specific genres. This produced numerical balance across the specific labels, with a total of 740 specific prototypes.

Table 3.16 summarizes the complete allocation.

Table 3.16: Allocation of Arabic prototypes across the AraGenre hierarchy.

Broad Genre	Specific Genres	Broad Prototypes	Specific Prototypes
Creative	7	12	70
Informative	42	30	420
Interactive	4	8	40
Learning	12	15	120
Legal	3	6	30
Religious	6	10	60
Total	74	81	740

The broad allocation produced 81 generated prototypes. One official definition was retained for each of the six broad genres, resulting in 87 broad-level entries.

The generated Arabic corpus contained 821 texts in total: 81 broad prototypes and 740 specific prototypes. When the six official broad definitions were counted with these resources, the complete retrieval collection contained 827 entries.

The distinction between 821 and 827 reflects two experimental configurations rather than an inconsistency. The value 821 refers only to generated Arabic texts. The value 827 additionally includes the six official broad definitions. In later experiments, these definitions were either merged with the prototypes or processed through an independent definition branch.

3-7-3 Quality, Diversity, and Data-Integrity Validation

Prototype generation was conducted iteratively. Generated batches were reviewed, rejected when necessary, and regenerated using more specific instructions. The validation process examined:

- complete coverage of all broad and specific genres;
- unique and valid identifiers and label assignments;
- the required number of ten accepted prototypes per specific genre;
- exact duplicate texts and repeated templates;
- lexical, structural, topical, and length diversity; and
- consistency between the text, its specific label, and its broad parent.

Batches exhibiting substantial repetition or reliance on fixed templates were rejected. Common rejection reasons included repeated openings and conclusions, identical rhetorical structures with only minor lexical substitutions, insufficient variation among long texts, and weak representation of the target genre’s internal diversity. Rejected prototypes were regenerated while preserving their intended identifiers and labels.

The quality-control objective was not to produce uniformly formal and noise-free Arabic. Since the target data include variation in dialect, spelling, punctuation, style, and formatting, realistic variation was preserved when it remained consistent with the communicative function of the assigned genre.

The hidden final-test labels were not provided to ChatGPT and were not used to generate, select, or correct the prototypes. The generation instructions explicitly prohibited copying or closely paraphrasing final-test texts. ChatGPT was also not given example-level evaluation feedback that could be used to infer or modify the unknown labels.

The prototypes were therefore treated as externally generated synthetic resources based on the official genre definitions, not as indirectly labeled final-test examples.

3-7-4 Broad-Pool Imbalance

The construction process produced numerical balance at the specific level because each specific genre contributed exactly ten prototypes. However, this balance did not extend to the broad level.

The broad categories contain different numbers of specific children. Consequently, Informative inherited 420 specific prototypes, whereas Legal inherited only 30. A global nearest-neighbour vote over the complete pool could therefore favor categories with more available prototypes, since larger categories have more opportunities to place one or more examples near the input text.

This issue introduced an important distinction between prototype coverage and classification balance. Removing examples from Informative would reduce its representation of 42 internally

diverse specific genres. Retaining all examples without controlling score aggregation, however, could bias the broad decision toward the largest pool.

The later retrieval systems therefore had to consider:

- the similarity evidence provided by each prototype;
- the number of prototypes associated with each class;
- the method used to aggregate prototype-level evidence into a class-level score; and
- the relative contribution of official definitions and generated Arabic examples.

This imbalance motivated the transition from global neighbour voting to class-balanced score aggregation. Before introducing the final balanced method, the prototype collection was evaluated using Frozen E5, BM25, and inverse-distance Weighted KNN. These retrieval-based experiments are presented in the following section.

3-8 Retrieval-Based Hierarchical Classification

Following the construction of the Arabic prototype collection, the system moved from single-definition matching to example-based retrieval. The objective was to represent each genre through multiple Arabic texts and reduce dependence on the fine-tuned E5 model that had generalized poorly from the development set.

The initial retrieval system combined Frozen Multilingual E5, BM25, and inverse-distance Weighted KNN. No model was fine-tuned using the final test data.

3-8-1 Frozen Multilingual E5 and BM25

The model

intfloat/multilingual-e5-base

was used without additional fine-tuning to encode the input texts and prototype representations. Frozen E5 provided semantic similarity, while BM25 supplied complementary lexical evidence based on term frequency and document frequency [5].

The two signals were converted into comparable distance measures and combined as follows:

$$d_{\text{hybrid}} = 0.75 d_{\text{E5}} + 0.25 d_{\text{BM25}}, \quad (3.8)$$

where d_{E5} denotes the semantic distance and d_{BM25} denotes the normalized lexical distance.

E5 received the larger weight because semantic similarity represented the primary retrieval signal, whereas BM25 was used to preserve distinctive lexical evidence that might be weakened in dense embeddings.

3-8-2 Inverse-Distance Weighted KNN

After calculating the hybrid distance, the system retrieved the nearest k prototypes and aggregated their evidence according to the principle of distance-weighted nearest-neighbour classification [6]. Rather than assigning an equal vote to every retrieved neighbour, each prototype received a weight inversely proportional to its distance from the input text:

$$w_i = \frac{1}{d_i + \varepsilon}, \quad (3.9)$$

where d_i is the hybrid distance of neighbour i , and ε is a small positive constant used to prevent division by zero.

The score of class c was then calculated by summing the weights of the retrieved neighbours assigned to that class:

$$S(c) = \sum_{i \in \mathcal{N}_k} w_i \mathbb{I}[y_i = c], \quad (3.10)$$

where $\mathcal{N}_k$ denotes the set of retrieved neighbours and $\mathbb{I}[\cdot]$ is the indicator function.

An initial Frozen E5 and BM25 configuration with limited prototype coverage achieved an H-MF1 of 0.1938. This result showed that changing the retrieval method was insufficient when the prototype collection did not adequately cover the internal diversity of the genres.

After completing the 87-entry broad prototype collection, Weighted KNN V5 was evaluated using $k = 3$. It achieved:

- Hierarchical Macro F1: 0.343838;
- Broad Macro F1: 0.481224;
- Broad Accuracy: 0.580473;
- Specific Macro F1: 0.206452; and
- Specific Accuracy: 0.214107.

Compared with the preceding corrected broad-first system, H-MF1 increased from 0.321161 to 0.343838. The result demonstrated that retrieval over diverse Arabic examples provided more effective evidence than relying on one definition per genre.

3-8-3 Hierarchical Pool Expansion and Hard Gating

The next configuration extended the broad retrieval pool by including the 740 specific prototypes under their corresponding broad parents. The combined pool contained 827 entries: 81 broad prototypes, 740 specific prototypes, and six official broad definitions.

The system used $k = 3$ at both hierarchy levels. After predicting the broad genre, hard gating restricted the specific candidates to the labels belonging to the selected broad category:

$$C_{\text{specific}} = \mathcal{S}(\hat{y}_{\text{broad}}), \quad (3.11)$$

where $\mathcal{S}(\hat{y}_{\text{broad}})$ denotes the set of specific genres associated with the predicted broad genre. The resulting Hierarchical Weighted KNN V7 system achieved:

- Hierarchical Macro F1: 0.347119;
- Broad Macro F1: 0.517444;
- Broad Accuracy: 0.633634;
- Specific Macro F1: 0.176795; and
- Specific Accuracy: 0.210603.

Adding the specific prototypes to the broad retrieval pool increased Broad Macro F1 from 0.481224 to 0.517444 and Broad Accuracy from 0.580473 to 0.633634. However, Specific Macro F1 decreased from 0.206452 to 0.176795.

The specific prototypes strengthened the evidence available for selecting the broad parent, but global neighbour voting remained sensitive to the unequal broad-pool sizes and the semantic similarity between fine-grained genres.

Hard gating guaranteed hierarchical consistency, but it also caused every incorrect broad prediction to exclude the correct specific genre from the second-stage candidate set.

3-8-4 Weighted KNN Results

Table 3.17 compares the principal retrieval-based configurations.

Table 3.17: Results of the initial retrieval-based AraGenre systems.

System	H-MF1	Broad Macro F1	Broad Acc.	Specific Macro F1	Specific Acc.
Corrected broad-first	0.321161	0.447066	0.517375	0.195256	0.208709
Weighted KNN V5	0.343838	0.481224	0.580473	0.206452	0.214107
Hierarchical Weighted KNN V7	0.347119	0.517444	0.633634	0.176795	0.210603

V7 obtained the highest hierarchical and broad-level scores, whereas V5 retained the strongest specific-level performance within this stage. This difference showed that the same neighbour aggregation strategy was not equally suitable for broad and specific classification.

3-8-5 Neighbourhood Size Experiments

Neighbourhood sizes

$$k \in \{1, 3, 5, 7, 10\}$$

were evaluated. The best result in this group of experiments was obtained with $k = 5$, reaching an H-MF1 of approximately 0.3510.

Increasing k sometimes improved Broad Accuracy but reduced Macro F1. Larger broad categories benefited more from additional neighbours because they contained more prototypes. Consequently, increasing the number of neighbours did not resolve the imbalance of the global retrieval pool.

The results indicated that the principal limitation was not the value of k alone, but the method used to aggregate prototype-level evidence into class-level scores. This motivated the replacement of global neighbour voting with class-balanced aggregation and the addition of an Arabic semantic encoder in the following system.

3-9 Final Class-Balanced Hybrid System

The preceding experiments showed that using the same aggregation method at the broad and specific levels did not produce the strongest hierarchical performance. Global Weighted KNN benefited from the Arabic prototypes but remained sensitive to the number of examples available for each class.

The final system therefore aggregates evidence separately within each label before comparing labels. It follows a two-stage broad-first architecture with two different aggregation mechanisms:

- class-balanced score fusion for broad classification; and
- Weighted Reciprocal Rank Fusion for specific classification within the predicted broad category.

Score fusion was retained for broad classification because it produced stronger results across the six broad genres. Rank fusion was used at the specific level because the retrieval channels produced scores on different scales and the fine-grained labels exhibited substantial semantic overlap.

3-9-1 Final Classification Pipeline

Each Arabic input text is processed independently by three retrieval channels. Their outputs are aggregated to select the broad genre. Low-confidence broad predictions may be examined by a selective Qwen reranker. After selecting the final broad genre, hard gating restricts the specific candidate set, and Weighted RRF produces the final specific prediction.

Figure 3.2 presents the complete classification pipeline.

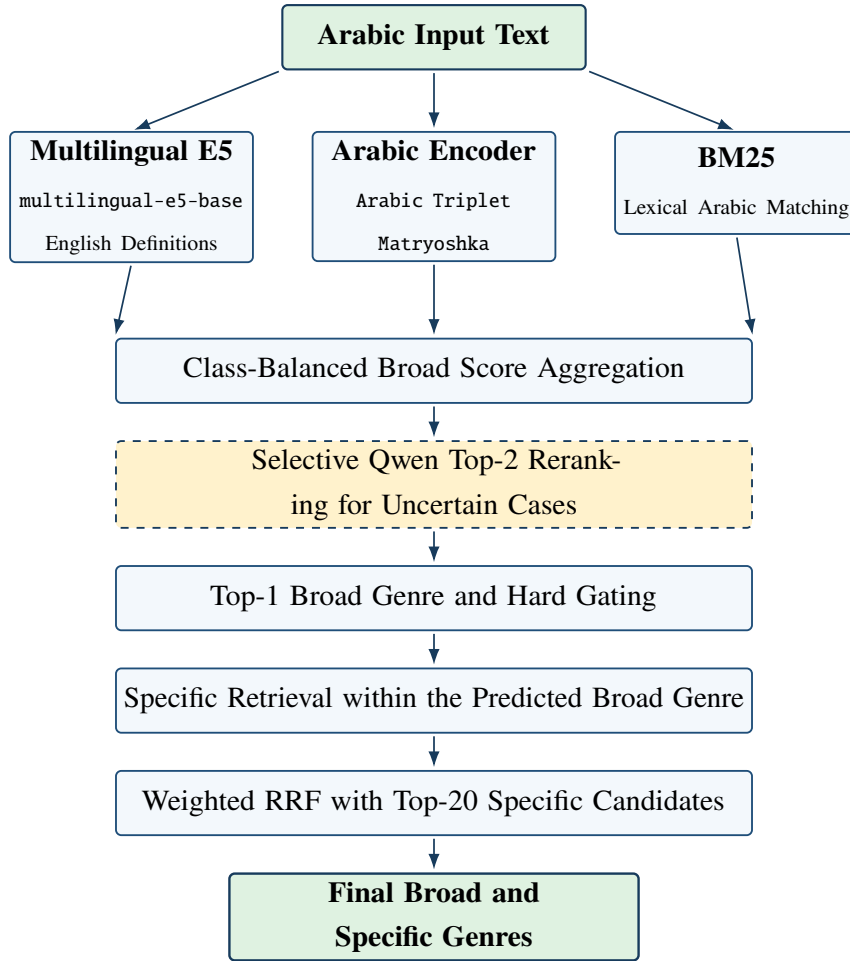


Figure 3.2: Final class-balanced broad-first AraGenre classification pipeline.

3-9-2 Evidence Channels

The final system uses three independent evidence channels, summarized in Table 3.18.

Table 3.18: Evidence channels used by the final AraGenre system.

Channel	Role	Weight
Multilingual E5	Cross-lingual semantic comparison between Arabic input texts and English genre definitions.	0.30
Arabic Triplet Matryoshka	Direct semantic comparison between Arabic input texts and Arabic examples or translated broad definitions.	0.50
BM25	Lexical matching between input texts and generated Arabic examples.	0.20

The Multilingual E5 channel uses

intfloat/multilingual-e5-base

to compare Arabic documents with the official English genre definitions. This channel preserves the relationship between the input text and the original task definitions.

The Arabic semantic channel uses the Arabic Triplet Matryoshka embedding model [10] to compare the input text directly with generated Arabic prototypes and translated broad definitions. It receives the largest weight because both the queries and prototype examples are written in Arabic.

BM25 provides complementary lexical evidence. It preserves the influence of distinctive words and phrases that may not be represented with equal strength in dense semantic embeddings.

The three channels do not perform a direct global vote over the retrieved prototypes. Their evidence is first aggregated at the class level, and the aggregation method differs between broad and specific classification.

3-9-3 Class-Balanced Broad Retrieval

For broad classification, prototype evidence is aggregated separately within each broad genre. The five highest-scoring examples associated with each broad category are retained.

Let $\mathcal{T}_5(b)$ denote the five highest-scoring examples of broad genre b , and let s_i denote the score of example i . The normalized softmax weight of each retained example is

$$\alpha_i = \frac{\exp(s_i)}{\sum_{j \in \mathcal{T}_5(b)} \exp(s_j)}. \quad (3.12)$$

The resulting example-based broad score is

$$S_{\text{examples}}(b) = \sum_{i \in \mathcal{T}_5(b)} \alpha_i s_i. \quad (3.13)$$

This procedure assigns one aggregate example score to each broad genre, regardless of the total number of prototypes associated with that category. Categories with larger pools therefore retain their internal coverage without receiving an automatic advantage from their number of examples.

3-9-4 Broad-Level Score Fusion

The broad decision combines example-based evidence with English and Arabic definition evidence.

The English and Arabic broad-definition scores are combined as

$$S_{\text{definition}}(b) = 0.40 S_{\text{EnglishDef}}(b) + 0.60 S_{\text{ArabicDef}}(b). \quad (3.14)$$

The final broad score is then calculated as

$$S_{\text{broad}}(b) = 0.85 S_{\text{examples}}(b) + 0.15 S_{\text{definition}}(b). \quad (3.15)$$

Generated examples receive the larger contribution because they represent multiple realizations of each genre. The definition branch retains a stable connection to the intended meaning of the official label.

The predicted broad genre is

$$\hat{b} = \arg \max_{b \in \mathcal{B}} S_{\text{broad}}(b), \quad (3.16)$$

where $\mathcal{B}$ denotes the set of six broad genres.

Score fusion was retained at the broad level because it produced stronger broad classification than rank fusion in the official intermediate experiments.

3-9-5 Specific Retrieval and Weighted RRF

After selecting the broad genre, hard gating restricts the specific candidate set to the child genres of the predicted broad category.

Each evidence channel then produces an independent ranking of the remaining specific genres:

- E5 ranks specific genres according to their English definitions;
- Arabic Triplet Matryoshka ranks them according to their generated Arabic examples; and
- BM25 ranks them according to lexical similarity with the Arabic examples.

The evidence associated with each specific genre is summarized using its three highest-scoring examples. This prevents a label from benefiting from repeated similar examples and provides one example-based representation for every specific candidate.

The raw scores generated by E5, Arabic Triplet Matryoshka, and BM25 are not directly comparable. The system therefore converts each channel output into an independent label ranking and combines the rankings using a weighted extension of Reciprocal Rank Fusion [7].

For specific genre s , the fusion score is

$$S_{\text{RRF}}(s) = \sum_{m \in \mathcal{M}} \frac{w_m}{10 + r_m(s)}, \quad (3.17)$$

where $\mathcal{M}$ is the set of retrieval channels, $r_m(s)$ is the rank assigned to genre s by channel m , and w_m is the weight of that channel.

The channel weights are

$$w_{\text{E5}} = 0.30, \quad w_{\text{Arabic}} = 0.50, \quad w_{\text{BM25}} = 0.20. \quad (3.18)$$

Each channel contributes its Top-20 specific candidates to the fusion process. The final specific prediction is

$$\hat{s} = \arg \max_{s \in \mathcal{S}(\hat{b})} S_{\text{RRF}}(s). \quad (3.19)$$

When two candidates receive the same RRF score, the Arabic Triplet Matryoshka ranking is used first to break the tie, followed by E5 and then BM25.

3-9-6 Combination of the Selected Broad and Specific Stages

The strongest broad and specific configurations did not originate from the same aggregation method. The final pipeline therefore combines:

1. class-balanced broad score fusion;
2. Top-1 broad selection;
3. hard gating of the specific candidate space; and
4. Top-20 Specific Weighted RRF within the selected broad category.

This combination is not an average of two completed prediction files. Specific scores are recomputed under the broad predictions generated by the class-balanced broad stage.

3-9-7 Selective Qwen Reranking

Qwen is used only as a selective broad-level reranker for uncertain cases. It is not applied as a global classifier to every final-test document.

The reranker uses

Qwen3-4B-Instruct-2507

with 4-bit NF4 quantization [9]. Candidate cases are restricted to documents that satisfy two conditions:

- the broad score margin falls within the lowest 6% of margins; and
- the retrieval channels or evidence branches disagree.

A maximum of 1,200 documents is processed by Qwen. For every selected document, the model receives:

- the input Arabic text;
- the two highest-ranked broad candidates;
- the English and Arabic definitions of both candidates; and
- two nearby Arabic examples for each candidate.

Candidate order is reversed deterministically across prompts to reduce position bias. The original broad prediction is changed only if Qwen selects the second-ranked candidate and that candidate is supported by at least two internal retrieval channels.

If the broad prediction changes, the specific-stage Weighted RRF is recomputed within the revised broad category. Otherwise, the original retrieval-based decision is preserved.

These restrictions make Qwen a conservative correction layer rather than the primary source of classification decisions.

3-9-8 Implementation Details

The principal implementation settings of the final system are summarized in Table 3.19.

Table 3.19: Implementation settings of the final AraGenre system.

Component	Configuration
independent-model	20260801
Multilingual model	intfloat/multilingual-e5-base
Arabic model	Arabic Triplet Matryoshka
Encoder status	Frozen during final evaluation
E5 maximum input length	256 tokens
Arabic-model maximum input length	512 tokens
E5 batch size	64
Arabic-model batch size	32
Processing chunk size	1,024 documents
Channel weights	0.30 E5, 0.50 Arabic Triplet Matryoshka, and 0.20 BM25
Broad example aggregation	Top-5 examples per broad genre
Specific example aggregation	Top-3 examples per specific genre
RRF constant	10
Specific candidate limit	Top-20
Qwen model	Qwen3-4B-Instruct-2507
Qwen quantization	4-bit NF4
Qwen margin quantile	0.06
Maximum Qwen cases	1,200
Qwen maximum input	4,096 tokens
Maximum text length for Qwen	3,200 characters
Minimum runner-up support	Two internal channels

The following section reports the experimental results of the class-balanced system, the Weighted RRF configurations, the combined broad-specific pipeline, and the selective Qwen reranker.

3-10 Experimental Results and Ablation Analysis

This section reports the results obtained after replacing global neighbour voting with the final class-balanced retrieval design. The earlier E5 and Weighted KNN results are not repeated here.

3-10-1 Class-Balanced Retrieval Result

The Class-Balanced Arabic Ensemble V10 achieved an H-MF1 of 0.441909, compared with 0.347119 for the preceding Hierarchical Weighted KNN system. This absolute improvement of 0.094790 was the largest single gain obtained during the final-test experiments.

V10 achieved a Broad Macro F1 of 0.622465 and a Broad Accuracy of 0.715358. At the specific level, it achieved a Macro F1 of 0.261353 and an accuracy of 0.288002.

The result showed that class-balanced aggregation was more effective than global neighbour voting. However, V10 introduced the Arabic semantic encoder, translated broad definitions, and class-balanced aggregation together. Their individual effects cannot therefore be isolated from this submission.

3-10-2 Weighted RRF and Candidate-Size Experiments

Several candidate limits were evaluated for Weighted RRF. The documented results are presented in Table 3.20.

Table 3.20: Results of the documented Weighted RRF candidate-size experiments.

Configuration	H-MF1	Broad Macro F1	Broad Acc.	Specific Macro F1	Specific Acc.
Broad $K = 5$, Specific $K = 10$	0.425396	0.589536	0.702810	0.261257	0.302946
Broad $K = 6$, Specific $K = 15$	0.437289	0.601901	0.716109	0.272677	0.315029
Broad $K = 6$, Specific $K = 20$	0.438313	0.601901	0.716109	0.274725	0.317603
True retrieval $K = 8/K = 20$	0.442123	0.611853	0.727763	0.272394	0.312706

Increasing the specific candidate limit from 10 to 20 improved specific-level performance within the corresponding RRF configurations. The True Retrieval configuration produced the highest H-MF1 and broad-level performance among the independent RRF systems.

The meaning of K was not identical in all rows. In the first three configurations, K represented the number of labels retained from each channel ranking. In True Retrieval, $K = 8$ represented the number of Arabic examples retrieved by Arabic Triplet Matryoshka and BM25. These results should therefore not be interpreted as a single-variable ablation of K .

3-10-3 Combined Broad and Specific System

The strongest broad predictions were produced by the class-balanced V10 system, whereas Specific RRF with Top-20 produced stronger fine-grained predictions. Combining these stages increased H-MF1 from 0.441909 to 0.454655.

The combined system retained the V10 Broad Macro F1 of 0.622465 and Broad Accuracy of 0.715358. Specific Macro F1 increased from 0.261353 to 0.286846, while Specific Accuracy increased from 0.288002 to 0.322430.

This result confirmed that separate aggregation mechanisms were more suitable for the two hierarchy levels than one shared fusion method.

3-10-4 Effect of Selective Qwen

Adding Selective Qwen increased H-MF1 from 0.454655 to 0.455939. Broad Macro F1 increased from 0.622465 to 0.624961, and Broad Accuracy increased from 0.715358 to 0.718611.

Specific Macro F1 changed only slightly, from 0.286846 to 0.286916, while Specific Accuracy increased from 0.322430 to 0.327578.

The H-MF1 improvement produced by Qwen was 0.001283. Its contribution was therefore positive but limited, indicating that most of the final improvement originated from retrieval redesign and class-balanced aggregation rather than from language-model intervention.

3-10-5 Final Official Result

Table 3.21 summarizes the principal systems evaluated after introducing class-balanced retrieval.

Table 3.21: Selected official results from the final system stage.

System	H-MF1	Broad Macro F1	Broad Acc.	Specific Macro F1	Specific Acc.
Class-balanced V10	0.441909	0.622465	0.715358	0.261353	0.288002
Broad V10 + Specific RRF	0.454655	0.622465	0.715358	0.286846	0.322430
Final Selective Qwen	0.455939	0.624961	0.718611	0.286916	0.327578

The final official system additionally achieved a Broad Weighted F1 of 0.718575 and a Specific Weighted F1 of 0.308048.

Overall, H-MF1 increased from 0.178453 for the initial direct E5 final-test submission to 0.455939 for the selected final system. The improvement was associated primarily with broad-first classification, Arabic prototype retrieval, class-balanced broad aggregation, and independent Specific Weighted RRF.

The final Broad Macro F1 of 0.624961 remained substantially higher than the Specific Macro F1 of 0.286916. This difference is analyzed in Section 3-11-5, rather than being attributed solely to the performance of the broad classifier.

3-11 Error Analysis, Discussion, and Limitations

3-11-1 Scope and Restrictions of the Error Analysis

Qualitative error analysis was restricted to the labeled development set. The final-test reference labels were hidden, and only aggregate official metrics were available. It was therefore not possible to construct a reliable final-test confusion matrix or determine which individual final-test predictions were incorrect.

Development-set errors are used to examine the behavior of the classification components, but they are not treated as a representative sample of final-test errors. The substantial difference between development and final-test performance indicates that the small development set did not fully represent the larger evaluation distribution.

The predicted final-test label distribution was examined only to detect severe prediction collapse. It must not be interpreted as the true distribution of the test data. Without the hidden reference labels, the analysis cannot reliably determine:

- the most frequent final-test error types;
- the specific label pairs with the greatest confusion;
- the relationship between correctness and text length, dialect, or noise;
- the number of cases with a correct broad prediction and an incorrect specific prediction;
- or
- the genres that benefited most from each retrieval channel.

Accordingly, no example-level claims are made about final-test errors. All qualitative examples discussed in this section are explicitly derived from the labeled development set.

3-11-2 Communicative Function and Surface Cues

The development error patterns reported in Table 3.12 showed that some classification errors resulted from surface cues shared by communicatively different genres.

Direct advice expressions appeared in some Motivational Writing texts, causing confusion with Advice Columns. Similarly, formulas and formal language in Educational Explanations sometimes favored Analytical Reports. These cases demonstrate that vocabulary and local structure may suggest one genre while the overall communicative purpose corresponds to another.

This behavior reflects the difference between genre and topic classification. The system must identify the function performed by the text, not only its subject matter or most distinctive lexical features. Combining semantic and lexical evidence reduced dependence on one type of signal but did not eliminate ambiguity between genres with closely related communicative boundaries.

3-11-3 Reranking Errors

Development analysis also showed that unrestricted language-model reranking did not improve every retrieval decision. In one documented case, E5 produced the correct Advice Columns prediction, but Qwen changed it to the semantically related Motivational Writing label.

This case demonstrates the risk of allowing a generative model to override a strong retrieval decision without additional evidence. The final system therefore restricted Qwen to uncertain cases and accepted the runner-up candidate only when it was supported by at least two internal retrieval channels.

The resulting policy treats Qwen as a conservative correction layer rather than the primary classification mechanism.

3-11-4 Broad-to-Specific Error Propagation

Hard gating guarantees hierarchical consistency between broad and specific predictions. However, it also prevents recovery from an incorrect broad decision. If the predicted broad genre is wrong, the correct specific genre is removed from the candidate space before specific retrieval begins.

Specific errors may therefore originate from two distinct sources:

1. an incorrect broad prediction that excludes the correct specific genre; or
2. an incorrect distinction between semantically related specific genres within the correct broad category.

Improving specific retrieval alone cannot overcome the upper bound imposed by broad classification. At the same time, improving the broad stage does not guarantee strong hierarchical performance if the system remains unable to distinguish between specific children of the correct parent.

Future work may investigate soft gating or retaining multiple broad candidates. Such approaches could improve recovery from broad-level errors but would increase the candidate space and could introduce hierarchically inconsistent predictions.

3-11-5 Specific-Level Classification Difficulty

Specific classification remained more difficult than broad classification. The broad stage distinguishes between six general functional families, whereas the specific stage distinguishes between 74 labels, several of which share similar vocabulary, topics, and stylistic characteristics.

The difficulty is particularly significant within large broad branches. Informative contains 42 specific genres and therefore requires substantially more fine-grained distinctions than smaller branches such as Legal.

The final Specific Weighted F1 was higher than Specific Macro F1. This difference indicates that performance was not uniform across the specific label space. The system performed relatively

better on larger or easier classes, while less frequent or semantically difficult genres remained weaker.

3-11-6 System Limitations

The final AraGenre system is subject to the following limitations:

- Generated prototypes may not cover the complete diversity of naturally occurring Arabic genre expressions.
- The generated texts may contain stylistic patterns associated with the model used to create them.
- The exact ChatGPT version used during prototype generation was not recorded, which limits exact regeneration of the same prototype collection.
- Fusion weights, candidate limits, and selection thresholds were chosen manually under a limited official-submission budget.
- Although the hidden final-test labels and example-level errors were unavailable, aggregate Codabench scores were used to compare a limited number of successive configurations. The final evaluation process may therefore include a limited degree of leaderboard-guided adaptation.
- The Arabic model, translated definitions, and class-balanced aggregation were introduced together in V10. Their individual contributions cannot therefore be isolated from that submission.
- Hard gating prevents the specific stage from recovering when the broad prediction is incorrect.
- Selective Qwen introduces additional computational cost and prompt sensitivity, despite being restricted to uncertain cases.
- Input-length limits may truncate long texts and remove passages containing useful genre evidence.
- The system has not yet been evaluated on a separately labeled collection of governmental documents from the Wathiq application domain.

3-11-7 Reproducibility Limitations

The final model names, fusion weights, input limits, batch sizes, candidate limits, RRF constant, and independent-model were documented. Nevertheless, exact reproduction may still be affected by:

- library and dependency versions;
- hardware and numerical-computation differences;
- language-model service and inference configurations;
- the absence of complete metric files for some rejected experiments; and

- the fact that the implementation and generated prototype collection were not publicly released at the time of submission.

Only results preserved in official evaluation records or complete experiment outputs are reported. Metrics are not reconstructed for experiments whose detailed outputs were not retained.

3-11-8 Discussion

The largest performance improvement did not result from increasing model size or applying Qwen to a larger number of documents. Instead, it resulted primarily from restructuring the classification process.

The experimental progression indicates that separating broad and specific classification, constructing diverse Arabic prototypes, balancing evidence at the class level, and using different fusion mechanisms at the two hierarchy levels were more effective than limited score calibration or increases in the global neighbourhood size.

The final retrieval channels provided complementary forms of evidence: cross-lingual definition matching, Arabic example-based semantic retrieval, and lexical similarity. Weighted RRF allowed the specific-stage rankings to be combined without assuming that raw scores from different retrieval models shared a common numerical scale. Selective Qwen provided only a small additional improvement, confirming that the principal gains originated from retrieval redesign and class-balanced aggregation.

Despite these improvements, the difference between the final Broad Macro F1 of 0.624961 and Specific Macro F1 of 0.286916 demonstrates that fine-grained genre discrimination remains the main unresolved difficulty. Future improvements should therefore focus on representing the boundaries between closely related specific genres more effectively, rather than only increasing the number of prototypes or retrieved candidates.

3-12 Integration with Wathiq

The AraGenre classifier was developed and evaluated as an independent research component. Its integration into Wathiq is therefore presented as a proposed extension that must be validated on representative documents from the operational environment before deployment. The general architectural and service-engineering principles of Wathiq were discussed in Chapter 2; this section focuses specifically on the information exchanged between AraGenre and the document-management workflow.

3-12-1 Position in the Document Workflow

AraGenre is intended to operate after the OCR stage, once the textual content of a document has been extracted and validated. AraSeg may also process the extracted text independently; consequently, the two components do not need to execute sequentially and may be invoked in parallel when the system infrastructure permits.

The proposed AraGenre processing sequence is as follows:

1. Wathiq extracts the document text through the OCR component.
2. The backend verifies that the extracted text is non-empty and predominantly Arabic.
3. The text is submitted to the AraGenre classification service.
4. The service predicts the broad genre and then the corresponding specific genre.
5. The predicted labels, model information, and classification scores are returned to Wathiq.
6. The results are stored as proposed metadata and, when necessary, forwarded for human review.

This separation allows the classification model to be updated independently of the OCR and document-storage components. It also prevents a classification failure from interrupting document ingestion or causing the extracted content to be lost.

3-12-2 Service Input and Validation

A classification request should contain the document identifier, the OCR-extracted text, the detected language, and the version of the processing request. An institutional or organizational identifier may also be included when required for traceability and access control.

The AraGenre service should receive the text through a controlled application interface rather than accessing the document database or stored files directly. Before submitting a request, the Wathiq backend should verify that:

- the extracted text is not empty;
- the text contains sufficient Arabic content;
- the request does not exceed the permitted input size;
- the requesting user or service is authorized; and
- the same processing request has not already been completed.

These checks reduce unnecessary model calls and preserve a clear separation between document management and intelligent text processing.

3-12-3 Output and Stored Metadata

The classification response should preserve both the model prediction and any subsequent human decision. Table 3.22 presents the principal metadata fields proposed for integration.

The retrieval and fusion scores produced by E5, Arabic Triplet Matryoshka, BM25, and Weighted RRF are ranking scores rather than inherently calibrated probabilities. They should therefore not be displayed as confidence percentages unless a separate calibration study is conducted. For operational use, the system may instead derive qualitative confidence levels, such as *high*, *medium*, and *low*, from validation-based thresholds or from the score margin between the leading candidates.

Table 3.22: Proposed AraGenre metadata stored in Wathiq.

Field	Purpose
predictedBroadGenre	Broad genre predicted by the classification system.
predictedSpecificGenre	Specific genre predicted within the selected broad genre.
broadScore	Score associated with the broad-genre decision.
specificScore	Score associated with the specific-genre decision.
classificationMethod	Identifier of the classification pipeline or configuration used.
modelVersion	Version of the deployed model and its associated resources.
processedAt	Date and time at which classification was performed.
reviewStatus	Current review state, such as pending, accepted, or corrected.
approvedBroadGenre	Broad genre approved or corrected by an authorized reviewer.
approvedSpecificGenre	Specific genre approved or corrected by an authorized reviewer.
reviewedBy and reviewedAt	Identity of the reviewer and the time of the review action.

3-12-4 Human Review

The predicted genre should initially be treated as proposed metadata rather than an irreversible system decision. Human review is particularly appropriate when:

- the margin between the highest-ranked candidates is small;
- the retrieval channels produce conflicting predictions;
- the selective Qwen reranker is unavailable or does not resolve the uncertainty;
- the OCR output is unusually short or noisy;
- the document belongs to a sensitive administrative context; or
- the predicted genre conflicts with existing manually assigned metadata.

To preserve hierarchical consistency, the review interface should restrict the available specific genres to those belonging to the selected broad genre. The original model prediction must remain stored even when it is corrected, allowing later auditing and comparison between automated and human decisions.

Reviewer corrections should not be incorporated automatically into model training. Instead, they should be collected in a controlled, versioned dataset that can be inspected before being used in future evaluation or retraining experiments.

3-12-5 Search and Document Organization

Once reviewed or accepted, genre metadata can enhance Wathiq’s search and organization functions. Users may filter documents by broad or specific genre and combine these filters with existing metadata, such as institution, department, date, or document source. Genre labels

may also support document grouping, collection-level statistics, and the selection of suitable downstream language-processing operations.

However, textual genre must not be interpreted as a replacement for administrative document type, confidentiality level, access permissions, or security classification. AraGenre describes the communicative function and linguistic form of the text, whereas Wathiq’s administrative metadata serves different organizational and security purposes.

3-12-6 Traceability and Failure Handling

Each classification result should be associated with the model version, processing configuration, and execution time. This information allows Wathiq to identify documents processed by a particular model, compare outputs across versions, and selectively reprocess older documents after an approved system update.

If classification fails, the document and its OCR output should remain available. The AraGenre status may be marked as *pending* or *failed*, and the classification request can be retried without repeating OCR. Maintaining the processing history ensures that previous predictions are not silently overwritten and supports subsequent auditing.

3-12-7 Validation Before Operational Use

Performance on the AraGenre shared-task dataset does not guarantee equivalent performance on the governmental and administrative documents managed by Wathiq. Before operational deployment, the integrated component should therefore be evaluated on a manually labelled sample of representative Wathiq documents.

The validation process should examine:

- broad- and specific-genre classification performance;
- the effect of OCR errors on classification;
- performance across short, long, and noisy documents;
- the suitability of confidence or review thresholds;
- the proportion of documents requiring human review;
- processing latency and computational requirements; and
- the practical value of genre metadata for search and organization.

Some shared-task genres may not correspond directly to the administrative categories used by Wathiq. Consequently, AraGenre should initially be introduced as supporting linguistic metadata. Any mapping to internal document categories must be designed and validated separately using domain-specific evidence.

3-13 Chapter Summary

This chapter presented the development and evaluation of a hierarchical Arabic text genre-classification component intended as a research extension to Wathiq. The investigation began with the reproduction of the official AraGenre baselines and progressed through task-specific text-definition matching, hierarchical prediction, Arabic prototype construction, and retrieval-based classification.

The development experiments produced strong results on the small labeled development set. However, direct transfer of the E5 retrieval component to the substantially larger final-test distribution resulted in a considerable performance reduction. Because the official development system included Qwen reranking while the first final-test submission used direct E5 Top-1 prediction, these scores were not treated as a strict comparison between identical systems. Nevertheless, the results revealed a clear limitation in the generalization of the development-stage representations.

The final system combined multilingual E5 retrieval, Arabic Triplet Matryoshka representations, and BM25 lexical matching. Class-balanced score fusion was used for broad-genre prediction, while hard hierarchical gating and Weighted Reciprocal Rank Fusion were applied to specific-genre selection. Selective Qwen reranking was restricted to uncertain broad-genre cases. The resulting system achieved an official Hierarchical Macro F1 of 0.455939.

Broad-genre classification remained substantially more reliable than specific-genre classification, indicating that fine-grained distinction between semantically related genres is the principal remaining research challenge. The proposed Wathiq integration preserves AraGenre as an independent and traceable service and stores its outputs as reviewable linguistic metadata. Operational adoption nevertheless requires additional evaluation on representative OCR-extracted governmental documents from the Wathiq domain.

Chapter 4

AraSeg: Arabic Sentence Segmentation

4-1 Introduction and Relationship to Wathiq

The previous phase of the Wathiq project focused on converting document images into searchable digital text through Optical Character Recognition (OCR). Although this process enables document storage, indexing, and retrieval, the extracted text does not always preserve a reliable sentence structure.

OCR output may contain missing punctuation, incorrect punctuation, line-break inconsistencies, recognition errors, and long blocks of continuous text. As a result, a document may be successfully digitized while still remaining difficult to process at the sentence level. This limitation affects both human readability and later Natural Language Processing (NLP) operations.

Sentence segmentation addresses this problem by determining where individual sentences begin and end. Instead of treating the complete OCR output as a single unstructured sequence, a segmentation system transforms the text into smaller and linguistically meaningful units.

For Wathiq, this capability is particularly important because sentence-level structure can improve several later operations. Search results can present more precise contextual snippets, relevant sentences can be indexed independently, extracted information can be linked to its surrounding context, and future components such as summarization or question answering can operate on more meaningful units of text.

Based on these requirements, the AraSeg shared task was selected as the second research extension of Wathiq. AraSeg studies Arabic sentence segmentation under multiple structural conditions, including settings in which punctuation or paragraph information is unavailable. These conditions are closely related to the difficulties that may occur in OCR-extracted Arabic documents.

In this project, AraSeg is therefore investigated both as a research task and as an intelligent component that can subsequently be integrated into the Wathiq processing pipeline after OCR. This chapter presents the task definition, datasets, evaluation protocol, baseline systems, experimental development process, final systems, error analysis, and software integration approach.

4-2 AraSeg 2026 Shared Task, Dataset, and Evaluation

4-2-1 Task Definition and Sentence-Boundary Representation

AraSeg 2026 formulates Arabic sentence segmentation as a binary classification problem at the token level. Given an Arabic document represented as an ordered sequence of tokens

$$X = (x_1, x_2, \dots, x_n), \quad (4.1)$$

the system predicts one binary label for every token:

$$y_i = \begin{cases} 1, & \text{if a sentence ends after token } x_i, \\ 0, & \text{otherwise.} \end{cases} \quad (4.2)$$

The objective is therefore not to generate new text or restore punctuation. Instead, the system determines whether each original input token represents a sentence boundary.

This formulation is particularly suitable for integration with Wathiq because the predicted boundaries can be mapped directly back to the OCR-extracted tokens. The original text can then be divided into sentence units without rewriting or replacing its content. The token-level formulation also makes exact alignment important. Transformer models internally divide some words into multiple subtokens. However, the AraSeg task requires exactly one prediction for every original task token. The experimental pipeline must therefore preserve the relationship between original tokens, Transformer subtokens, and final sentence-boundary labels.

4-2-2 The Four Closed-Track Conditions

A central characteristic of AraSeg is that sentence segmentation is evaluated under four different combinations of punctuation and paragraph information. These conditions allow the experiments to measure how strongly a segmentation system depends on explicit structural cues.

Table 4.1 summarizes the four conditions used in this work.

Table 4.1: AraSeg closed-track conditions used in this project.

Track	Punctuation	Paragraph Information	Main Difficulty
PA	Available	Available	Both punctuation and paragraph structure can support boundary prediction.
NoPnx-PA	Removed	Available	Sentence boundaries must be inferred without punctuation while paragraph structure remains usable.
NP	Available	Removed	Punctuation remains visible, but paragraph-boundary information cannot be used.
NoPnx-NP	Removed	Removed	The system must infer boundaries primarily from linguistic context without punctuation or paragraph cues.

The four conditions form a useful progression in difficulty. PA contains the greatest amount of explicit structural information, whereas NoPnx-NP removes both major sources of surface-level boundary evidence.

This design is especially relevant to Wathiq. OCR quality is not uniform across documents. Some extracted texts preserve punctuation and line structure relatively well, whereas others may lose one or both of these signals. Studying all four conditions therefore provides a more realistic understanding of how sentence segmentation behaves under different levels of structural degradation.

Relationship to OCR Output

The four AraSeg conditions should not be interpreted as exact simulations of OCR errors. Real OCR may additionally contain character substitutions, deleted words, merged tokens, incorrect reading order, or partially preserved punctuation. Nevertheless, the NoPnx and no-paragraph conditions provide controlled experimental settings for studying two important forms of structural information loss that are directly relevant to OCR-extracted documents.

4-2-3 Dataset and Experimental Use

The AraSeg shared task provides separate data resources for training, development, public evaluation, and final blind evaluation. A strict separation between these resources was maintained during the experiments.

The official training partition contains 174 documents. These documents constitute the supervised training source used for fine-tuning the closed-track models. The development partition contains 222 documents and was used for checkpoint selection, threshold calibration, ensemble design, document-level error analysis, and comparison between experimental configurations. A separate released test partition contains 262 documents. This partition was used during the public development phase of the shared task and must not be confused with the later official blind evaluation set.

The final blind-test data were released separately for the official testing phase. Their reference labels were unavailable to the project team. Blind data were used only after an experimental configuration had been selected and frozen using development evidence.

Table 4.2 summarizes the role of the principal data partitions.

Table 4.2: AraSeg data partitions and their experimental use.

Data Partition	Documents	Experimental Use
Official Train	174	Used as the supervised training data for closed-track fine-tuning experiments.
Development	222	Used for checkpoint selection, probability analysis, threshold calibration, ensemble selection, error analysis, and controlled held-out comparisons.
Released Test	262	Used during the public development phase of the shared task; distinct from the later official blind-test set.
Official Blind Test	Hidden evaluation set	Used only for frozen final inference and official Codabench evaluation. Reference labels were not available to the team.

The number of tokens is not identical across all four task variants because punctuation removal and paragraph representation modify the visible token sequence. Consequently, experimental outputs were validated separately for every track rather than assuming a single shared token count.

This distinction became especially important during ensemble construction. Probability files from different models were accepted only when their document identifiers, token identifiers, and token ordering were correctly aligned with the corresponding task variant.

4-2-4 Closed-Track Experimental Protocol

The experiments documented in this chapter focus on the closed-track setting. Under this protocol, the supervised task data used for model training are restricted to the official training partition.

Development labels are used only for evaluation and model-selection decisions. These decisions include:

- selecting the best checkpoint;
- calibrating the sentence-boundary threshold;
- selecting probability-ensemble weights;
- comparing window and context configurations;
- analyzing false-positive and false-negative boundaries;
- evaluating experimental variants on held-out document subsets; and
- deciding whether a candidate is sufficiently stable to justify blind inference.

Blind reference labels are not used during training, threshold selection, or model design. Once a final configuration is selected, its model weights, preprocessing rules, ensemble weights,

threshold, and inference behavior are frozen before generating the corresponding blind prediction file.

This separation became an important methodological principle during the later AraSeg experiments. A small improvement on the complete development set was not considered sufficient by itself. When a candidate differed only slightly from the current system, its behavior was additionally examined across held-out document subsets to reduce the risk of selecting an unstable improvement.

4-2-5 Arabic Sentence Boundary Challenges

Arabic sentence segmentation is more complex than searching for a period or another sentence-ending punctuation mark. The relationship between punctuation and actual sentence boundaries is not perfectly deterministic, and two of the AraSeg conditions explicitly remove punctuation from the input.

Several sources of difficulty were relevant during the experimental development.

4-2-5-1 Missing Punctuation

When punctuation is removed, the model cannot rely directly on periods, question marks, or other explicit sentence-ending markers. The decision must instead be inferred from lexical, syntactic, semantic, and contextual transitions between neighboring tokens.

This challenge is central to the NoPnx-PA and NoPnx-NP conditions and is also directly relevant to OCR documents in which punctuation may be omitted during extraction.

4-2-5-2 Ambiguous Punctuation

The presence of punctuation does not always imply the end of a sentence. Punctuation marks may occur inside quotations, numerical expressions, abbreviations, titles, or other constructions.

A model that converts every punctuation mark into a boundary can therefore obtain high recall while also introducing false-positive sentence breaks.

4-2-5-3 Paragraph Structure

Paragraph boundaries provide a strong structural cue because a sentence frequently ends before a paragraph transition. However, this information is unavailable in the NP and NoPnx-NP conditions.

The experiments therefore distinguish carefully between learned boundary probabilities and deterministic paragraph-related rules. This separation allows the contribution of explicit document structure to be evaluated independently.

4-2-5-4 Long Documents and Limited Context Windows

Transformer encoders process sequences with a bounded maximum input length. AraSeg documents may contain more tokens than a model can process in a single forward pass.

The system must therefore divide long documents into overlapping windows, infer probabilities within each window, and merge repeated predictions without losing the original token order. The selected window size and stride can influence both context availability and computational cost.

4-2-5-5 OCR-Generated Noise

The previous Wathiq evaluation showed that punctuation and recognition errors remain present after OCR. Such errors may remove an expected boundary cue or modify words that provide contextual evidence for a boundary.

For this reason, the AraSeg experiments are particularly useful as a research foundation for Wathiq, although the shared-task data should not be treated as a complete substitute for direct evaluation on real OCR-extracted governmental documents.

4-2-6 Official Evaluation Metrics

AraSeg evaluates sentence-boundary prediction using Precision, Recall, and F1-score. The positive class corresponds to tokens after which a sentence boundary occurs.

Let TP denote correctly predicted boundaries, FP predicted boundaries that are not present in the reference labels, and FN reference boundaries that the system fails to predict.

Boundary Precision is calculated as

$$P = \frac{TP}{TP + FP}. \quad (4.3)$$

Precision measures the reliability of predicted sentence boundaries. A low precision value indicates that the model introduces too many false sentence breaks.

Boundary Recall is calculated as

$$R = \frac{TP}{TP + FN}. \quad (4.4)$$

Recall measures how many true sentence boundaries are recovered by the system. A low recall value indicates that sentences are frequently merged because their boundaries are missed.

The F1-score is the harmonic mean of Precision and Recall:

$$F1 = 2 \times \frac{P \times R}{P + R}. \quad (4.5)$$

The official evaluation computes the metrics at the document level and then averages them across documents. This detail is important because a configuration that performs strongly on a few long documents should not dominate the evaluation solely because those documents contain more tokens.

Accordingly, the development pipeline reproduced the document-level evaluation procedure when comparing thresholds, ensemble configurations, and later experimental variants.

Model-Selection Principle

The final AraSeg systems were not selected solely according to the highest full-development F1-score. When improvements were small, the team also examined stability across held-out documents, changes in Precision and Recall, reproducibility of the inference configuration, and whether the proposed modification could be reconstructed from frozen model and probability artifacts.

4-3 Official Baselines and Initial Reproduction

The experimental development of the AraSeg component began by studying two complementary reference systems: Segment Any Text (SaT) as an off-the-shelf multilingual segmenter and CAMeLBERT-MSA as the main trainable Arabic contextual encoder.

The purpose of this stage was not immediately to construct the final shared-task system. Instead, it aimed to establish a reproducible experimental pipeline, verify token-to-boundary conversion, validate the official evaluation procedure, and determine which type of model provided the strongest foundation for later improvements.

The two systems are methodologically different. SaT performs general-purpose sentence segmentation without AraSeg-specific fine-tuning, whereas CAMeLBERT-MSA is fine-tuned directly for the binary token-classification formulation of the task.

4-3-1 Related Work and Reference Systems

Sentence segmentation is a fundamental preprocessing task in Natural Language Processing because downstream components frequently assume that input text has already been divided into coherent sentence units. Traditional segmentation approaches often depend strongly on explicit punctuation or manually designed boundary rules. Such assumptions become unreliable when punctuation is missing, inconsistent, or corrupted.

Recent research has therefore investigated sentence segmentation that is less dependent on surface punctuation. Minixhofer et al. [25] introduced *Where’s the Point?* (WtP), a multilingual punctuation-agnostic approach that learns sentence segmentation from weak structural signals and can be adapted to new segmentation conventions using a relatively small amount of labeled data. This work demonstrated that sentence segmentation can be modeled without treating punctuation as a mandatory boundary signal.

Frohmann et al. [26] subsequently introduced *Segment Any Text* (SaT). SaT extends this direction by targeting robustness to missing punctuation, adaptation to different domains, and efficient multilingual inference. These properties make SaT particularly relevant to AraSeg conditions in which punctuation or formatting information is unreliable.

Arabic sentence segmentation additionally benefits from language-specific contextual representations. CAMeLBERT was introduced as part of a study of Arabic pretrained language models covering Modern Standard Arabic, dialectal Arabic, Classical Arabic, and mixed Arabic

varieties [27]. The study showed that the relationship between the pretraining language variety and the target task can be important when selecting an Arabic pretrained encoder. Because the AraSeg documents are primarily processed as Arabic contextual sequences, CAMELBERT-MSA provides a natural trainable foundation for token-level sentence-boundary classification.

A second Arabic encoder family relevant to this work is AraELECTRA. Unlike BERT-style masked-language-model pretraining, AraELECTRA uses a replaced-token detection objective in which a discriminator learns to distinguish original tokens from generated replacements [28]. The difference in pretraining objective makes AraELECTRA methodologically relevant as a source of backbone diversity rather than merely another random initialization of the same CAMELBERT architecture.

The AraSeg benchmark itself was introduced by Elkholy et al. [24] to study Arabic sentence segmentation across genres and under different punctuation and document-structure conditions. The study evaluates multiple classes of segmentation systems and shows that the difficulty of Arabic sentence segmentation changes substantially when explicit structural cues are unavailable. This benchmark provides the direct research context for the four conditions investigated in this thesis.

Within the AraSeg reference experiments, the organizers evaluated rule-based and neural sentence-segmentation systems, including SpaCySENT, PySBD, Punkt-based segmentation, Ersatz, SaT, and task-specific Transformer encoders [24]. Among the principal reference systems considered in this thesis, SaT represents a strong off-the-shelf multilingual segmenter, whereas CAMELBERT-MSA represents a trainable Arabic-specific contextual encoder.

Table 4.3 summarizes the published Boundary F1 values of these two principal reference systems.

Table 4.3: Published organizer reference results across the four AraSeg conditions.

System	NoPnx-NP	NoPnx-PA	NP	PA	Average
SaT	53.0	63.7	61.8	66.8	61.3
CAMELBERT-MSA	85.9	88.4	93.2	95.2	90.7

Source: AraSeg organizer reference experiments [24].

The reference results provide two complementary starting points. SaT allows the experimental pipeline to be evaluated using an off-the-shelf segmenter specifically designed for difficult punctuation conditions, whereas CAMELBERT-MSA provides a substantially stronger task-specific Arabic encoder that can be fine-tuned and combined with additional contextual models.

The experiments presented in the following sections therefore do not attempt to reproduce the complete literature on sentence segmentation. Instead, they use these published approaches to establish the research context and then investigate track-specific calibration, transfer, context diversity, ensemble design, and Arabic backbone diversity.

Relationship to Previous Work

The AraSeg experiments in this thesis build on two complementary research directions: punctuation-robust multilingual sentence segmentation, represented by WtP and SaT, and Arabic-specific pretrained contextual encoders, represented principally by CAMELBERT-MSA and AraELECTRA. The contribution of the present work is not the introduction of these pretrained architectures themselves, but their controlled adaptation, calibration, combination, and evaluation under the four AraSeg structural conditions, followed by integration of the selected segmentation pipeline into Wathiq.

4-3-2 Segment Any Text Baseline

Segment Any Text (SaT) was selected as the first reproducible off-the-shelf baseline. SaT is a multilingual sentence segmentation model designed to operate even when punctuation and formatting are not completely reliable.

The reproduced baseline used a pretrained SaT model through the `wtpsplit` implementation. The model was applied as an off-the-shelf sentence segmenter without AraSeg-specific fine-tuning.

The organizer study also reports results for the SaT model family. Because the exact organizer checkpoint is not specified in the available task description, the organizer result is treated as a published reference point rather than as an exact checkpoint-level reproduction.

No AraSeg-specific fine-tuning was applied to SaT. Instead, the original AraSeg tokens were joined to reconstruct a document-level text sequence. SaT then generated sentence-boundary probabilities at the character level.

To map these probabilities back to the shared-task representation, the boundary probability associated with the final character of each original AraSeg token was selected. The resulting probability was converted into a binary decision using a threshold of 0.25.

The reproduced SaT pipeline can therefore be summarized as

AraSeg tokens → reconstructed document text → character-level SaT probabilities → final character of each token → threshold → binary boundary labels.

The threshold of 0.25 corresponded to the default behavior of the model used in the experiment and was verified against the boundaries returned by the standard SaT splitting operation.

The final token of each document was treated as the end of the document segment. Because SaT required no task-specific training, it provided a useful reference for validating preprocessing, alignment, probability extraction, and evaluation before introducing supervised fine-tuning.

4-3-3 CAMELBERT-MSA Trainable Baseline

CAMELBERT-MSA was selected as the principal supervised baseline. It is an Arabic-specific pretrained Transformer encoder designed for Modern Standard Arabic and provides a suitable contextual representation for token-level classification.

For AraSeg, CAMELBERT-MSA was adapted as a two-class token classifier. Each original task token may be divided by the tokenizer into multiple Transformer subtokens. To preserve the original task formulation, the sentence-boundary label was assigned only to the final visible subtoken representing each original token. Other subtokens were ignored in the training loss.

After contextual encoding, a two-class classification head produced boundary logits. Long documents were processed using overlapping windows because their token sequences could exceed the Transformer context limit.

The initial reproduced configuration used:

- maximum sequence length of 512;
- stride of 64;
- five training epochs;
- learning rate of 3×10^{-5} ;
- full fine-tuning of the encoder and classification head;
- AdamW optimization;
- physical batch size of 2;
- 16 gradient-accumulation steps, producing an effective batch size of 32; and
- averaging of repeated predictions for tokens appearing in overlapping windows.

The initial baseline study focused on the NoPnx-NP condition because it removes both punctuation and paragraph information and therefore provides the most challenging setting for evaluating contextual sentence-boundary prediction.

For this condition, the official training partition contained 174 documents and 98,457 original tokens. After window construction, this produced 362 overlapping training windows.

The development partition contained 222 documents and 127,722 original tokens and produced 464 overlapping windows under the same configuration.

The resulting processing pipeline was

AraSeg tokens → CAMELBERT tokenizer → subtoken alignment → overlapping windows → CAMELBERT-MSA → binary classification head → merged probabilities → one boundary label per original token.

4-3-4 Reproduced NoPnx-NP Baseline Results

Both reference systems were evaluated independently on the labeled NoPnx-NP development set using the document-level evaluation procedure.

The evaluation implementation was checked using the shared-task evaluation script. During this verification, a presentation-level Precision/Recall labeling issue in an earlier experimental script was identified; the corrected values shown in Table 4.4 are the values retained in the verified experimental record.

Table 4.4: Reproduced baseline results on the NoPnx-NP development set.

System	Precision	Recall	Macro F1
SaT off-the-shelf	0.8822	0.4323	0.5257
CAMeLBERT-MSA	0.8060	0.7556	0.7699

The results reveal substantially different error profiles. SaT achieved high Precision but low Recall. It was therefore conservative: most boundaries that it predicted were correct, but it failed to recover a large proportion of the reference sentence boundaries.

CAMeLBERT-MSA produced a much more balanced Precision–Recall profile and increased Macro F1 from 0.5257 to 0.7699. This result confirmed that task-specific Arabic contextual fine-tuning was a much stronger foundation for the difficult NoPnx-NP condition.

The purpose of this reproduction was not to reproduce the organizer reference values numerically under every condition. Instead, it was to establish a verified participant-side baseline using the official training and development data, correct token alignment, overlapping window inference, and the shared-task evaluation procedure.

4-3-5 Initial CAMeLBERT–SaT Probability Ensemble

After reproducing both baselines independently, the first improvement experiment investigated whether their different error profiles were complementary.

Instead of choosing between the two hard predictions, their sentence-boundary probabilities were combined before thresholding. For token x_i , the ensemble probability was defined as

$$p_i^{\text{ens}} = \alpha p_i^{\text{CAMeLBERT}} + (1 - \alpha) p_i^{\text{SaT}}, \quad (4.6)$$

where α controls the contribution of the supervised CAMeLBERT model.

A development grid search examined 1,701 weight and threshold configurations. CAMeLBERT weights were varied from 0.00 to 1.00 in increments of 0.05, while decision thresholds were varied from 0.10 to 0.90 in increments of 0.01.

The selected development configuration used

$$p_i^{\text{ens}} = 0.55 p_i^{\text{CAMeLBERT}} + 0.45 p_i^{\text{SaT}}, \quad (4.7)$$

with a decision threshold of 0.25.

Table 4.5 compares the resulting system with the two independent baselines.

Table 4.5: Initial NoPnx-NP probability-ensemble experiment on the development set.

System	Precision	Recall	Macro F1
SaT	0.8822	0.4323	0.5257
CAMeLBERT-MSA	0.8060	0.7556	0.7699
CAMeLBERT–SaT ensemble	0.7824	0.8104	0.7860

The ensemble improved Macro F1 by approximately 0.0161, or 1.61 F1 points, relative to CAMeLBERT with its default 0.50 decision threshold. The principal change was an increase in Recall from 0.7556 to 0.8104, although Precision decreased from 0.8060 to 0.7824.

A subsequent CAMeLBERT-only threshold sweep showed that probability calibration itself accounted for a substantial part of this gain. CAMeLBERT reached a Macro F1 of 0.7801 at a threshold of 0.32, compared with 0.7699 at the default threshold.

Consequently, approximately 0.0102 absolute F1 of the original improvement can be attributed to calibration, while the addition of SaT provided a further improvement of approximately 0.0059.

A paired document-level bootstrap estimated an approximate 95% confidence interval of [0.0007, 0.0110] for the ensemble gain over threshold-calibrated CAMeLBERT. The experiment therefore provided evidence of a small but positive complementary contribution from SaT, rather than indicating that the complete 0.0161 improvement resulted from model diversity alone.

The selected ensemble also changed the types of errors produced by the system. Relative to CAMeLBERT with its default threshold, it recovered 635 previously missed sentence boundaries but introduced 670 additional false boundaries. Thus, the gain came primarily from increased Recall and not from uniformly improving every prediction.

At the document level, the ensemble improved performance on 140 of the 222 development documents, reduced performance on 58 documents, and left 24 documents unchanged.

These observations demonstrated that SaT contained useful complementary information, but they also showed that the first ensemble should not be interpreted as a universally better boundary predictor.

This configuration was therefore treated as a development-stage experiment rather than as the final NoPnx-NP system. Its weights and threshold were selected on Dev, and the later development process investigated stronger task-specific CAMeLBERT variants, transfer learning, contextual diversity, alternative Arabic encoders, and more rigorous held-out-document validation.

4-3-6 Initial Error-Analysis Findings

The saved baseline outputs also enabled an initial analysis of the behavior of the three systems without additional training.

At the global token level, SaT behaved as a conservative segmenter: it produced relatively few false-positive boundaries but missed a large number of true boundaries. CAMELBERT was more balanced, while the ensemble shifted the system toward higher Recall.

Table 4.6 summarizes the global token-level error counts recorded during this stage.

Table 4.6: Global token-level boundary counts in the initial NoPnx-NP baseline study.

System	True Positives	False Positives	False Negatives	True Negatives
CAMELBERT	9,974	2,693	2,952	112,103
SaT	5,806	1,151	7,120	113,645
Ensemble	10,609	3,363	2,317	111,433

Document length was also examined because long AraSeg documents require overlapping Transformer windows. No monotonic degradation in CAMELBERT document-level F1 was observed as document length increased. The measured Spearman correlation was approximately $\rho = -0.012$ with $p = 0.856$.

Documents longer than 510 original tokens, which necessarily required multiple model windows, did not show a systematic reduction in document-level F1. This provided preliminary evidence that the 512-token, stride-64 overlapping-window strategy handled long development documents without a measurable length-related penalty.

In contrast, SaT performance decreased as gold sentence-boundary density increased. The observed Spearman correlation was approximately $\rho = -0.227$ with $p < 0.001$, which is consistent with its high-Precision, low-Recall under-segmentation behavior.

The baseline error study also identified several representative error patterns. SaT occasionally supplied very strong evidence for a true boundary that CAMELBERT assigned a low probability. In other cases, the lower ensemble decision threshold recovered a boundary primarily through calibration rather than through SaT support.

Heading-like and template-based text structures could also introduce false sentence breaks. These observations suggested that the remaining segmentation errors were not explained by a single factor and motivated more controlled experimentation in the following development stages.

Outcome of the Baseline Stage

The baseline study established several important conclusions. CAMELBERT-MSA was considerably stronger than the off-the-shelf SaT model on NoPnx-NP. SaT nevertheless exhibited a distinctive high-Precision, low-Recall behavior that provided complementary boundary evidence. Probability calibration explained a substantial part of the first ensemble improvement, while SaT contributed a smaller additional gain. Finally, the experiment demonstrated the importance of separating calibration effects from genuine model diversity when interpreting later AraSeg ensembles.

4-4 Common Modeling and Experimental Methodology

After establishing the initial SaT and CAMELBERT baselines, the experimental pipeline was reorganized around a common methodology that could support all four AraSeg conditions while preserving the exact token-level structure required by the shared task.

Although the final systems differ across tracks, they share the same general sequence:

AraSeg document $\rightarrow$ task tokens $\rightarrow$ Transformer tokenization $\rightarrow$ subtoken alignment $\rightarrow$ context windows $\rightarrow$ boundary probabilities $\rightarrow$ probability aggregation $\rightarrow$ threshold calibration $\rightarrow$ track-specific rules $\rightarrow$ one binary label per original token.

The individual models, context lengths, ensemble weights, thresholds, and structural rules were selected separately for each track. This section describes only the methodology shared across the experimental development.

4-4-1 Binary Token-Classification Formulation

The principal supervised systems treat sentence segmentation as two-class token classification. For every original AraSeg token x_i , the model estimates a boundary probability

$$p_i = P(y_i = 1 \mid X), \quad (4.8)$$

where $y_i = 1$ indicates that a sentence ends after token x_i .

The final binary prediction is obtained by applying a decision threshold τ :

$$\hat{y}_i = \begin{cases} 1, & p_i \geq \tau, \\ 0, & p_i < \tau. \end{cases} \quad (4.9)$$

This separation between probability estimation and the final binary decision became important throughout the project. Rather than treating the default 0.50 threshold as fixed, the development process considered threshold calibration as a separate model-selection step.

4-4-2 Original-Token and Subtoken Alignment

AraSeg evaluates exactly one binary prediction for every original task token. Transformer tokenizers, however, may divide one original token into multiple subtokens.

For example, an original token x_i may be represented internally as

$$x_i \rightarrow (s_{i,1}, s_{i,2}, \dots, s_{i,k}). \quad (4.10)$$

The project therefore preserved an explicit mapping between task tokens and Transformer subtokens.

During supervised training, the gold sentence-boundary label of an original token was attached to its final visible subtoken. The other subtokens associated with the same original token were excluded from the token-level training objective.

Conceptually,

$$\text{target}(s_{i,j}) = \begin{cases} y_i, & j = k, \\ \text{ignore}, & j < k. \end{cases} \quad (4.11)$$

The ignored positions were represented using the standard ignored classification target in the corresponding training implementation.

During inference, the output associated with the final visible subtoken was mapped back to the original AraSeg token. This ensured that the number and ordering of predictions remained identical to the number and ordering of task tokens.

Alignment Requirement

Token alignment was treated as a correctness condition rather than only as a preprocessing detail. A model output was used in an ensemble or final evaluation only when document identifiers, token identifiers, token order, and expected token counts matched the corresponding AraSeg condition.

4-4-3 Long-Document Processing

Many AraSeg documents exceed the context length that can be processed by a Transformer in a single forward pass. Long documents were therefore divided into overlapping windows.

Let L denote the maximum sequence length and S the selected stride. Instead of dividing a document into independent, non-overlapping blocks, the next window begins before all tokens from the previous window have been discarded.

Consequently, some original tokens occur in more than one context window. This overlap reduces the risk that a potential sentence boundary is always evaluated near an artificial window edge.

Different experimental families used different values of L and S . For example, the initial CAMeLBERT reproduction used

$$L = 512, \quad S = 64,$$

whereas later track-specific experiments also examined shorter and intermediate context configurations. These values are reported in the corresponding track-development sections rather than being treated as one universal setting.

4-4-4 Merging Predictions from Overlapping Windows

Because overlapping windows may contain the same task token more than once, repeated model outputs must be reduced to one decision value before final thresholding.

In the initial CAMELBERT reproduction, repeated window-level logits for the same original token were averaged before producing the final binary decision.

In later experiments, the aggregated output was represented as one aligned boundary probability for each original task token. These aligned probabilities provided a common interface for threshold calibration, model comparison, and ensemble construction.

If token x_i occurs in m_i windows and the corresponding model-level scores are $z_i^{(1)}, \dots, z_i^{(m_i)}$, the basic aggregation operation can be represented as

$$\bar{z}_i = \frac{1}{m_i} \sum_{j=1}^{m_i} z_i^{(j)}. \quad (4.12)$$

The aggregated value is then converted into or retained as the token-level probability used by subsequent model-selection steps.

This design has an additional software-engineering benefit: later ensemble experiments can operate on verified token-level probability tables without repeatedly loading every neural model.

4-4-5 Probability-Level Ensembling

One of the recurring techniques throughout AraSeg development was probability-level ensembling.

For M component models, the combined boundary probability is

$$p_i^{\text{ens}} = \sum_{m=1}^M w_m p_{i,m}, \quad (4.13)$$

subject to

$$\sum_{m=1}^M w_m = 1. \quad (4.14)$$

Here, $p_{i,m}$ denotes the boundary probability produced by model m for original token i , and w_m denotes its ensemble weight.

Combining probabilities before applying the final threshold provides more information than majority voting over hard labels. A model can therefore contribute useful evidence even when its independent binary decision would be negative.

Probability ensembling was used in several forms during the project:

- combining independently trained instances of the same pretrained encoder;
- combining models trained through different task-transfer strategies;
- combining models with different context-window configurations;
- combining independently pretrained Arabic encoder families; and
- testing whether an off-the-shelf segmentation signal could complement a task-specific model.

Not every ensemble was retained. The final decision depended on development performance, stability, reproducibility, and the complexity added by the new component.

4-4-6 Threshold Calibration

The baseline study demonstrated that the default threshold of 0.50 was not necessarily optimal for sentence-boundary classification. Threshold calibration therefore became a standard component of later experiments.

For a candidate probability stream p , the development procedure evaluates a set of candidate thresholds

$$\mathcal{T} = \{\tau_1, \tau_2, \dots, \tau_k\}$$

and computes document-level Precision, Recall, and F1 for each candidate.

A simple development-time threshold selection can be expressed as

$$\tau^* = \arg \max_{\tau \in \mathcal{T}} F1_{\text{Dev}}(\tau). \quad (4.15)$$

However, the complete experimental decision was not always based only on the highest full-development F1. When differences were small, additional document-level stability analysis was performed before replacing an existing system.

Threshold calibration also allowed the project to examine the Precision–Recall trade-off directly. Lowering a threshold generally recovers more candidate boundaries and may improve Recall, but may also increase false-positive sentence breaks.

This trade-off later became particularly important in PA and NoPnx-NP.

4-4-7 Track-Specific Structural Rules

The availability of explicit structure differs across the four AraSeg conditions. The inference pipeline therefore separates neural boundary probabilities from track-specific structural rules.

For paragraph-aware conditions, paragraph information may be retained and processed explicitly. In particular, later NoPnx-PA development used an explicit paragraph marker and rules associated with paragraph transitions.

For punctuation-aware conditions, punctuation information remains available to the model and may also support calibrated decision rules.

In contrast, punctuation restoration was not introduced as a preprocessing stage for either NoPnx condition. The system predicts sentence boundaries directly from the available token sequence rather than first reconstructing missing punctuation.

Similarly, the decision to force the final token of a document as a sentence boundary was treated as a track-specific inference choice rather than a universal rule. Some selected systems used this constraint, while others did not.

4-4-8 Development-Only Model Selection

A strict separation was maintained between supervised training, development-time model selection, and blind evaluation.

The official training partition was used for model fitting. The development partition was used for:

- checkpoint selection;
- threshold calibration;
- ensemble-weight selection;
- comparison of context-window configurations;
- probability calibration and error analysis;
- examination of Precision–Recall trade-offs; and
- acceptance or rejection of experimental variants.

Blind labels were never used for these decisions.

Once a configuration had been accepted for blind evaluation, the relevant inference recipe was frozen. Depending on the system, the frozen configuration could include:

- model checkpoint or checkpoints;
- tokenizer and preprocessing behavior;
- maximum sequence length and stride;
- probability-ensemble weights;
- decision threshold;
- paragraph-related rules;
- final-boundary behavior; and
- output formatting rules.

The blind stage then consisted of inference only. It did not retrain the model or recalibrate the selected threshold.

4-4-9 Document-Level Held-Out Validation

As development progressed, small full-development improvements became increasingly common. Selecting every configuration with a marginally higher aggregate score would increase the risk of overfitting the development set.

For this reason, later NP and NoPnx-NP experiments introduced document-level held-out comparisons.

The 222 development documents were assigned to fixed document folds. Candidate systems were then compared on documents that had not been used for the corresponding fold-level selection decision.

The objective was not to produce a second official test set. Instead, the held-out procedure provided an additional stability signal for deciding whether an apparent improvement generalized across different groups of documents.

In several later experimental stages, a candidate was required to improve multiple held-out folds before it was allowed to replace the current system. A common requirement in the stricter experiments was to improve at least four of five folds.

The exact minimum F1 gain and allowable Precision or Recall change were not identical for every experiment. They were defined according to the purpose and expected magnitude of each experimental stage. For example, some later NoPnx-NP experiments used explicit minimum held-out gains together with a four-of-five-fold requirement.

Therefore, the general principle was

A more complex candidate should not replace a simpler verified system solely because it produces a small improvement on the complete development set.

This principle became particularly important when evaluating additional encoder families, meta-classifiers, structured decoding, and other late-stage rescue experiments.

4-4-10 Acceptance and Rejection of Experimental Variants

The experimental process distinguished between a promising development result and an accepted system replacement.

A candidate could be rejected for several reasons even when one aggregate score improved:

- the improvement was too small to be practically meaningful;
- the gain appeared in only a minority of held-out folds;
- Precision or Recall deteriorated substantially;
- the candidate added considerable complexity without a stable benefit;

- the improvement could be explained primarily by threshold recalibration rather than the proposed new component;
- the complete inference configuration could not be reconstructed with sufficient confidence; or
- the candidate did not satisfy the predefined development acceptance rule for that experimental stage.

Conversely, an accepted candidate was expected to have a reproducible configuration, correct token alignment, a documented development comparison, and sufficient evidence that it improved or stabilized the current pipeline.

This selection policy explains why several technically successful experiments described later in this chapter were not retained. Successful model training alone was not sufficient to justify replacing an existing system without evidence of a stable improvement.

4-4-11 Reproducibility Controls

Because the AraSeg development process involved multiple models, training configurations, ensemble combinations, and evaluation experiments, reproducibility was treated as an important part of the experimental methodology.

For the principal candidate systems, the project preserved sufficient information to reconstruct the corresponding experimental configuration. This included the selected model checkpoints, training settings, aligned development probabilities, threshold and ensemble configurations, document-level validation results, and the final inference settings.

Before two model outputs were combined, their compatibility was verified at the document and token levels. Document identifiers, token ordering, token counts, and expected prediction counts were checked to ensure that the probability streams represented the same AraSeg input sequence.

This procedure was particularly important when comparing models trained with independent initializations, different context lengths, or transfer strategies.

Reproducibility was also considered during final model selection. A historical evaluation result could be reported as part of the experimental record while still being excluded from the final system description if its complete inference configuration could not be reconstructed with sufficient confidence.

This distinction prevents a numerical result from being presented as the final reproducible system when the exact experimental conditions that produced it are uncertain.

Experimental Selection Principle

Throughout the later AraSeg development, preference was given to stable, reproducible improvements over additional complexity. A candidate that produced a small isolated development gain could be rejected, while a simpler system with consistent held-out behavior and complete experimental provenance could remain the selected configuration.

4-5 PA Track Development

The PA condition retains both punctuation and paragraph information. It therefore provides the richest explicit structural evidence among the four AraSeg conditions.

The development strategy for this condition differed from that of the more difficult punctuation-free settings. Rather than relying mainly on increasingly complex model architectures, PA development focused on adapting a strong Arabic contextual encoder and improving its final boundary decisions through probability calibration and explicit structural information.

The principal investigated directions included track-specific CAMELBERT fine-tuning, transfer initialization, decision-threshold calibration, document-final boundary handling, punctuation-aware thresholding, paragraph-aware post-processing, and a later low-weight two-model ensemble.

4-5-1 Track-Specific CAMELBERT Fine-Tuning

The principal PA predictor was obtained through task-specific fine-tuning of CAMELBERT-MSA.

Instead of initializing the model only from the original pretrained encoder, a strong model previously adapted to the NoPnx-PA condition was used as the initialization point.

This transfer strategy was motivated by the relationship between the two conditions. Both preserve paragraph information, while PA additionally contains punctuation. The transferred model therefore already contained sentence-boundary representations learned in a paragraph-aware setting, and PA fine-tuning allowed these representations to adapt to the additional punctuation cues.

The selected training configuration used:

- CAMELBERT-MSA as the contextual encoder;
- initialization from the strongest available NoPnx-PA checkpoint;
- five PA fine-tuning epochs;
- learning rate of 2×10^{-5} ;
- maximum sequence length of 512;
- stride of 64; and
- overlapping-window inference for long documents.

The model produced one aligned boundary probability for every original PA token.

Because the resulting contextual model was already strong, the next development steps concentrated mainly on calibration and track-specific inference rules rather than replacing the underlying encoder.

4-5-2 Global Threshold Calibration

The initial PA experiments investigated the decision threshold used to convert model probabilities into binary sentence-boundary labels.

As demonstrated by the earlier baseline study, the conventional threshold of 0.50 was not assumed to be universally optimal.

Development-set calibration selected a threshold close to

$$\tau_{\text{PA}} = 0.567. \quad (4.16)$$

A preserved development record associated with this configuration reports approximately

$$P \approx 0.9414, \quad R \approx 0.9429, \quad F1 \approx 0.9422. \quad (4.17)$$

These values are treated as supporting development evidence. The final comparison in this chapter relies primarily on the official blind-evaluation results of the selected PA configurations.

4-5-3 Final-Document Boundary Constraint

Development analysis also examined whether the final token of each document should be explicitly treated as a sentence boundary. The final-document boundary rule was selected using development evidence before blind evaluation.

For completeness, preserved blind-evaluation records are reported here retrospectively for two nearby operating points, 0.567 and 0.548, both with and without the final-document boundary constraint. These results are presented to document the behavior of the corresponding frozen configurations and were not used to tune the threshold or select the boundary rule.

Table 4.7 summarizes the comparison.

Table 4.7: Retrospective blind-evaluation comparison of nearby PA operating points with and without the final-document boundary constraint.

Configuration	P	R	F1	Observation
Threshold 0.567	93.5	93.2	93.1	Without final boundary
Threshold 0.548	93.3	93.3	93.1	Without final boundary
Threshold 0.567 + final boundary	93.5	93.4	93.3	With final boundary
Threshold 0.548 + final boundary	93.3	93.5	93.2	With final boundary

The retrospective comparison shows that changing the global threshold within this narrow interval mainly shifted the Precision–Recall balance. For the 0.567 operating point, adding the final-document constraint was associated with an increase in Recall from 93.2 to 93.4 and in F1 from 93.1 to 93.3.

The subsequent PA experiments accordingly continued to use the final-document boundary rule, consistent with its selection during development.

4-5-4 Punctuation-Aware Dual-Threshold Calibration

Further development analysis indicated that a single global threshold treated all PA tokens too uniformly.

Because punctuation remains available in PA, likely sentence-ending punctuation provides stronger boundary evidence than ordinary lexical tokens. The PA inference rule was therefore extended to use a lower decision threshold for punctuation-supported cases while retaining the calibrated default threshold for other tokens.

The decision threshold was defined as

$$\tau_i = \begin{cases} 0.32, & \text{for likely sentence-ending punctuation,} \\ 0.567, & \text{otherwise.} \end{cases} \quad (4.18)$$

The configuration retained the previously established paragraph-aware processing and final-document boundary handling.

The paragraph-aware processing used in PA should not be assumed to follow the explicit paragraph-marker logic later introduced for NoPnx-PA. The preserved PA configuration documents the use of paragraph-aware processing, but does not provide sufficient evidence to assign the same internal marker rules to both conditions.

After the configuration had been fixed, its preserved blind-evaluation record was

$$P = 93.4, \quad R = 93.9, \quad F1 = 93.4. \quad (4.19)$$

Relative to the preceding frozen single-threshold configuration, the observed Recall increased from 93.4 to 93.9, while Precision changed only slightly from 93.5 to 93.4. The corresponding F1 increased from 93.3 to 93.4.

This retrospective comparison shows that punctuation-aware decision calibration provided useful boundary information without requiring a larger neural architecture.

4-5-5 Low-Weight Two-Model Ensemble

After establishing the strong single-model configuration, a later experiment investigated whether a small contribution from a second PA predictor could provide complementary sentence-boundary evidence.

The two probability streams were combined as

$$p_i^{\text{PA}} = 0.95p_i^{\text{base}} + 0.05p_i^{\text{aux}}, \quad (4.20)$$

where the principal PA model retained 95% of the ensemble weight and the auxiliary model contributed 5%.

The ensemble used the previously established PA inference-window settings, with a recalibrated default threshold of 0.543 and a lower punctuation-related threshold of 0.310.

The additional model did not improve F1 beyond the strongest single-model configuration. Instead, it produced a small change in the Precision–Recall operating point, with slightly higher Recall and slightly lower Precision.

The low-weight ensemble corresponds to the selected PA configuration preserved for the final cross-track comparison. Its final blind-evaluation metrics are reported once in Section 4-9-1.

4-5-6 Selective LLM-Assisted Boundary Rescue

A later experiment investigated whether a large language model could correct uncertain PA boundary decisions.

Rather than asking the language model to segment entire documents, the experiment selected only development examples whose probabilities were close to the current decision boundary.

A total of 250 uncertain cases were examined.

The selected correction policy ultimately replaced

0

predictions.

Development F1 therefore remained approximately

0.93718,

with no absolute improvement and no improvement on any of the five held-out document folds.

The LLM-assisted correction layer was therefore not retained in the selected PA configuration..

4-6 NoPnx-PA Track Development

The NoPnx-PA condition removes punctuation while preserving paragraph information.

The absence of punctuation makes sentence-boundary detection more dependent on contextual evidence. However, paragraph structure remains available and provides an important source of explicit document organization.

The development strategy for this condition therefore focused on two complementary directions:

- representing paragraph transitions explicitly; and
- improving neural prediction robustness through independent-model diversity and probability-level ensembling.

4-6-1 Explicit Paragraph Representation

Removing punctuation changes the nature of the segmentation problem.

In PA, punctuation marks can provide direct evidence that a sentence has ended. In NoPnx-PA, these cues are intentionally removed.

Paragraph information remains available, however. A transition between paragraphs therefore provides comparatively strong structural evidence that the preceding textual segment has ended.

To make this information explicit and consistently available to the inference pipeline, paragraph transitions were represented using an internal marker:

[PAR].

This marker is an internal processing representation rather than an assumption that the original shared-task token sequence necessarily contains a literal [PAR] token.

Its purpose is to preserve paragraph-transition information after the punctuation-free input has been prepared for neural processing.

4-6-2 Paragraph-Aware Boundary Rules

Two deterministic rules were associated with the paragraph marker.

The marker itself is not treated as the final lexical token of a sentence:

$$\hat{y}_i = 0 \quad \text{if } x_i = [\text{PAR}]. \quad (4.21)$$

The token immediately preceding a paragraph marker is treated as a sentence boundary:

$$\hat{y}_{i-1} = 1 \quad \text{if } x_i = [\text{PAR}]. \quad (4.22)$$

All remaining tokens are classified using the neural probability ensemble.

The complete rule can therefore be expressed as

$$\hat{y}_i = \begin{cases} 0, & \text{if } x_i = [\text{PAR}], \\ 1, & \text{if } x_{i+1} = [\text{PAR}], \\ \mathbb{I}[p_i^{\text{ens}} \geq \tau], & \text{otherwise.} \end{cases} \quad (4.23)$$

This design prevents the neural model from having to infer a structural fact that is already explicitly available from the document representation.

4-6-3 Independent Model Diversity

The principal neural system consisted of two independently fine-tuned CAMELBERT-MSA token classifiers, referred to as Model A and Model B.

Both models used the same architecture and paragraph-aware task formulation and were trained only on the official NoPnx-PA training data. Independent training initializations were used to introduce prediction diversity between the two classifiers.

Their development performance was very similar, which made direct selection between the two models less informative than evaluating whether their probability estimates provided useful complementary evidence.

This motivated the probability-level ensemble evaluated in the following section. The individual and combined development results are reported together in Table 4.8.

4-6-4 Two-Model Probability Ensemble

The two CAMELBERT probability streams were combined using a weighted average.

For token i ,

$$p_i^{\text{NoPnxPA}} = 0.40p_i^{(A)} + 0.60p_i^{(B)}. \quad (4.24)$$

Table 4.8 reports the development performance of the two individual models and their weighted ensemble.

Table 4.8: NoPnx-PA development comparison for the two-model ensemble.

System	Dev F1	Interpretation
CAMeLBERT Model A	≈ 0.8438	Ensemble component
CAMeLBERT Model B	≈ 0.8448	Ensemble component
0.40/0.60 weighted ensemble	$\approx \mathbf{0.8473}$	Selected

Although the improvement was modest, the weighted ensemble outperformed both individual models on the development set.

This result provided development evidence that combining the two independently trained probability streams was useful.

4-6-5 Decision-Threshold Calibration

After combining the two probability streams, the final decision threshold was calibrated using development evidence.

The selected threshold was

$$\tau_{\text{NoPnxPA}} = 0.40. \quad (4.25)$$

Therefore, for tokens not directly controlled by the paragraph rules,

$$\hat{y}_i = \mathbb{I} \left[0.40p_i^{(A)} + 0.60p_i^{(B)} \geq 0.40 \right]. \quad (4.26)$$

The final inference strategy consequently combines neural confidence with deterministic structural information rather than forcing one mechanism to handle both types of evidence.

The preserved experimental record supports the ensemble weights, decision threshold, and paragraph-aware decision rules. However, it does not establish the final maximum sequence length and stride of both selected component models with the same level of confidence. These parameters are therefore not asserted for the selected NoPnx-PA configuration.

4-6-6 Retrospective Blind-Evaluation Record

The preserved experimental history contains several NoPnx-PA configurations that were evaluated on the blind set after their corresponding inference settings had been fixed.

An early frozen configuration achieved an F1-score of 87.2.

A later recall-oriented configuration obtained

$$P = 86.4, \quad R = 90.0, \quad F1 = 87.7. \quad (4.27)$$

These results are reported retrospectively as part of the experimental record and were not used to tune the model or select its decision threshold.

The blind-evaluation metrics of the final selected NoPnx-PA configuration are reported once in Section 4-9-1.

4-6-7 Meta-Classification Experiment

An additional experiment investigated whether a lightweight meta-classifier could improve uncertain boundary decisions produced by the two-model ensemble.

The meta-classifier used the two CAMELBERT probability or logit streams together with general contextual features.

Because the training collection is relatively small, feature design was intentionally restricted to reduce lexical memorization.

The experiment therefore excluded:

- character n -grams;
- direct token identity; and
- memorized prefix and suffix features.

The meta-classifier was designed to learn only from the official training data, while development data remained reserved for evaluation.

The available development evidence did not establish a stable improvement over the selected two-model ensemble.

Consequently, the meta-classifier was not included in the selected NoPnx-PA configuration.

4-6-8 Selective LLM-Assisted Verification

A related experiment investigated whether a large language model could be used selectively to verify uncertain sentence-boundary decisions.

The objective was not to segment complete documents using an LLM.

Instead, only cases close to the classifier decision threshold were considered for additional verification.

The verification layer was applied only to uncertain development cases identified by the existing classifier.

A candidate correction was considered only when development evidence indicated a positive improvement.

The available development evidence did not establish an accepted improvement from this additional verification layer.

Consequently, LLM-assisted verification was not included in the selected NoPnx-PA configuration.

4-7 NP Track Development

The NP condition preserves punctuation while removing paragraph information.

Unlike NoPnx-PA, the model can still exploit punctuation cues, but it cannot rely on explicit paragraph transitions when determining sentence boundaries.

The development process revealed an important methodological challenge: strong development performance did not always correspond to equally strong blind-evaluation behavior. NP development therefore emphasized controlled calibration, document-level analysis, independent-model diversity, and held-out validation before selecting a reproducible configuration.

4-7-1 Initial CAMELBERT-MSA Model

The principal NP model was a CAMELBERT-MSA token classifier trained directly on the NP condition.

The official training partition contained 174 documents and approximately 124,444 NP tokens, while the development partition contained 222 documents and approximately 158,765 tokens.

The training configuration used:

- CAMELBERT-MSA;
- maximum sequence length 128;
- stride 16;
- five training epochs;
- learning rate 3×10^{-5} ;
- weight decay 0.01;
- batch size 8; and
- final-visible-subtoken alignment to the original AraSeg tokens.

At the conventional decision threshold of 0.50, the model achieved

$$F1_{\text{Dev}}(0.50) = 0.91438. \quad (4.28)$$

A development threshold search selected

$$\tau = 0.55, \quad (4.29)$$

at which development F1 increased slightly to approximately

$$F1_{\text{Dev}} \approx 0.91466. \quad (4.30)$$

This configuration established the principal development baseline for the subsequent NP experiments.

4-7-2 Historical Strong Blind Result

An earlier NP evaluation produced the strongest numerical blind result preserved in the experimental record:

$$P = 92.4, \quad R = 91.0, \quad F1 = 91.2. \quad (4.31)$$

However, the complete inference configuration associated with this result could not later be reconstructed with the same level of confidence as the subsequent controlled experiments.

In particular, the preserved evidence did not fully establish the exact combination of checkpoint, decision threshold, final-document handling, and other inference settings that produced this result.

The 91.2-F1 score is therefore retained as a historical blind-evaluation record, but it is not used as the primary reproducible NP configuration described in this thesis.

4-7-3 Low-Recall Diagnostic Experiment

A later frozen NP configuration used the previously described principal model and calibrated decision threshold, together with a forced final-document boundary.

Its blind-evaluation record was

$$P = 93.4, \quad R = 81.3, \quad F1 = 86.0. \quad (4.32)$$

The combination of high Precision and substantially lower Recall indicated under-segmentation: predicted boundaries were generally reliable, but many reference sentence boundaries were missed.

This result is retained as a diagnostic blind-evaluation record rather than as evidence for selecting a final configuration. The corresponding under-segmentation behavior motivated further investigation using development and held-out document evidence, with particular attention to decision calibration and document-level boundary behavior.

4-7-4 Document-Adaptive Boundary Calibration

The under-segmentation observed in the diagnostic configuration led to a controlled investigation of document-level decision calibration.

A single global threshold assumes that one probability operating point is appropriate for all documents. To examine whether document-level variation could affect this assumption, the development documents were divided into controlled folds.

Threshold-related decisions were derived from the training portion of each fold and evaluated on held-out documents. This procedure avoided selecting and evaluating the adaptive strategy on the same development instances.

After the document-adaptive configuration had been evaluated using this controlled development procedure and its inference settings had been fixed, its preserved blind-evaluation record was

$$P = 93.0, \quad R = 87.1, \quad F1 = 89.4. \quad (4.33)$$

Relative to the earlier diagnostic blind record, the adaptive configuration exhibited substantially higher Recall and F1. These blind metrics are reported retrospectively and were not used to select the adaptive decision strategy.

The experiment provided evidence that document-level calibration was a useful direction for addressing the under-segmentation behavior and served as the comparator for the later global ensemble experiments.

4-7-5 Independent Model Diversity

The next development direction examined whether independent model initializations could provide useful prediction diversity beyond the principal NP model.

Two additional CAMELBERT-MSA token classifiers were trained independently using the same controlled NP training configuration described in Section 4-7-1. They are referred to as Auxiliary Model A and Auxiliary Model B.

The models shared the same architecture and token-boundary classification formulation, while independent initializations produced separate boundary-probability streams.

Rather than selecting one auxiliary model as a standalone replacement, the two additional probability streams were retained for combination with the principal model.

The subsequent ensemble therefore considered three aligned CAMELBERT-MSA predictors: the principal NP model, Auxiliary Model A, and Auxiliary Model B.

4-7-6 Global Three-Model Probability Ensemble

The three aligned boundary-probability streams were combined using a weighted average:

$$p_i^{\text{NP}} = 0.40p_i^{\text{principal}} + 0.30p_i^{\text{auxA}} + 0.30p_i^{\text{auxB}}. \quad (4.34)$$

The corresponding global decision threshold was

$$\tau_{\text{NP}} = 0.415. \quad (4.35)$$

Unlike the earlier diagnostic configuration, this ensemble did not apply an automatic final-document boundary constraint.

This global ensemble was subsequently compared with the document-adaptive strategy using held-out document folds.

4-7-7 Held-Out Comparison of Global and Adaptive Strategies

The global three-model ensemble was compared with the earlier document-adaptive strategy using held-out development folds.

An additional variant combined the three-model probability ensemble with document-adaptive threshold prediction. This allowed the global and adaptive decision strategies to be compared under the same controlled evaluation framework.

Table 4.9 summarizes the comparison.

Table 4.9: Document-level held-out comparison of the principal NP decision strategies.

System	Held-Out F1	Decision
Document-adaptive calibration	≈ 0.916862	Comparator
Global three-model ensemble	$\approx \mathbf{0.918331}$	Selected
Adaptive three-model ensemble	≈ 0.918300	Not selected

The global ensemble outperformed the document-adaptive comparator on four of the five held-out document folds.

The adaptive three-model variant produced a nearly identical aggregate score but did not exceed the simpler global ensemble. Because it added an additional adaptive decision layer without a measurable held-out advantage, it was not retained.

The global three-model ensemble was therefore selected using held-out development evidence before blind evaluation.

After selection, the corresponding inference configuration was frozen before blind evaluation. Its final blind-evaluation metrics are reported once in Section 4-9-1.

4-7-8 Nested Token-Level Meta-Classification

After selecting the global ensemble, an additional experiment examined whether a supervised token-level meta-model could exploit interactions between the component probabilities more effectively than the fixed weighted average.

The experiment considered lightweight stacking models such as logistic regression and gradient-boosting methods.

Model selection was performed using leakage-controlled document folds.

Because the selected global ensemble already provided a strong development reference, the meta-classifier was required to satisfy stricter acceptance criteria:

- minimum F1 gain of 0.001;
- minimum Recall gain of 0.003;
- Precision loss no greater than 0.003; and
- improvement on at least four of five folds.

The available held-out evidence did not establish a sufficiently stable improvement over the selected three-model ensemble.

Consequently, token-level stacking was not included in the selected NP configuration.

4-7-9 Selective LLM-Assisted Boundary Verification

A later experiment investigated whether an LLM could selectively improve uncertain NP sentence-boundary decisions.

Only development cases with predictions close to the current decision threshold were considered. A total of 250 uncertain cases were examined.

Under the evaluated policy, no predictions were changed. Development F1 therefore remained approximately 0.91585, with no improvement and no held-out fold wins.

The available development evidence did not establish an improvement from the LLM-assisted verification layer. Consequently, it was not included in the selected NP configuration.

4-8 NoPnx-NP Track Development

The NoPnx-NP condition removes both punctuation and paragraph information, leaving the model with the least explicit structural evidence among the four AraSeg conditions.

Unlike PA, NoPnx-PA, and NP, sentence boundaries in this condition must be inferred primarily from lexical, syntactic, semantic, and contextual transitions.

This condition became the most experimentally demanding part of the AraSeg study. Its development did not follow a simple monotonic progression: an early blind-evaluation configuration exhibited severe under-segmentation, which motivated verification of the inference pipeline before further model development.

The subsequent experiments therefore proceeded through controlled model reconstruction, calibration, model diversity, and held-out document-level validation.

4-8-1 Direct NoPnx-NP Fine-Tuning

The first principal model was obtained by directly fine-tuning CAMeLBERT-MSA on the official NoPnx-NP training data.

Unlike a transfer-based system, this model learned the target task directly without initialization from another AraSeg condition.

The selected short-context configuration used

$$L = 128, \quad S = 16. \quad (4.36)$$

Because neither punctuation nor paragraph information is available, the model must rely primarily on contextual representations to detect linguistic transitions associated with sentence completion.

This directly trained predictor served as the principal target-specific probability source in the subsequent NoPnx-NP experiments.

4-8-2 Cross-Track Transfer from NP

A second model investigated transfer learning from NP to NoPnx-NP.

The motivation came from the structural relationship between the two conditions.

Both NP and NoPnx-NP remove paragraph information. Their principal difference is that NP retains punctuation while NoPnx-NP removes it.

A CAMELBERT model previously trained on NP therefore already contains representations associated with sentence boundaries in a paragraph-free setting.

The transfer process can be summarized as

pretrained CAMELBERT $\rightarrow$ NP sentence-boundary adaptation $\rightarrow$ NoPnx-NP fine-tuning.

The transferred model used the same short-context inference settings as the directly fine-tuned NoPnx-NP model.

The objective was not to assume that transfer learning would necessarily outperform direct training.

Instead, the direct and transferred models were treated as potentially complementary probability sources because their different training histories could produce different confidence estimates on difficult boundary cases.

4-8-3 Initial Clean-Transfer Ensemble

The two CAMELBERT probability streams were initially combined using equal weights:

$$p_i^{\text{CT}} = 0.50p_i^{\text{clean}} + 0.50p_i^{\text{transfer}}. \quad (4.37)$$

This combination established an important reference system.

Although both components use CAMELBERT-MSA, they were trained through different task trajectories. Their predictions therefore need not be identical.

The ensemble was retained as a controlled reference configuration for the subsequent NoPnx-NP experiments.

4-8-4 Severe Initial Blind Recall Collapse

An early frozen NoPnx-NP configuration produced a severe blind-evaluation performance collapse:

$$P = 79.7, \quad R = 53.1, \quad F1 = 62.7. \quad (4.38)$$

The particularly low Recall indicated severe under-segmentation, with a large proportion of reference sentence boundaries left undetected.

The preserved experimental record shows that this configuration differed from the later verified pipeline in several important ways. It used:

- only the directly fine-tuned NoPnx-NP model;
- a decision threshold of approximately 0.38;
- a forced final-document boundary; and
- probability extraction from the first subtoken associated with each original token.

The 62.7-F1 result is therefore retained as a diagnostic blind-evaluation record of the earlier inference configuration. It motivated verification of token alignment and reconstruction of the controlled pipeline using labeled development evidence before further model experiments were performed.

4-8-5 Alignment Verification and System Recovery

Following the early blind failure, the NoPnx-NP inference pipeline was reconstructed using the verified model configurations and labeled development data.

A central requirement was consistent alignment between original AraSeg tokens and Transformer subtokens. The corrected mapping was

original AraSeg token $\rightarrow$ Transformer subtokens $\rightarrow$ final visible subtoken $\rightarrow$ boundary
probability.

When an original token appeared in multiple overlapping Transformer windows, its repeated boundary probabilities were aggregated using the mean.

After this correction, subsequent candidate systems were evaluated under a common document ordering, token alignment, window-aggregation procedure, and document-level evaluation protocol.

This standardized pipeline provided the controlled basis for the recovery and ensemble experiments described in the following sections.

4-8-6 Controlled Two-Model Recovery

After alignment had been standardized, the previously defined clean-transfer ensemble was re-evaluated using the corrected inference pipeline.

The controlled configuration used a decision threshold of

$$\tau = 0.42, \tag{4.39}$$

with no automatic sentence boundary inserted at the end of the document.

Development evaluation under the corrected pipeline established this configuration as the reference system for the subsequent context-diversity experiments. Its development metrics are reported once in the held-out comparison in Section 4-8-10.

After the configuration had been fixed, its blind-evaluation record was

$$P = 82.9, \quad R = 84.1, \quad F1 = 83.0. \quad (4.40)$$

Compared with the earlier diagnostic blind record, the corrected configuration exhibited substantially stronger boundary recovery. The result is retained as the blind-evaluation record of the controlled two-model reference system.

4-8-7 Diagnostic Analysis before Further Model Development

After establishing the controlled two-model reference system, a development-side diagnostic analysis was performed to determine the most appropriate direction for further improvement.

The analysis considered boundary-count behavior, false-positive and false-negative patterns, and performance across held-out documents.

The available evidence suggested that simple global threshold adjustments were unlikely to provide a substantial additional gain. Further improvement therefore required probability sources with different contextual behavior rather than repeated calibration of the same predictors.

This motivated the subsequent investigation of context-length diversity.

4-8-8 Context-Length Diversity

The next development direction examined whether models trained with different context lengths could provide complementary sentence-boundary evidence.

Candidate CAMELBERT models with maximum context lengths of 256 and 512 tokens were investigated in addition to the existing short-context predictors.

The context-256 model used

$$L = 256, \quad S = 32. \quad (4.41)$$

The objective was not to select a model solely on the basis of its standalone F1-score. A predictor could also be useful when its probability estimates complemented those of the existing models.

Development analysis indicated that the context-256 model provided the most useful complementary signal among the examined context-length variants.

It was therefore retained for the subsequent controlled ensemble experiment.

4-8-9 Context-Diverse CAMELBERT Ensemble

The directly fine-tuned, NP-transfer, and retained context-256 CAMELBERT predictors were combined at the probability level.

The selected ensemble was

$$p_i^{\text{ctx}} = 0.10p_i^{\text{clean}} + 0.50p_i^{\text{transfer}} + 0.40p_i^{\text{context256}}. \quad (4.42)$$

The corresponding decision threshold was

$$\tau_{\text{ctx}} = 0.445. \quad (4.43)$$

No sentence boundary was inserted automatically at the end of the document.

The held-out development evidence supporting selection of this configuration is presented in the following section.

4-8-10 Held-Out Validation of Context Diversity

The context-diverse ensemble was compared with the controlled clean-transfer reference system using document-level held-out folds.

For each fold, configuration decisions were derived from the training portion and evaluated on unseen development documents. This provided a controlled basis for determining whether the additional context-256 probability stream produced a stable improvement.

Table 4.10 summarizes the comparison.

Table 4.10: Held-out comparison of the recovered and context-diverse NoPnx-NP ensembles.

System	P	R	F1	Fold Wins
Clean-transfer ensemble	0.80370	0.82832	0.80869	–
Context-diverse ensemble	0.81217	0.82645	0.81235	4/5

The context-diverse ensemble achieved a modest aggregate improvement and outperformed the reference system on four of the five held-out document folds.

Recall remained similar, while Precision increased. This pattern provided evidence that the additional context-256 probability stream contributed complementary information rather than merely increasing the overall number of predicted boundaries.

The context-diverse ensemble was therefore retained for subsequent blind evaluation.

4-8-11 Blind Evaluation of the Context-Diverse Ensemble

After selection using held-out development evidence, the context-diverse ensemble was frozen before blind evaluation.

Its preserved blind-evaluation result was

$$P = 83.7, \quad R = 83.7, \quad F1 = 83.1. \quad (4.44)$$

The improvement over the preceding controlled two-model reference was numerically small. However, the configuration had been selected using document-level held-out evidence and represented the strongest retained CAMELBERT-only NoPnx-NP system.

It therefore served as the reference configuration for the subsequent experiments involving more substantially different modeling strategies.

4-8-12 Beyond the Context-Diverse CAMeLBERT Ensemble

The context-diverse CAMeLBERT ensemble established a controlled CAMeLBERT-only reference system for NoPnx-NP.

The next development phase examined whether further improvement could be obtained through more sophisticated decoding, additional same-family model diversity, cross-condition transfer, alternative training strategies, or a different pretrained Arabic encoder.

These experiments were evaluated using the same held-out document-level methodology applied during the preceding development stages.

Their purpose was to determine whether additional complexity within the existing modeling family could provide stable generalization gains or whether a more substantially different source of representation diversity was required.

4-8-12-1 Structure-Aware Sequence Decoding

The first experiment examined whether sequence-level structural information could improve the token-level predictions of the context-diverse CAMeLBERT system.

Instead of applying an independent threshold to each token, a structure-aware decoder considered the resulting sequence of sentence boundaries jointly, including sentence-length preferences.

Conceptually, the standard decision process

token probabilities $\rightarrow$ threshold $\rightarrow$ sentence boundaries

was replaced by

token probabilities $\rightarrow$ sequence-level structural scoring $\rightarrow$ best boundary sequence.

The available held-out evidence did not establish a sufficiently stable improvement over the context-diverse CAMeLBERT reference system.

Consequently, structure-aware decoding was not retained for the selected NoPnx-NP configuration.

4-8-12-2 Additional Same-Backbone Model Diversity

The next experiment examined whether additional independently trained CAMeLBERT models could provide useful diversity beyond the selected context-diverse ensemble.

Two additional CAMeLBERT predictors were trained independently using the same context-256 configuration described earlier.

Although the models produced valid probability streams, their additional same-backbone diversity did not provide a sufficiently stable held-out improvement over the existing CAMeLBERT reference system.

This indicated that model disagreement alone was not sufficient; the additional predictors also needed to contribute reliably different boundary evidence.

Consequently, these additional same-backbone models were not retained in the selected NoPnx-NP configuration.

4-8-12-3 Calibration-Aware Same-Backbone Ensembling

A further experiment examined whether differences in probability calibration between independently trained CAMELBERT models could limit the effectiveness of direct probability averaging.

To reduce the effect of model-specific probability scales, ordinary probability averaging was compared with a threshold-centered logit representation.

For model m , the centered score was defined as

$$c_i^{(m)} = \text{logit} \left(p_i^{(m)} \right) - \text{logit} \left(\tau_m \right), \quad (4.45)$$

where τ_m denotes the independently calibrated decision threshold of model m .

Candidate combinations were evaluated using the same controlled document-level held-out framework applied to the preceding ensemble experiments.

The best calibration-aware candidate produced

$$\Delta F1 = -0.001635, \quad (4.46)$$

and improved on only 2/5 held-out folds.

The available held-out evidence therefore did not establish an improvement over the existing context-diverse CAMELBERT reference system.

Consequently, calibration-aware same-backbone ensembling was not retained.

4-8-12-4 Transfer from the Punctuation-Free Paragraph-Aware Condition

A further experiment investigated transfer learning from NoPnx-PA.

This source condition was particularly relevant because both NoPnx-PA and NoPnx-NP remove punctuation, while differing in the availability of paragraph information.

Two CAMELBERT models previously fine-tuned for NoPnx-PA were used as source checkpoints and subsequently fine-tuned on the official NoPnx-NP training partition.

The target fine-tuning used the previously described context-256 configuration together with:

- five fine-tuning epochs;
- learning rate 2×10^{-5} ;
- weight decay 0.01;
- effective batch size 8; and
- no class weighting.

The two transferred models achieved development F1-scores of approximately 0.803118 and 0.806113, respectively.

Although both were individually weaker than the existing context-diverse CAMELBERT reference system, their probability streams were retained temporarily to evaluate whether they could contribute useful complementary evidence within a larger ensemble.

4-8-12-5 Large Same-Family Probability Ensemble

The next experiment examined whether a larger same-family ensemble could exploit complementary errors beyond the selected context-diverse CAMELBERT system.

The three previously defined CAMELBERT probability streams were augmented with the two NoPnx-PA-transfer predictors described in the preceding section, producing a five-stream ensemble.

Candidate weights and decision thresholds were selected using nested document-level evaluation.

Although some combinations appeared promising when evaluated on the complete development set, the gains did not remain sufficiently stable on held-out documents.

The five-stream ensemble therefore did not provide the controlled improvement required to replace the existing context-diverse CAMELBERT reference system.

Consequently, it was not retained.

4-8-12-6 Cross-Condition Teacher Analysis

A diagnostic experiment examined whether probability signals from other AraSeg conditions could reveal sentence boundaries that were difficult for the NoPnx-NP system to infer directly.

Predictions from models trained on richer conditions, including NP and NoPnx-PA, were aligned with the NoPnx-NP token sequence and analyzed as teacher-like signals.

The objective was to examine whether access to punctuation or paragraph information could provide useful evidence for boundaries missed by the NoPnx-NP predictors.

These richer-condition signals were informative for analysis, but they depended on structural information that is not available in the NoPnx-NP input at inference time.

They were therefore treated as diagnostic evidence rather than as a deployable NoPnx-NP prediction system.

4-8-12-7 Multi-View Knowledge Distillation

Because the richer cross-condition teacher signals could not be used directly during NoPnx-NP inference, a subsequent experiment examined whether part of their information could be transferred into a NoPnx-NP student model.

The student received only valid NoPnx-NP input at inference time, while training incorporated both the gold sentence-boundary labels and an auxiliary teacher signal.

Conceptually, the training objective was

$$\mathcal{L} = \lambda \mathcal{L}_{\text{gold}} + (1 - \lambda) \mathcal{L}_{\text{teacher}}. \quad (4.47)$$

The distilled student was evaluated using the same controlled held-out document framework applied to the other NoPnx-NP candidates.

The available held-out evidence did not establish a sufficiently stable improvement over the context-diverse CAMELBERT reference system.

Consequently, knowledge distillation was not retained.

4-8-12-8 Joint Multi-Track Training

A further experiment examined whether sentence-boundary knowledge could be shared jointly across the four AraSeg conditions.

The model used a shared Arabic Transformer representation together with condition-specific boundary-classification heads:

shared Arabic Transformer encoder $\rightarrow$ condition-specific boundary classifiers.

Training jointly exposed the shared representation to the different combinations of punctuation and paragraph information before specialization to the NoPnx-NP target condition.

The objective was to determine whether this multi-condition training could produce a more general representation of sentence completion than the preceding single-track and transfer-based approaches.

The available held-out evidence did not establish a sufficiently stable improvement over the context-diverse CAMELBERT reference system.

Consequently, joint multi-track training was not retained.

4-8-12-9 Evidence of a Same-Family Modeling Ceiling

The preceding experiments examined multiple ways of extending the CAMELBERT-based NoPnx-NP system through additional decoding, same-family model diversity, cross-condition transfer, and alternative training strategies.

Despite these different approaches, the available held-out evidence did not establish a sufficiently stable improvement over the selected context-diverse CAMELBERT reference system.

This suggested that further gains were unlikely to come simply from adding more complexity around closely related CAMELBERT probability streams.

The next development step therefore introduced a substantially different pretrained Arabic encoder in order to test whether backbone-level representation diversity could provide more useful complementary boundary evidence.

4-8-13 AraELECTRA Backbone Diversity

AraELECTRA was introduced as a substantially different pretrained Arabic encoder in order to provide backbone-level representation diversity beyond the preceding CAMELBERT-based systems.

The model was fine-tuned on the official NoPnx-NP training partition. Input windowing followed the previously described context-256 configuration, while token-level prediction alignment followed the verified inference procedure described in Section 4-8-5.

The AraELECTRA-specific training configuration used:

- physical batch size 4 with gradient accumulation over 2 steps, giving an effective batch size of 8;
- four training epochs;
- encoder learning rate 2×10^{-5} ;
- classification-head learning rate 5×10^{-5} ;
- weight decay 0.01;
- warm-up ratio 0.10; and
- maximum gradient norm 1.0.

The four resulting checkpoints were compared using development evidence, and the checkpoint from the third training epoch was selected for the subsequent backbone-diverse ensemble experiment.

4-8-13-1 Hierarchical CAMELBERT–AraELECTRA Ensemble

The previously selected context-diverse CAMELBERT ensemble was retained as a single aggregate probability source and combined with the AraELECTRA predictor.

The internal CAMELBERT combination is defined in Equation 4.42. The backbone-diverse probability was computed as

$$p_i^{\text{backbone}} = 0.40p_i^{\text{ctx}} + 0.60p_i^{\text{AraELECTRA}}. \quad (4.48)$$

The corresponding decision threshold was

$$\tau = 0.405. \quad (4.49)$$

No sentence boundary was inserted automatically at the end of the document.

This hierarchical formulation preserved the strongest CAMELBERT evidence while introducing AraELECTRA as a substantially different backbone-level probability source.

Its effectiveness was evaluated using held-out development documents in the following section.

4-8-13-2 Held-Out Validation of Backbone Diversity

The backbone-diverse ensemble was compared with the context-diverse CAMeLBERT reference system using document-level held-out evaluation.

Table 4.11 summarizes the comparison.

Table 4.11: Held-out effect of introducing AraELECTRA backbone diversity.

System	P	R	F1	Fold Wins
Context-diverse CAMeLBERT ensemble	0.81357	0.82674	0.81327	–
CAMeLBERT–AraELECTRA ensemble	0.82145	0.86338	0.83394	5/5

The backbone-diverse ensemble produced a clear held-out improvement and outperformed the CAMeLBERT-only reference on all evaluated document folds.

The improvement was driven primarily by stronger Recall, while Precision also increased. This provided evidence that AraELECTRA contributed complementary boundary information rather than merely shifting the Precision–Recall operating point.

The backbone-diverse configuration was therefore retained for complete-development evaluation.

4-8-13-3 Complete-Development Evaluation

After passing held-out validation, the selected backbone-diverse configuration was evaluated on the complete development set.

It achieved

$$P = 0.8215189, \quad R = 0.8653779, \quad F1 = 0.8349489. \quad (4.50)$$

Following this evaluation, the previously selected inference configuration was frozen before blind evaluation.

4-8-13-4 Blind Evaluation of the Backbone-Diverse System

After the backbone-diverse configuration had been selected using held-out and complete-development evidence, it was frozen before blind evaluation.

The blind data were used only for final evaluation. No blind labels were used for retraining, threshold calibration, or ensemble selection.

The final blind-evaluation metrics of the selected NoPnx-NP configuration are reported in Table 4.12.

4-8-14 Further Evaluation after Backbone-Diverse Selection

After the backbone-diverse CAMeLBERT–AraELECTRA configuration had been selected, the experimental objective shifted from constructing a strong NoPnx-NP system to determining

whether any additional modeling or decision strategy could provide sufficient evidence to replace it.

Subsequent experiments therefore examined alternative pretrained encoders, additional model diversity, alternative boundary representations, and targeted correction strategies.

Each candidate was required to demonstrate a sufficiently stable development or held-out improvement over the selected backbone-diverse reference system.

Unless otherwise stated, these experiments used training and development evidence only. Blind data were not used for model, threshold, or decision-rule selection.

4-8-14-1 Additional Backbone-Diversity Experiments

After the successful introduction of AraELECTRA, additional experiments examined whether other Arabic pretrained encoders or additional AraELECTRA variants could provide further useful diversity.

AraBERTv2 and MARBERTv2 were fine-tuned on the official NoPnx-NP training data and evaluated both independently and as auxiliary probability sources.

Neither encoder provided a sufficiently stable held-out improvement over the selected CAMeLBERT–AraELECTRA reference system.

AraModernBERT was also evaluated using training and development evidence only. Its strongest documented development configuration achieved approximately

$$P = 0.6219, \quad R = 0.7335, \quad F1 = 0.6627. \quad (4.51)$$

When AraModernBERT was considered as an additional ensemble component, the selected combination assigned it zero weight.

Additional diversity within AraELECTRA was then examined using neighboring training checkpoints and independently trained models.

The available held-out evidence did not establish a sufficiently stable improvement from either checkpoint combination or additional AraELECTRA model averaging.

Consequently, none of these additional backbone- or model-diversity variants replaced the selected backbone-diverse reference system.

4-8-14-2 Selective and Boundary-Transition Alternatives

Two additional experiments examined whether the selected backbone-diverse system could be improved through more selective decision logic or a more explicit representation of sentence transitions.

The first experiment applied selective consensus gating, in which additional AraELECTRA predictions were consulted only for uncertain positive boundary decisions.

The available held-out evidence did not establish a sufficiently stable improvement over the selected backbone-diverse reference system.

A second experiment examined explicit boundary-transition modeling. Two alternatives were evaluated: a boundary-pair classifier and a dual-head architecture combining token-level and neighboring-token pair predictions.

Although these formulations modeled sentence transitions more directly, neither produced a sufficiently stable held-out improvement.

Consequently, neither selective consensus gating nor the boundary-transition alternatives were retained in the selected NoPnx-NP configuration.

4-8-14-3 Waw-Aware Diagnostic Modeling

A targeted diagnostic experiment examined sentence-boundary errors associated with word-initial Arabic *waw*.

Because *waw* may connect phrases, clauses, or complete sentences while remaining orthographically attached to the following word, these cases can create ambiguous boundary transitions.

A modified preprocessing representation was constructed for selected word-initial *waw* cases while preserving the original AraSeg token indexing required for evaluation.

A specialized AraELECTRA predictor was then trained using the previously established context-256 configuration. The experiment used training and development evidence only.

The specialized predictor was evaluated both independently and as a conditional auxiliary expert allowed to intervene only on candidate boundaries associated with word-initial *waw*.

Although the experiment provided useful diagnostic evidence about linguistically targeted boundary modeling, the available held-out evidence did not establish a sufficiently stable improvement over the selected backbone-diverse reference system.

Consequently, the *waw*-aware predictor was not retained in the selected NoPnx-NP configuration.

4-8-14-4 Meta-Classification and Selective LLM Verification

A final post-selection experiment examined whether difficult NoPnx-NP boundary decisions could be corrected using a lightweight meta-classifier based on neural confidence, cross-model agreement, and general contextual features.

Direct token identity and sparse lexical n -gram features were excluded in order to reduce the risk of memorizing specific development examples.

An optional LLM verification stage was then evaluated only on highly uncertain development cases. Among 250 examined cases, the conservative policy changed only six predictions.

The resulting development improvement was

$$\Delta F1 \approx +0.00003262. \quad (4.52)$$

The candidate improved on only 1/5 held-out document folds.

A less conservative variant that allowed all queried LLM suggestions to replace the neural decisions reduced F1 by approximately

$$0.001485. \quad (4.53)$$

A follow-up experiment removed the LLM component and retained only the meta-classification layer. This variant also failed to establish a sufficiently stable held-out improvement over the selected backbone-diverse reference system.

Consequently, neither the LLM-assisted nor the meta-classification-only variant was retained in the selected NoPnx-NP configuration.

4-8-14-5 Additional Targeted Post-Selection Checks

Three additional targeted checks were evaluated after selection of the backbone-diverse reference system.

A final-document boundary constraint produced an absolute F1 gain of only 0.00007538, which was insufficient to justify modifying the selected inference rule.

An NPMI-based correction strategy decreased F1 by 0.00010830. The evaluated correction harmed three true-positive boundary decisions and corrected no false positives.

Agreement-based suppression was also examined as a conservative method for removing likely false-positive boundaries. The available evidence did not establish a sufficiently strong improvement.

Consequently, none of these additional decision-level modifications were retained in the selected NoPnx-NP configuration.

4-9 Comparative Experimental Results

The preceding sections described the development of the four AraSeg conditions independently. This section compares the selected reproducible systems across the complete task.

The comparison uses the final blind-evaluation results as a common reference for examining the effects of structural information and the relative difficulty of the four conditions.

4-9-1 Final Blind-Evaluation Results

Table 4.12 presents the selected blind-evaluation result for each AraSeg condition.

Table 4.12: Selected blind-evaluation results for the four AraSeg conditions.

Track	Precision	Recall	F1
PA	93.2	94.0	93.4
NoPnx-PA	87.4	89.3	87.9
NP	92.5	88.5	90.0
NoPnx-NP	83.9	87.0	84.8

These values serve as the common numerical reference for the cross-condition comparisons in the following sections.

4-9-2 Effect of Removing Punctuation

The final results in Table 4.12 show a similar punctuation-related difference under both paragraph conditions.

When paragraph information was available, removing punctuation was associated with an F1 difference of 5.5 points.

When paragraph information was unavailable, the corresponding difference was 5.2 points.

Thus, punctuation removal was associated with a reduction of slightly more than five F1 points in both comparisons.

This pattern is consistent with the role of punctuation as an explicit cue for many sentence endings. Without punctuation, the segmentation models must rely more heavily on lexical, syntactic, semantic, and contextual transitions.

These comparisons should nevertheless be interpreted cautiously. The four AraSeg conditions use independently optimized systems rather than a single fixed model evaluated under controlled ablations.

The observed differences therefore describe the relative difficulty of the selected AraSeg conditions and should not be interpreted as a precise causal measurement of the contribution of punctuation alone.

4-9-3 Effect of Removing Paragraph Information

The final results in Table 4.12 also show a consistent difference associated with paragraph information.

When punctuation was available, removing paragraph information was associated with an F1 difference of 3.4 points.

When punctuation was unavailable, the corresponding difference was 3.1 points.

The observed paragraph-related differences were therefore smaller than the punctuation-related differences reported in the preceding section.

This pattern is consistent with the NoPnx-PA experiments, where explicit paragraph information provided useful structural evidence even in the absence of punctuation.

As with the punctuation comparison, these differences should not be interpreted as controlled causal estimates because the four conditions were optimized independently.

4-9-4 Structural Information and Observed Difficulty

The cross-condition results in Table 4.12 show a consistent relationship between the amount of explicit structural information available and the observed difficulty of sentence segmentation.

PA provides both punctuation and paragraph information and required primarily track-specific calibration and structural decision rules.

NoPnx-PA removes punctuation but retains paragraph information. Explicit paragraph handling therefore remained useful despite the loss of punctuation cues.

NP preserves punctuation while removing paragraph information. In this condition, independent-model diversity and probability ensembling provided the selected reproducible configuration.

NoPnx-NP removes both forms of explicit structural evidence and required the most extensive model-development process. Its selected configuration ultimately incorporated both context diversity and backbone-level diversity.

Overall, the experiments indicate that the modeling strategy became more dependent on contextual and model-diversity evidence as explicit structural cues were removed.

4-9-5 Precision–Recall Behavior Across Conditions

The final Precision–Recall values reported in Table 4.12 reveal different decision profiles across the four AraSeg conditions.

PA exhibits the most balanced relationship between Precision and Recall.

NoPnx-PA shows a slight Recall-oriented tendency, whereas NP exhibits the opposite pattern, with Precision exceeding Recall. This indicates that the selected NP configuration was comparatively more conservative in predicting sentence boundaries.

NoPnx-NP also shows a Recall-oriented profile. This behavior is particularly relevant to the development history of this condition, where under-segmentation had been an important diagnostic issue before the inference pipeline and modeling strategy were improved.

Overall, the selected systems do not converge to a single Precision–Recall operating pattern. Their behavior reflects the different structural information available in each condition and the track-specific calibration and modeling decisions applied during development.

4-10 AraSeg Error Analysis

The aggregate evaluation metrics describe overall segmentation performance but do not identify the boundary-level behaviors that produce these results.

A more detailed error analysis was therefore conducted, with particular attention to NoPnx-NP because it removes both punctuation and paragraph information and provided the most demanding setting for boundary prediction.

The analysis examined false-negative and false-positive behavior, probability calibration, model complementarity, document-level variation, and selected linguistic and processing-related error patterns.

Detailed quantitative error attribution was performed using labeled development data. Blind labels were unavailable and were therefore not used for token-level error analysis.

4-10-1 Missed Sentence Boundaries

A false negative occurs when the gold annotation contains a sentence boundary after token x_i , but the system predicts continuation:

$$y_i = 1, \quad \hat{y}_i = 0. \quad (4.54)$$

False negatives were particularly important in the initial NoPnx-NP baseline analysis, especially for SaT, whose comparatively high Precision and low Recall reflected substantial under-segmentation.

Table 4.13 summarizes the corresponding false-negative counts.

Table 4.13: False-negative boundary counts in the initial NoPnx-NP development analysis.

System	False Negatives
SaT	7,120
CAMeLBERT	2,952
CAMeLBERT–SaT ensemble	2,317

Relative to the original CAMeLBERT operating point, the ensemble therefore recovered 635 previously missed sentence boundaries.

This reduction explains the Recall improvement observed in the initial ensemble experiment. However, recovered boundaries do not necessarily indicate genuine complementary model evidence, because changes in the effective decision calibration can produce similar behavior.

4-10-2 False Sentence Boundaries

A false positive occurs when the system predicts a sentence boundary where the gold annotation indicates continuation:

$$y_i = 0, \quad \hat{y}_i = 1. \quad (4.55)$$

The Recall improvement produced by the initial CAMELBERT–SaT ensemble was accompanied by an increase in false-positive boundaries.

Table 4.14 summarizes the corresponding development counts.

Table 4.14: False-positive boundary counts in the initial NoPnx-NP development analysis.

System	False Positives
CAMELBERT	2,693
CAMELBERT–SaT ensemble	3,363

Relative to the original CAMELBERT operating point, the ensemble introduced 670 additional false sentence boundaries.

This pattern shows that the initial ensemble increased the number of positive boundary decisions, improving Recall while reducing Precision.

The result also illustrates the risk of optimizing sentence segmentation primarily for Recall: excessive false positives produce over-segmentation by dividing coherent sentences into unnecessarily small fragments.

4-10-3 Document-Level Effect of the Initial Ensemble

The effect of the initial CAMELBERT–SaT ensemble was not distributed uniformly across the NoPnx-NP development documents.

Table 4.15 summarizes the document-level outcomes.

Table 4.15: Document-level effect of the initial CAMELBERT–SaT ensemble.

Outcome	Documents	Share of Dev
Improved	140	63.1%
Weakened	58	26.1%
Unchanged	24	10.8%

The ensemble improved performance for the majority of development documents, but its effect was not uniformly positive.

The weakened subset shows that complementary model signals can benefit some documents while degrading others. This observation supported the later use of stricter document-level held-out validation when comparing candidate systems.

4-10-4 Representative Missed-Boundary Recovery

Development probability outputs revealed two qualitatively different forms of recovered sentence boundaries.

In one representative example, CAMELBERT assigned a boundary probability of 0.299, while SaT assigned 0.999. Their combined probability was approximately 0.614, and the gold annotation contained a sentence boundary.

This example represents genuine complementary evidence: SaT strongly supported a correct boundary that CAMELBERT had missed.

In a second example, CAMELBERT assigned approximately 0.500, while SaT assigned only 0.106. The resulting ensemble probability was approximately 0.323.

In this case, the boundary was recovered because of the lower ensemble operating threshold rather than additional positive evidence from SaT.

The two examples illustrate why probability-level inspection is useful for distinguishing genuine model complementarity from changes caused primarily by decision calibration.

4-10-5 Template- and Heading-Driven False Boundaries

Qualitative analysis identified several semi-structured document patterns that could resemble genuine sentence transitions after paragraph and layout information had been removed.

In one representative heading-like example, CAMELBERT assigned a boundary probability of approximately 0.482, while SaT assigned approximately 0.990. The resulting ensemble probability was approximately 0.710, leading to an incorrect sentence boundary.

This type of error is particularly relevant when heading or title structure is no longer explicitly represented in the input.

A second error pattern appeared in educational templates and repeated instructional prompts, where local contextual structure could resemble a genuine discourse transition.

In one recorded example, CAMELBERT itself assigned an incorrect boundary probability of approximately 0.996.

This second case indicates that the error was not caused solely by SaT or by the ensemble threshold. It instead represented a high-confidence contextual error from the supervised encoder.

Together, these examples show that headings, repeated prompts, forms, and other semi-structured document elements can create difficult boundary contexts when structural layout information is unavailable.

4-10-6 Punctuation-Related Boundary Difficulty

The comparative results showed that punctuation-free conditions were more difficult than their punctuated counterparts.

This difficulty should not be interpreted as evidence that every Arabic sentence boundary is explicitly marked by punctuation.

The corpus analysis reports that approximately

39% (4.56)

of sentence boundaries are word-based rather than explicitly marked by punctuation.

A robust sentence segmenter must therefore learn contextual boundary evidence even when punctuation is available.

Nevertheless, punctuation remains an important structural signal. When it is removed, the model must rely more heavily on lexical, syntactic, semantic, and contextual transitions.

The quantitative cross-condition differences associated with punctuation removal are reported separately in Section 4-9-2.

4-10-7 Long Documents and Sliding-Window Processing

Because Transformer models process long documents through overlapping input windows, document length was examined as a possible source of segmentation degradation.

The baseline development analysis did not identify a monotonic relationship between NoPnx-NP document length and CAMELBERT document-level F1. The measured Spearman correlation was

$$\rho = -0.012, \quad p = 0.856. \quad (4.57)$$

Table 4.16 summarizes the corresponding development results across document-length groups.

Table 4.16: Baseline development performance by NoPnx-NP document length.

Document Length	Docs	CAMELBERT F1	SaT F1	Ensemble F1	Gain
≤ 250	63	0.7756	0.5457	0.8029	+0.0273
251–500	57	0.7487	0.4016	0.7603	+0.0116
501–750	62	0.7694	0.6050	0.7810	+0.0116
751–1,000	29	0.7828	0.5668	0.7973	+0.0145
> 1,000	11	0.8160	0.4994	0.8213	+0.0053

The available development evidence therefore did not indicate an obvious monotonic performance penalty as document length increased.

This analysis has an important limitation. The examined probabilities had already been aggregated across overlapping windows, so individual false-positive or false-negative errors cannot be attributed precisely to positions near a particular window boundary.

A dedicated window-edge analysis would require preserving window-specific probability outputs before aggregation.

4-10-8 Boundary Density and SaT Under-Segmentation

Boundary density was examined as another possible factor affecting NoPnx-NP segmentation performance.

SaT performance decreased as the density of true sentence boundaries increased. The measured Spearman correlation was

$$\rho = -0.227, \quad p < 0.001. \quad (4.58)$$

This relationship indicates that documents containing more frequent sentence transitions were more difficult for the conservative SaT predictor.

As the number of true boundaries increases, a system that produces relatively few positive predictions is more likely to miss a larger share of sentence endings.

The finding therefore provides document-level evidence consistent with the under-segmentation behavior identified in the earlier false-negative analysis.

4-10-9 OCR-Related Error Risks in Wathiq

AraSeg is applied within Wathiq after OCR processing. Consequently, OCR errors can alter the lexical and structural context from which sentence boundaries are inferred.

However, the official NoPnx-NP condition should not be interpreted as a direct simulation of raw OCR output. NoPnx-NP removes punctuation and paragraph information in a controlled manner, whereas OCR may also introduce character substitutions, incorrect word splitting or concatenation, spacing errors, and text-order distortions.

The earlier Wathiq OCR analysis additionally identified low text similarity in approximately 37.4% of the analyzed documents.

This value is an OCR-pipeline statistic rather than an AraSeg sentence-boundary error rate. It therefore indicates a potential robustness challenge for the integrated pipeline but should not be used to infer AraSeg segmentation performance directly.

Accordingly, robustness to real OCR distortions should be evaluated separately from performance on the official NoPnx-NP condition.

4-10-10 Lexical and Linguistic Error Attribution

Numbers, abbreviations, headings, conjunctions, and repeated lexical patterns can create ambiguous sentence-boundary contexts.

However, the available development evidence does not provide validated independent error counts for each lexical or linguistic category. Unsupported percentages are therefore not assigned to these error types.

Targeted experiments involving Arabic *waw* and lexical association were examined during NoPnx-NP development, but neither provided sufficiently stable evidence to support a deterministic linguistic correction rule.

Lexical and linguistic patterns are therefore treated as contextual signals that may influence boundary decisions rather than as independent rules for sentence segmentation.

4-10-11 Limits of the Error Analysis

The quantitative error analysis has two principal limitations.

First, reliable genre-specific error rates could not be reconstructed because the required mapping between development document identifiers and genre labels was not available in the analyzed records. Consequently, no quantitative comparison of segmentation errors across document genres is reported.

Second, the detailed token-level quantitative analysis was conducted primarily on NoPnx-NP development data. The other AraSeg conditions use different structural information, calibration strategies, and selected modeling configurations.

The numerical error patterns reported in this section should therefore be interpreted as evidence about the analyzed NoPnx-NP development setting and should not be generalized directly to all four AraSeg conditions.

4-11 Integration with Wathiq

AraSeg was integrated into Wathiq as a sentence-level linguistic structuring stage rather than remaining only a shared-task research component.

Wathiq already extracts and persists text from uploaded Arabic documents for archival and search operations. AraSeg operates after usable normalized document text becomes available and produces an ordered sentence representation without replacing or rewriting the stored OCR text.

This separation allows Wathiq to preserve the original document-level text while maintaining an additional sentence-level representation that can support reading, search, and future NLP processing.

4-11-1 Production Integration Strategy

Although four AraSeg conditions were developed during the research phase, the initial Wathiq production integration deliberately deploys only the validated PA pipeline.

This choice reflects an engineering constraint rather than a claim that PA is universally optimal for OCR-derived documents. At the current stage, Wathiq does not automatically determine which of the four punctuation and paragraph conditions best represents a particular document.

The production design therefore begins with one validated inference pipeline while preserving software abstractions that can support additional AraSeg conditions and future condition-aware routing.

This approach separates track-specific research evaluation from the engineering task of introducing sentence segmentation into Wathiq in a controlled and extensible manner.

4-11-2 AraSeg as an Independent AI Service

AraSeg is deployed as an independent Python-based AI service rather than embedding the Transformer runtime directly inside the ASP.NET backend.

The service owns the production sentence-segmentation pipeline. It loads the validated PA model artifacts, receives normalized document text from the Wathiq backend, performs inference, and returns the resulting ordered sentence sequence.

The production model is initialized when the service starts and is reused across subsequent requests rather than being reloaded for every document.

This separation keeps machine-learning dependencies and model runtime concerns outside the main Wathiq application while providing a stable service boundary for sentence segmentation.

The deployed service uses the validated PA configuration selected during the preceding AraSeg development experiments.

4-11-3 Model-Artifact Validation

Before constructing the production inference pipeline, the AraSeg service validates the local model artifacts against predefined metadata.

The validation metadata identifies the expected model role, artifact location, architecture, inference configuration, file sizes, and cryptographic SHA-256 checksums.

The production service does not silently replace missing artifacts or download alternative checkpoints. If a required artifact is missing, modified, or incompatible with the expected configuration, pipeline initialization fails instead of loading an unverified model.

This validation step helps ensure that the model used in deployment corresponds to the intended production artifact and reduces the risk of accidentally executing a different checkpoint.

4-11-4 Communication between the Wathiq Backend and AraSeg

The ASP.NET backend acts as the application-level owner of sentence segmentation and communicates with the internal AraSeg service when segmentation is required.

For each request, the backend provides the document identifier, the stored normalized OCR text, and the configured AraSeg condition.

The service returns a structured response containing the document and pipeline identifiers, processing status, the applied AraSeg condition, the normalized source text, and the ordered sentence collection.

Each sentence preserves its sequence position together with the token range associated with that sentence.

Before accepting a successful response, the backend verifies that the normalized source text returned by AraSeg matches the text originally submitted for segmentation.

This validation preserves text ownership within Wathiq: AraSeg adds sentence boundaries to the existing normalized OCR representation but does not rewrite, correct, trim, or reconstruct its textual content.

4-11-5 Preserving the OCR Processing Pipeline

AraSeg was integrated without replacing Wathiq's existing text extraction and OCR responsibilities.

The OCR layer remains responsible for obtaining usable document text and producing the normalized representation persisted by the backend. Searchable PDF documents may use locally extractable text when available, while scanned documents continue through the OCR-based extraction path.

Sentence segmentation is performed only after the normalized document text has been successfully obtained and stored.

The processing order is therefore:

Text Extraction / OCR → OCR Text Persistence → AraSeg Sentence Segmentation.

This ordering keeps OCR extraction and sentence segmentation as separate processing stages and ensures that AraSeg operates on the persisted Wathiq text representation rather than replacing the extraction pipeline.

4-11-6 Automatic Segmentation after OCR

After the normalized document text has been successfully persisted, the Wathiq backend automatically initiates sentence segmentation for that document.

The backend retrieves the stored normalized text and submits it to the internal AraSeg service. The browser therefore does not need to resend the document text or invoke the AI service directly.

This server-side trigger ensures that automatic segmentation operates on the same persisted text representation used by the rest of Wathiq rather than on a separate client-provided copy.

When the AraSeg response is successfully validated, the resulting sentence representation is passed to the persistence stage described in the following subsection.

4-11-7 Persistent Sentence Representation

The validated AraSeg result is persisted in MongoDB separately from the primary Wathiq document record.

This separation allows sentence segmentation to maintain its own processing state without tightly coupling the main document model to a particular AI pipeline.

The persisted segmentation representation stores the associated document identifier, AraSeg condition, pipeline identity, normalized source-text snapshot, processing status, and ordered sentence collection.

Each sentence records its sequential index, exact sentence text, and the corresponding start and end token positions.

The segmentation record also maintains creation, update, and completion timestamps so that its processing lifecycle can be tracked independently from the main document workflow.

This separate representation allows Wathiq to retain the original document text while storing sentence-level structure as an additional derived artifact.

4-11-8 Failure Isolation and Retry

Sentence segmentation is treated as a recoverable processing stage rather than a prerequisite for retaining the document.

If AraSeg inference fails because of service unavailability, invalid input, model-readiness problems, or request timeout, the already persisted document text remains unchanged.

The segmentation record is updated to reflect the failed operation, while the primary Wathiq document and its extracted text remain available for normal archival use.

The ASP.NET backend exposes document-level operations for retrieving the current segmentation state, initiating segmentation when needed, and retrying a failed segmentation attempt.

Retry therefore re-executes only the sentence-segmentation stage and does not require repeating OCR or replacing the previously stored document text.

This separation isolates AI-service failures from the main document processing workflow and allows segmentation to recover independently.

4-11-9 Sentence-Aware Elasticsearch Indexing

After successful sentence segmentation, the completed sentence texts are incorporated into the Elasticsearch representation of the document.

The document remains Wathiq's principal searchable unit; AraSeg does not replace the existing document-level search model. Instead, the segmented sentences provide an additional analyzed representation that can improve search-result highlighting.

When a query matches text contained within a stored sentence, Wathiq can prefer that complete sentence as the displayed search snippet.

This allows search results to expose a coherent linguistic unit rather than an arbitrary fragment extracted from a longer OCR text block.

4-11-10 Segmented-Text User Interface

The Wathiq document view includes a dedicated interface for presenting the current AraSeg segmentation state and the resulting sentence sequence.

When no segmentation result is available, an authorized user can initiate the operation. During processing, the interface displays the current state and periodically retrieves updated segmentation status from the Wathiq backend.

After successful completion, the segmented-text view presents the ordered sentence sequence together with the applied AraSeg condition, the number of extracted sentences, and selected pipeline metadata for traceability.

The interface also provides copy and plain-text download operations for the completed sentence representation.

Sentence text is rendered without client-side punctuation reconstruction or content rewriting, preserving the text returned by the backend.

If processing fails, the interface presents a controlled failure state and exposes the available retry action to authorized users.

4-11-11 Integration Architecture

The complete AraSeg production integration within Wathiq is summarized by the following end-to-end architecture:

User uploads an Arabic document



Wathiq text-extraction / OCR layer



Normalized document text persisted by the backend



ASP.NET segmentation application layer



Internal AraSeg AI service



Validated PA model inference



Ordered sentence sequence



MongoDB segmentation persistence



Elasticsearch sentence-aware indexing

4-11-12 Validation of the Integrated Components

The AraSeg integration was validated across the Python service, ASP.NET backend, and frontend layers.

The Python service tests covered deterministic local model loading, source-text preservation, sentence ordering, controlled failure responses, reuse of the initialized production model, and isolation of the internal inference path.

Backend validation covered request and response mapping, timeout and invalid-input handling, model-readiness failures, and preservation of the normalized document text exchanged with AraSeg.

The frontend integration was checked through type checking, production build validation, linting, and manual workflow verification.

At the completion of the integration, the frontend production build completed without TypeScript errors, the backend regression build completed without source-code errors, and the complete Python AraSeg test suite contained 273 passing tests.

4-12 Limitations and Discussion

The AraSeg results should be interpreted within the constraints of the available data, experimental protocol, reproducibility of the recorded configurations, and the scope of the initial Wathiq deployment.

This section discusses these limitations and interprets the broader experimental findings without repeating the detailed track-specific results reported earlier.

4-12-1 Training-Data Scale

The official AraSeg training data are relatively limited at the document level, even though each document contributes many token-level training examples.

This limits the diversity of discourse structures, templates, and lexical contexts that can be observed during supervised training.

The limitation is particularly relevant to methods that depend on sparse lexical patterns. A token or expression that appears frequently near sentence boundaries in the training data may reflect a document-specific, genre-specific, or template-specific pattern rather than a broadly generalizable linguistic signal.

For this reason, highly specific lexical rules and direct token-identity features were treated cautiously during the later experiments.

Larger document-level training collections would make it possible to evaluate richer contextual and meta-classification approaches with lower risk of overfitting to recurring patterns in the available training data.

4-12-2 Development-Based Model Selection

A substantial part of the experimental decision-making relied on the same labeled development collection.

Repeated comparison of alternative models, calibration choices, and decision strategies on a fixed development set can gradually increase the risk of adapting experimental decisions to characteristics of that set rather than to patterns that generalize more broadly.

Document-level held-out comparisons were introduced during the later experiments to provide an additional stability check and reduce this risk.

However, internal held-out evaluation does not replace validation on a fully independent labeled corpus. The absence of such an additional corpus therefore remains a limitation of the present study.

4-12-3 Track-Specific Models Rather than a Universal System

The four AraSeg conditions were optimized independently rather than using one fixed model and inference configuration across all tracks.

This allowed each system to exploit the structural information available in its corresponding condition, but it also limits the interpretation of cross-condition performance differences.

Differences between the selected track results reflect not only the presence or absence of punctuation and paragraph information, but also changes in model composition, calibration, contextual configuration, and structural decision rules.

The comparative results should therefore be interpreted as practical system-level differences between the four conditions rather than as controlled causal estimates of the contribution of any single structural feature.

A stricter causal analysis would require keeping the model architecture, training procedure, and inference configuration fixed while varying only the structural information supplied to the system.

4-12-4 Historical Performance and Experimental Reproducibility

The experimental history includes configurations whose recorded evaluation performance was stronger than that of the later system selected for reproducible reporting.

In such cases, the higher historical result was retained as part of the experimental record, but it was not treated as the principal final configuration when the corresponding inference procedure could not be reconstructed with sufficient confidence.

This limitation highlights the distinction between numerical performance and experimental reproducibility.

A result that cannot be associated reliably with its model composition, preprocessing, calibration, and inference behavior is difficult to analyze, reproduce, or deploy consistently.

For this reason, reproducibility was treated as part of final system selection rather than only as documentation performed after the experiments.

4-12-5 OCR Dependency in Wathiq

Because sentence segmentation operates on extracted document text, end-to-end Wathiq performance depends on both the quality of text extraction and the quality of boundary prediction.

These represent two distinct evaluation questions:

1. how accurately was the document text extracted; and
2. given that extracted text, how accurately were sentence boundaries identified?

The present AraSeg evaluation primarily addresses the second question. A complete end-to-end evaluation of Wathiq should therefore measure OCR quality and sentence-segmentation quality separately while also examining how extraction errors propagate into downstream boundary decisions.

4-12-6 Computational Complexity

The selected AraSeg systems differ in inference complexity because some conditions rely on multiple probability streams and, in the most complex case, more than one pretrained encoder family.

Such multi-model inference can require greater memory, model-loading resources, and runtime effort than a simpler single-pipeline system.

However, the present study did not perform a controlled benchmark of inference latency, memory consumption, throughput, or hardware cost across all selected AraSeg configurations.

The experimental comparison should therefore be interpreted primarily as a comparison of segmentation quality rather than deployment efficiency.

A complete production comparison would require evaluating accuracy and computational cost under the same hardware and runtime conditions.

4-12-7 Discussion of the Main Experimental Findings

Several broader conclusions emerge from the AraSeg experiments when the four structural conditions and their development trajectories are considered together.

4-12-7-1 Structural information is associated with easier segmentation

The comparative evaluation showed that conditions retaining more explicit structural information generally achieved stronger segmentation performance than conditions in which punctuation or paragraph information was removed.

However, because each track was optimized independently, these differences should not be interpreted as controlled causal estimates of the contribution of punctuation or paragraph information alone.

The results instead indicate that explicit structural cues provide useful evidence that can be exploited by appropriately designed sentence-segmentation systems.

4-12-7-2 Calibration is an important part of model performance

Several experiments showed that the quality of the final boundary decisions depended not only on the underlying neural predictions but also on how those probabilities were interpreted.

Threshold calibration and track-specific decision rules could improve the use of an existing probability distribution without requiring a new encoder architecture.

This distinction is important because model quality and decision calibration represent related but separate parts of the segmentation pipeline.

4-12-7-3 Model diversity is useful when it provides complementary evidence

The experiments also showed that increasing the number of models does not automatically improve generalization.

Additional same-family checkpoints, larger probability ensembles, and more elaborate decision strategies frequently failed to produce stable held-out improvements.

The strongest evidence for useful ensembling appeared when an additional model contributed genuinely different contextual representations rather than merely increasing the number of similar predictors.

Model diversity should therefore be evaluated through stable held-out evidence rather than assumed to be beneficial from architectural complexity alone.

4-12-7-4 Experimental complexity requires evidence

A wide range of alternative modeling, calibration, transfer, post-processing, and meta-level strategies was explored during the AraSeg study.

Many technically more complex alternatives did not provide sufficient evidence to replace simpler verified configurations.

The experiments therefore support a conservative model-selection principle: additional complexity should be retained only when it provides reproducible evidence of improved generalization.

4-13 Chapter Summary

This chapter presented the development, evaluation, analysis, and Wathiq integration of AraSeg for Arabic sentence segmentation.

The experimental study examined sentence segmentation under four structural conditions with different levels of punctuation and paragraph information. Across these conditions, the experiments showed the importance of structural evidence, probability calibration, reproducible model selection, and genuinely complementary model representations. Detailed track-specific results and the final cross-condition comparison were reported in the preceding sections.

The chapter also analyzed representative segmentation errors and identified limitations related to data scale, development-based model selection, reproducibility, OCR dependency, and computational complexity.

Finally, AraSeg was integrated into Wathiq as an independent sentence-segmentation service that operates on persisted document text and produces an ordered sentence-level representation for storage, search, and presentation.

The resulting integration extends Wathiq with sentence-level structural information while preserving the responsibilities of the existing text extraction and document-management pipeline.

Chapter 5

Conclusions and Future Work

5-1 Summary of the Current Work

This report presented the second-stage research extension of Wathiq, an intelligent document management and archiving system originally developed to transform scanned documents into searchable digital content. While the previous project phase established document upload, storage, access control, OCR, metadata management, and search, the current work extended this foundation toward structural and semantic Arabic text understanding.

Two complementary research tasks were investigated. AraGenre addressed hierarchical Arabic text genre classification by assigning each input one broad genre and one specific genre, while AraSeg addressed Arabic sentence boundary detection.

Together, the two tasks provide a research foundation for extending Wathiq beyond text extraction and keyword-based retrieval.

5-2 Main Research Findings

5-2-1 AraGenre Findings

The AraGenre experiments showed that genre classification cannot be treated as ordinary topic classification. Texts may discuss similar subjects while serving different communicative purposes, and the hierarchical label structure requires broad and specific predictions to remain consistent.

The early experiments achieved strong development performance but generalized poorly to the much larger hidden final-test distribution. This gap demonstrated that development results alone were not sufficient for selecting a robust final system. It also revealed that broad-level errors propagate directly to the specific stage when hard hierarchical gating is used.

The selected system addressed these problems through a broad-first, class-balanced hybrid architecture. It combined multilingual definition representations, Arabic semantic prototypes, and BM25 lexical evidence. Class-balanced aggregation reduced prototype-pool bias at the broad level, while weighted reciprocal rank fusion combined the specific-level rankings. Selective Qwen reranking was applied only to uncertain broad predictions.

The selected final system achieved:

$$H\text{-}MF1 = 0.455939, \quad \textit{Broad Macro F1} = 0.624961, \quad \textit{Specific Macro F1} = 0.286916. \quad (5.1)$$

Broad Accuracy reached 0.718611, while Specific Accuracy reached 0.327578. These results substantially improved upon the first final-test submission and reduced the early class-collapse behavior.

The remaining difference between broad and specific performance shows that fine-grained genre discrimination remains the principal challenge. The main AraGenre finding is that robust

hierarchical classification depends not only on the selected models, but also on balancing class evidence, preserving hierarchy, and combining complementary semantic and lexical signals.

5-2-2 AraSeg Findings

The AraSeg study showed that Arabic sentence segmentation is strongly influenced by the structural information available in the input.

Across the four evaluated conditions, the experiments demonstrated that successful boundary prediction depends not only on the underlying neural model but also on reliable token alignment, calibrated inference, appropriate use of available document structure, and evidence of genuine model complementarity.

5-3 Contributions to Wathiq

The original Wathiq system focused on digitizing, storing, and searching documents. OCR made document content electronically accessible, but the extracted text remained largely unstructured and did not explicitly describe the communicative genre of the content.

The current research extension contributes two additional forms of document representation:

- **Structural representation:** AraSeg adds sentence-level structure to the extracted document text.
- **Semantic representation:** AraGenre assigns broad and specific genre metadata that can support document organization and filtering.

Within the extended Wathiq workflow, these representations can support richer document organization, retrieval, filtering, traceability, and future human-review workflows.

5-4 Research and Software Engineering Contributions

The principal research contributions of the work are:

- analyzing the generalization gap between AraGenre development data and the hidden final-test distribution;
- developing a class-balanced, broad-first retrieval strategy for hierarchical Arabic genre classification;
- combining multilingual definitions, Arabic semantic prototypes, and lexical retrieval through rank-based fusion;
- providing a systematic experimental study of Arabic sentence segmentation under varying levels of structural information; and
- documenting successful and unsuccessful experimental directions to support reproducibility and evidence-based model selection.

From a software engineering perspective, the work defined modular service boundaries for incorporating the research components into the existing Wathiq architecture without tightly coupling their model runtimes to the core application.

This design supports independent evolution of the intelligent components while preserving the existing document-management workflow.

5-5 Current Limitations

Several limitations should be considered when interpreting the reported results:

- The shared-task datasets do not fully represent the governmental and administrative documents expected in Wathiq.
- Hidden test labels limited final-test error analysis, particularly for AraGenre, to aggregate scores and prediction distributions.
- AraGenre specific-level performance remained substantially lower than broad-level performance because of semantic overlap among fine-grained genres.
- Hard hierarchical gating guarantees valid predictions but may remove the correct specific genre after an incorrect broad decision.
- The generated Arabic prototypes improve genre coverage but may not reflect every form of naturally occurring Arabic text.
- Retrieval scores, fusion scores, and model probabilities require operational calibration before being used as confidence values for automatic acceptance or human review.
- End-to-end operational validation within the Wathiq environment remains incomplete.

These limitations define the boundary between the validated research results and the additional evaluation required for production use.

5-6 Future Work

5-6-1 Evaluation on Wathiq Documents

The most important next step is to construct a representative evaluation set from documents processed by Wathiq. It should include different institutions, document types, scanning qualities, OCR-error levels, and text lengths.

Domain experts should annotate sentence boundaries and appropriate genre labels where the AraGenre taxonomy is applicable. This would measure the extent to which the shared-task results transfer to the intended document-management environment.

5-6-2 OCR-Aware Processing

For AraGenre, future work may investigate noise-aware prototypes, OCR-based augmentation, and soft hierarchical gating.

For AraSeg, future work should investigate how OCR confidence and available layout information can support more robust boundary prediction and condition-aware routing among the available segmentation strategies.

5-6-3 Confidence-Based Human Review

Operational confidence thresholds should be calibrated using representative Wathiq documents rather than shared-task results alone. AraGenre confidence may incorporate agreement among retrieval channels and the margin between competing genres. AraSeg confidence may incorporate boundary probabilities, model disagreement, and structural inconsistencies.

5-6-4 End-to-End Evaluation

The complete extended workflow should be evaluated from document upload through OCR extraction, sentence segmentation, genre classification, metadata persistence, search indexing, and user review.

The evaluation should measure processing latency, service failure behavior, data consistency, prediction traceability, correction persistence, and the practical effect of the generated linguistic metadata on search and document-management tasks.

Such evaluation is necessary before the shared-task results can be generalized to the full range of documents processed by Wathiq.

5-7 Final Conclusion

This work extended Wathiq from Arabic document digitization toward structural and semantic document understanding. AraGenre introduced hierarchical genre classification, while AraSeg introduced sentence-level structural representation.

The AraGenre results showed the importance of class-balanced evidence, hierarchical decomposition, and complementary semantic and lexical retrieval. The AraSeg study established a reproducible approach to sentence-level structuring under different input conditions.

A common conclusion emerged from both tasks: increasing model complexity alone does not guarantee robust improvement. Effective Arabic text-processing systems must reflect the structure of the problem, control the evidence used during prediction, preserve reproducibility, and expose uncertain outputs for verification.

The resulting components provide Wathiq with a validated research foundation for richer document organization and sentence-aware text processing. Future domain-specific and end-to-end evaluation will determine how these capabilities should be calibrated and deployed within real document-management workflows.

Bibliography

- [1] AraGenre 2026 Organizers, *AraGenre 2026 Shared Task: Official Starter Kit, Genre Definitions, Baselines, and Evaluation Materials*, Codabench, 2026.
- [2] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 3982–3992, 2019. <https://aclanthology.org/D19-1410/>
- [3] Sentence-Transformers, *paraphrase-multilingual-MiniLM-L12-v2 Model Card*, Hugging Face. <https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2> Accessed: August 2026.
- [4] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, “Multilingual E5 Text Embeddings: A Technical Report,” *arXiv preprint arXiv:2402.05672*, 2024. <https://arxiv.org/abs/2402.05672>
- [5] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009. <https://doi.org/10.1561/15000000019>
- [6] S. A. Dudani, “The Distance-Weighted k -Nearest-Neighbor Rule,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-6, no. 4, pp. 325–327, 1976. <https://doi.org/10.1109/TSMC.1976.5408784>
- [7] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods,” in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 758–759, 2009. <https://doi.org/10.1145/1571941.1572114>
- [8] Qwen Team, “Qwen2.5 Technical Report,” *arXiv preprint arXiv:2412.15115*, 2024. <https://arxiv.org/abs/2412.15115>
- [9] A. Yang et al., “Qwen3 Technical Report,” *arXiv preprint arXiv:2505.09388*, 2025. <https://arxiv.org/abs/2505.09388>
- [10] O. Nacar and A. Koubaa, “Enhancing Semantic Similarity Understanding in Arabic NLP with Nested Embedding Learning,” *arXiv preprint arXiv:2407.21139*, 2024. <https://arxiv.org/abs/2407.21139>
 Omartificial Intelligence Space, *Arabic-Triplet-Matryoshka-V2 Model Card*, Hugging Face. <https://huggingface.co/Omartificial-Intelligence-Space/Arabic-Triplet-Matryoshka-V2> Accessed: August 2026.
- [11] OpenAI, *ChatGPT*, 2026. <https://chatgpt.com/> Accessed: July 2026.

- [12] Z. Xu et al., “Codabench: Flexible, Easy-to-Use, and Reproducible Meta-Benchmark Platform,” *Patterns*, vol. 3, no. 7, 2022. <https://doi.org/10.1016/j.patter.2022.100543>
- [13] T. Kuzman and N. Ljubešić, “Automatic Genre Identification: A Survey,” *Language Resources and Evaluation*, 2023. <https://doi.org/10.1007/s10579-023-09695-8>
- [14] J. Piskorski, N. Stefanovitch, G. Da San Martino, and P. Nakov, “SemEval-2023 Task 3: Detecting the Category, the Framing, and the Persuasion Techniques in Online News in a Multi-lingual Setup,” in *Proceedings of the 17th International Workshop on Semantic Evaluation (SemEval-2023)*, 2023. <https://aclanthology.org/2023.semeval-1.317/>
- [15] B. Wu, O. Razuvayevskaya, F. Heppell, J. A. Leite, C. Scarton, K. Bontcheva, and X. Song, “SheffieldVeraAI at SemEval-2023 Task 3: Mono and Multilingual Approaches for News Genre, Topic and Persuasion Technique Classification,” in *Proceedings of the 17th International Workshop on Semantic Evaluation (SemEval-2023)*, 2023. <https://aclanthology.org/2023.semeval-1.275/>
- [16] M. Hasanain, A. O. El-Shangiti, R. N. Nandi, P. Nakov, and F. Alam, “QCRI at SemEval-2023 Task 3: News Genre, Framing and Persuasion Techniques Detection Using Multilingual Models,” in *Proceedings of the 17th International Workshop on Semantic Evaluation (SemEval-2023)*, 2023. <https://aclanthology.org/2023.semeval-1.172/>
- [17] M. Hasanain, F. Alam, H. Mubarak, S. Abdaljalil, W. Zaghoulani, P. Nakov, G. Da San Martino, and A. A. Freihat, “ArAIEval Shared Task: Persuasion Techniques and Disinformation Detection in Arabic Text,” in *Proceedings of ArabicNLP 2023*, 2023. <https://aclanthology.org/2023.arabicnlp-1.44/>
- [18] L. Gao, D. Ghosh, and K. Gimpel, “The Benefits of Label-Description Training for Zero-Shot Text Classification,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. <https://aclanthology.org/2023.emnlp-main.853/>
- [19] G. Yu, L. Liu, H. Jiang, S. Shi, and X. Ao, “Retrieval-Augmented Few-Shot Text Classification,” in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023. <https://aclanthology.org/2023.findings-emnlp.447/>
- [20] C. N. Silla Jr. and A. A. Freitas, “A Survey of Hierarchical Classification Across Different Application Domains,” *Data Mining and Knowledge Discovery*, vol. 22, no. 1–2, pp. 31–72, 2011.
- [21] R. Plaud, M. Labeau, A. Saillenfest, and T. Bonald, “Revisiting Hierarchical Text Classification: Inference and Metrics,” in *Proceedings of the 28th Conference on Computational Natural Language Learning*, pp. 231–242, 2024.

- [22] N. Japkowicz and S. Stephen, “The Class Imbalance Problem: A Systematic Study,” *Intelligent Data Analysis*, vol. 6, no. 5, pp. 429–450, 2002.
- [23] R. C. Prati, G. E. A. P. A. Batista, and M. C. Monard, “Class Imbalances versus Class Overlapping: An Analysis of a Learning System Behavior,” in *MICAI 2004: Advances in Artificial Intelligence*, vol. 2972, pp. 312–321, Springer, 2004.
- [24] M. Elkholy, K. N. Elmadani, N. Habash, and B. Alhafni, “Arabic Sentence Segmentation Across Genres and Punctuation Conditions,” *arXiv preprint arXiv:2606.08025*, 2026. <https://arxiv.org/abs/2606.08025>
- [25] B. Minixhofer, J. Pfeiffer, and I. Vulić, “Where’s the Point? Self-Supervised Multilingual Punctuation-Agnostic Sentence Segmentation,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 7215–7235, 2023. <https://aclanthology.org/2023.acl-long.398/>
- [26] M. Frohmann, I. Sterner, I. Vulić, B. Minixhofer, and M. Schedl, “Segment Any Text: A Universal Approach for Robust, Efficient and Adaptable Sentence Segmentation,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 11908–11941, 2024. <https://aclanthology.org/2024.emnlp-main.665/>
- [27] G. Inoue, B. Alhafni, N. Baimukan, H. Bouamor, and N. Habash, “The Interplay of Variant, Size, and Task Type in Arabic Pre-trained Language Models,” in *Proceedings of the Sixth Arabic Natural Language Processing Workshop*, pp. 92–104, 2021. <https://aclanthology.org/2021.wanlp-1.10/>
- [28] W. Antoun, F. Baly, and H. Hajj, “AraELECTRA: Pre-Training Text Discriminators for Arabic Language Understanding,” in *Proceedings of the Sixth Arabic Natural Language Processing Workshop*, pp. 191–195, 2021. <https://aclanthology.org/2021.wanlp-1.20/>